Special Functions of Mathematical Physics and the Numerov Method

Laguerre, Legendre, cylindrical and spherical Bessel functions — theory, computation, and the Schrödinger equation. A PhD-level study course with Python 3 and Fortran 2008 codes, verification, error analysis and exercises

November 2026

Table of Contents

How to use this course 4

Part 0 — Common foundations 5

What a “special function” is 5

Sturm–Liouville structure 5

Liouville normal form — the bridge to Numerov 5

Three-term recurrences and their stability 6

Gaussian quadrature 7

Numerov near a regular singular point: the Frobenius start and the start fault 8

Verification and error-analysis toolkit 9

Chapter 1 — Laguerre polynomials and the hydrogen atom 10

1.1 The function 10

1.2 The physics: hydrogen atom 10

1.3 Analytical scheme 11

1.4 Logic chart 12

1.5 Python 3 code 12

1.6 Fortran 2008 code 16

1.7 Results, normalisation and verification 20

1.8 Error analysis and convergence 21

1.9 Exercises 22

Chapter 2 — Legendre functions and the angular Schrödinger equation 24

2.1 The function 24

2.2 The physics: the angular equation 25

2.3 Analytical scheme 25

2.4 Logic chart 26

2.5 Python 3 code 26

2.6 Fortran 2008 code 30

2.7 Results, normalisation and verification 33

2.8 Error analysis and convergence 34

2.9 Exercises 35

Chapter 3 — Cylindrical Bessel functions and the circular well 37

3.1 The function 37

3.2 The physics: circular infinite well 37

3.3 Analytical scheme 38

3.4 Logic chart 39

3.5 Python 3 code 39

3.6 Fortran 2008 code 43
3.7 Results, normalisation and verification 46
3.8 Error analysis and convergence 47
3.9 Exercises 48
Chapter 4 — Spherical Bessel functions and the spherical well 49
4.1 The function 49
4.2 The physics: spherical infinite well 49
4.3 Analytical scheme 50
4.4 Logic chart 50
4.5 Python 3 code 50
4.6 Fortran 2008 code 54
4.7 Results, normalisation and verification 57
4.8 Error analysis and convergence 58
4.9 Exercises 59
Chapter 5 — Cross-check of the Fortran and Python implementations 61
Summary 62
References 63

How to use this course

This course is the sequel to *The Numerov Method for the One-Dimensional Schrödinger Equation*. It treats the four families of special functions that a physicist meets first when the Schrödinger equation is separated in three dimensions:

Chapter	Special function	Where it appears	Schrödinger problem solved with Numerov
1	Generalised Laguerre polynomials $L_n^{(\alpha)}$	radial Coulomb problem	hydrogen atom
2	Legendre polynomials and associated functions P_l^m	polar-angle equation	rigid rotor / angular equation
3	Cylindrical Bessel functions J_m	radial equation in polar coordinates	particle in a circular (cylindrical) well
4	Spherical Bessel functions j_l	radial equation for $V = 0$	particle in a spherical well

Every chapter has the same structure, which mirrors the first course:

1. **The function**: definition, differential equation, the main formulas, with references to the NIST *Digital Library of Mathematical Functions* (DLMF) and to standard textbooks.
2. **The physics**: how the function arises from the Schrödinger equation, and the exact solution.
3. **Analytical scheme**: the formulas the program uses, written *before* the code — Part A (computing the function itself) and Part B (solving the Schrödinger equation with Numerov).
4. **Logic charts.**
5. **Codes**: a Python 3 program and a Fortran 2008 program implementing the same algorithms. They agree to all printed digits.
6. **Results, normalisation and verification.**
7. **Error analysis and convergence plots.**
8. **Worked examples and exercises with answers.**

Units: $\hbar = m = 1$ throughout (and atomic units, $e = 4\pi\varepsilon_0 = 1$, for hydrogen).

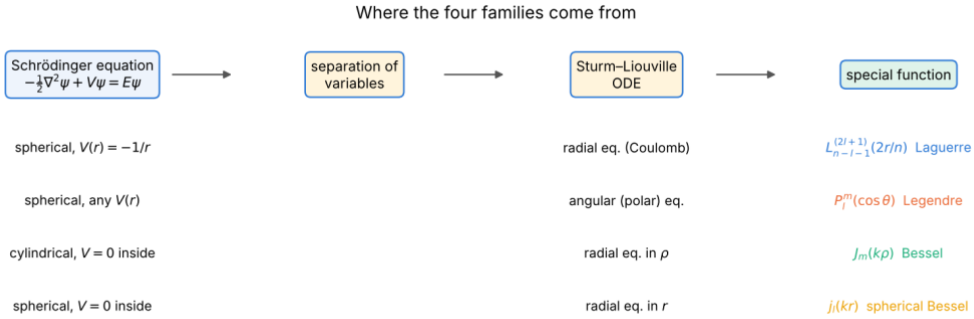
Software. Python 3 with NumPy, SciPy, Matplotlib and mpmath (used only as a 50-digit *reference*, never inside the algorithms); gfortran (Fortran 2008 is needed for the intrinsics `log_gamma`, `gamma` and `bessel_jn`).

```
python3 ch1_laguerre_hydrogen.py        # likewise ch2_..., ch3_..., ch4_...
gfortran -O2 -o ch1 ch1_laguerre_hydrogen.f90 && ./ch1    # likewise ch2 ... ch4
python3 plot_fortran_results.py         # plots the .dat files written by the Fortran codes
```

Part 0 — Common foundations

What a “special function” is

Special functions are functions that have established names and notations because of their importance in analysis, physics and engineering; the term is defined by consensus rather than formally. In the narrow sense used here they are the *special functions of mathematical physics*: solutions of the ordinary differential equations obtained when a partial differential equation (Laplace, Helmholtz, Schrödinger, wave, heat) is solved by separation of variables [EoM; DLMF Ch. 10, 14, 18; Lebedev 1965]. The standard modern reference is the DLMF, the online version of the *NIST Handbook of Mathematical Functions* (Olver et al. 2010), which continues Abramowitz & Stegun (1964).



Separation of variables leads from the Schrödinger equation to the four families of this course.

Sturm–Liouville structure

All four ordinary differential equations are of Sturm–Liouville form

$$-(p(x)\,y')' + q(x)\,y = \lambda\, w(x)\,y,$$

with a positive weight w. On an interval with suitable boundary conditions the eigenfunctions are **orthogonal** with weight w, the eigenvalues are real and simple, and the n-th eigenfunction has n nodes (Sturm oscillation theorem). Orthogonality is therefore a built-in verification tool for every numerical solution in this course.

Family	$p(x)$	$w(x)$	**interval**	**singular points**
Laguerre $L_n^{(\alpha)}$	$x^{\alpha+1}e^{-x}$	$x^{\alpha}e^{-x}$	$[0,\infty)$	regular at 0, irregular at ∞
Legendre P_l^m	$1-x^2$	1	$[-1,1]$	regular at ± 1
Bessel $J_m(kr)$	r	r	$[0,a]$	regular at 0
spherical Bessel $j_l(kr)$	r^2	r^2	$[0,a]$	regular at 0

Liouville normal form — the bridge to Numerov

Numerov’s method (first course, Part I) integrates equations of the form $u'' = -k^2(x)\,u$ without a first-derivative term. A Sturm–Liouville equation is brought to this form by a change of the dependent variable; in all our cases it is a multiplication by a power of the coordinate:

Equation	**Substitution**	**Normal form**
hydrogen, radial	$u = rR$	$u'' = [l(l+1)/r^2 - 2/r - 2E]u$

Equation	Substitution	Normal form
polar angle	$u = \sqrt{\sin\theta}\,\Theta$	$u'' = -[\lambda + 1/4 + 1/4 - m^2/\sin^2\theta]u$
cylindrical radial	$u = \sqrt{r}\,R$	$u'' = -[2E - m^2 - 1/4/r^2]u$
spherical radial	$u = rR$	$u'' = -[2E - l(l+1)/r^2]u$

The two "square-root" transformations were verified symbolically (sympy residual = 0). They are exact, but they have a hidden cost discussed below: they make the solution **non-analytic** at the singular point.

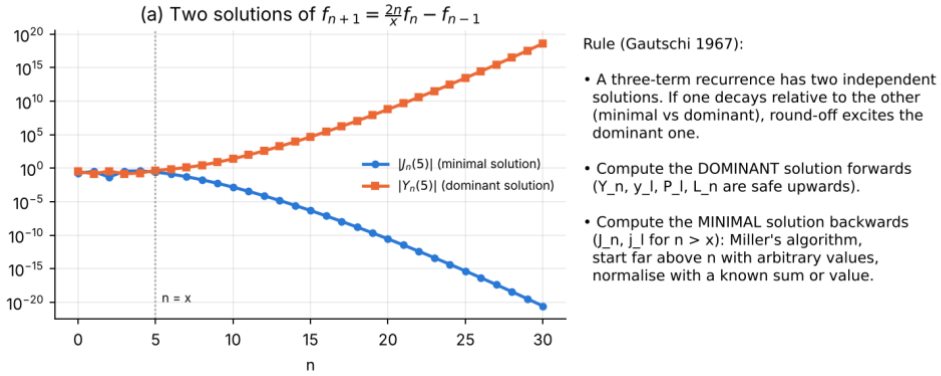
Three-term recurrences and their stability

Every family in this course satisfies a three-term recurrence in its index, for example

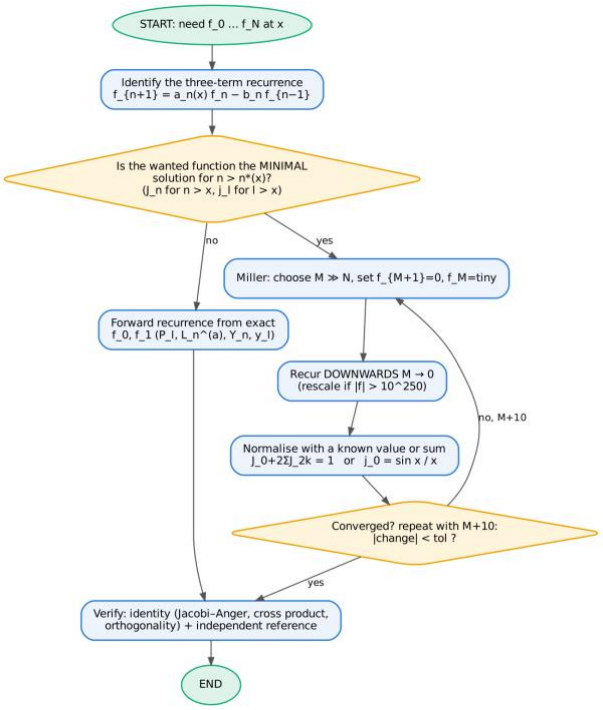
$$J_{n+1}(x) = \frac{2n}{x}J_n(x) - J_{n-1}(x), \qquad (l+1)P_{l+1} = (2l+1)xP_l - lP_{l-1}.$$

Recurrences are the fastest and usually the most accurate way to compute special functions — *provided they are run in the right direction*. A second-order difference equation has two independent solutions f_n, g_n. If $f_n/g_n \to 0$ as $n \to \infty$, f is called **minimal** and g **dominant**. Round-off errors excite the dominant solution, so (Gautschi 1967; DLMF §3.6):

- a **dominant** solution can be computed **forwards** (Legendre P_l, Laguerre L_n, Y_n, y_l);
- a **minimal** solution must be computed **backwards**. $J_n(x)$ and $j_l(x)$ are minimal once $n > x$. **Miller's algorithm** (Miller 1952; error analysis Olver 1964) starts far above the wanted index with arbitrary values, recurs downwards (the wanted solution grows in that direction), and fixes the scale with a known value or sum.



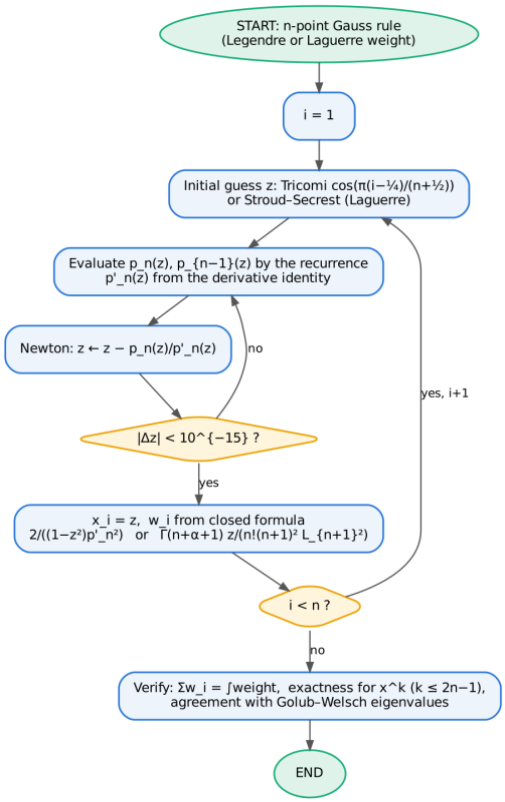
Minimal and dominant solutions of the Bessel recurrence, and the resulting rule.



Logic chart: choosing the direction of a recurrence; Miller's algorithm.

Gaussian quadrature

The zeros of orthogonal polynomials are the nodes of Gaussian quadrature: an n-point rule $\int w(x)f(x)\,dx \approx \sum_i w_i\, f(x_i)$ is exact for polynomials of degree $\leq 2n-1$ (DLMF §3.5(v)). Nodes are found either by Newton's method on p_n with good initial guesses, or as eigenvalues of the symmetric tridiagonal Jacobi matrix built from the recurrence coefficients (Golub & Welsch 1969). We use Newton (in both languages) and Golub–Welsch (Python) as an independent check. Gauss rules converge **exponentially** for analytic integrands — a property we use both as a verification and as an example of error analysis.



Logic chart of the Gauss-rule generator.

Numerov near a regular singular point: the Frobenius start and the start fault

Near $r = 0$ the solutions of the normal-form equations behave like r^s, where s solves the indicial equation:

Problem	regular solution	irregular solution	u analytic at 0?
hydrogen / spherical well	r^{l+1}	r^{-l}	**yes**
circular well (index m)	$r^{m+1/2}$	$r^{-m+1/2}$ (or $r^{1/2}\ln r$ for $m = 0$)	**no**
polar angle (index m)	$\theta^{m+1/2}$	$\theta^{-m+1/2}$ (log for $m = 0$)	**no**

Frobenius start. Inserting $u = r^s \sum_k a_k r^k$ into the equation gives a recurrence for the a_k; with 25–40 terms the series is accurate to machine precision for small r. The series values provide the starting values that Numerov needs (fault no. 2 of the first course).

The start fault. A useful way to understand what goes wrong near $r = 0$ is the following argument. Suppose that near $r \approx h$ the computation makes a *relative* error δ. Such a local error adds a small multiple of the **irregular** solution to the computed one, with amplitude

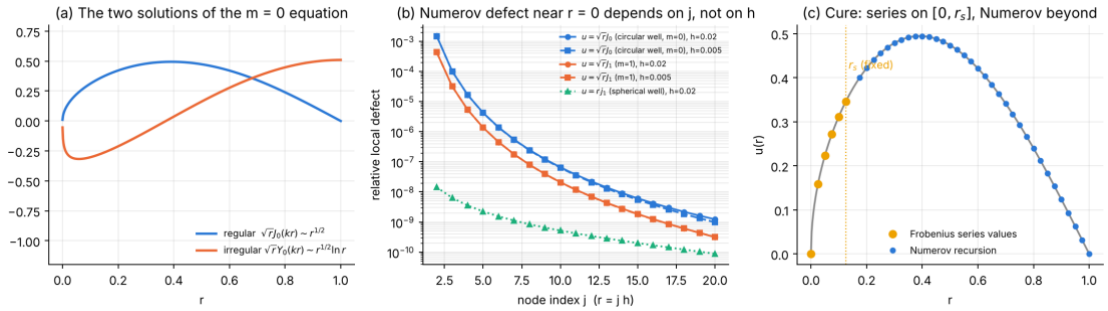
$$\alpha \sim \delta \frac{u_{\text{reg}}(h)}{u_{\text{irr}}(h)} = \delta\, h^{s_{\text{reg}} - s_{\text{irr}}},$$

and the error in the eigenvalue is proportional to α. Two mechanisms produce an h-independent δ:

1. **Non-analytic solution, start at the first grid nodes.** For $u \propto r^{m+1/2}$, the Numerov stencil centred at node j has a relative defect that depends on j but **not on** h (figure below, panel b), because $u^{(6)} \propto r^{m-11/2}$. Hence $\delta = O(1)$ and the eigenvalue error is $O(h^{2m})$: **no convergence for** $m = 0$**, second order for** $m = 1$**, fourth order only for** $m \geq 2$**.**

2. **Analytic solution, "naive" textbook start** $u_0 = 0,\ u_1 = h^{l+1}$ with $c_0 u_0$ taken as 0. The correct value of $c_0 u_0 = \lim_{r\to 0}(u + h^2/12\, k^2 u)$ is *not* zero when $k^2 u$ has a finite limit at $r = 0$ — this happens for the Coulomb term with $l = 0$ and for the centrifugal term with $l = 1$. The resulting defect gives order h^2 (hydrogen s states) and h^3 (p states).

The cure is simple: evaluate the Frobenius series on $[0, r_s]$ for a *fixed* r_s (independent of h; we use $r_s = 1/8$, $\theta_s = 0.4$ or $r_s = 0.25$) and start Numerov from there. The solution is then smooth on the Numerov interval and fourth order is restored in every case. This "start fault and its cure" is the central numerical lesson of this course; each chapter demonstrates it.



(a) Regular and irregular solutions for $m = 0$. (b) Relative defect of the Numerov stencil applied to the exact solution near the origin: for $\sqrt{r}J_m$ it does not decrease with h (curves for $h = 0.02$ and 0.005 coincide), for the analytic rj_1 it is tiny. (c) The cure: Frobenius series up to a fixed r_s, Numerov beyond.

Verification and error-analysis toolkit

Each program applies the following checks, which the student should regard as compulsory for any numerical work on special functions:

Check	What it tests
comparison with a 50-digit reference (mpmath)	absolute accuracy of the function values
independent identity (Jacobi–Anger, cross product, $\sum w_i$, closed forms)	correctness without any reference software
comparison with SciPy / Fortran intrinsic `bessel_jn`	independent implementation
normalisation $\int u^2 = 1$ (Simpson)	eigenfunction scale; Simpson has $O(h^4)$ error, consistent with Numerov
comparison with the exact eigenfunction built from Part A	shape of the numerical solution
orthogonality, parity, node count	Sturm–Liouville structure
expectation values ($\langle r\rangle$, $\langle r^2\rangle$, $\langle\cos^2\theta\rangle$, virial theorem)	physics content of the wavefunction
observed order p from a sequence of grids	correct implementation of a 4th-order method
Richardson extrapolation $E_R = \left(16E_{h/2} - E_h\right)/15$	consistency of the h^4 error model
round-off floor, box-size truncation	limits of attainable accuracy

Chapter 1 — Laguerre polynomials and the hydrogen atom

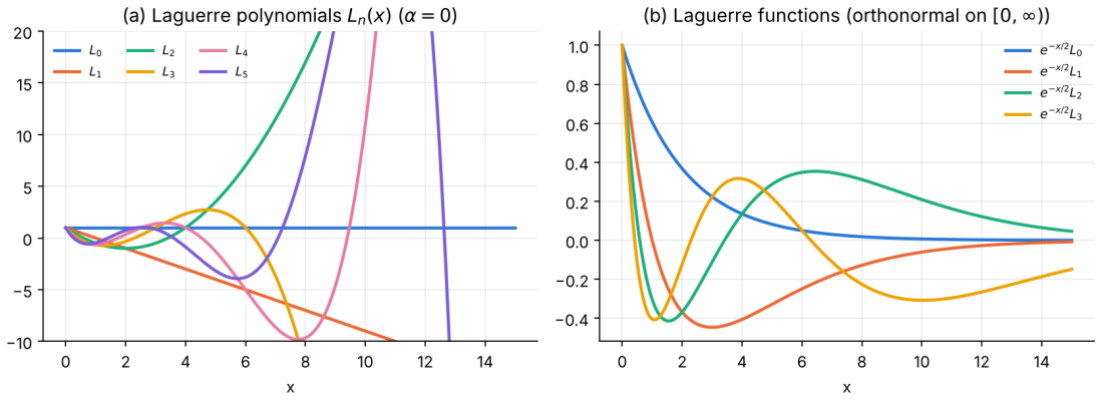
1.1 The function

The **generalised (associated) Laguerre polynomials** $L_n^{(\alpha)}(x)$, $\alpha > -1$, are the polynomial solutions of

$$x\,y'' + (\alpha + 1 - x)\,y' + n\,y = 0 \qquad \text{(DLMF §18.8).}$$

Main properties (DLMF §18.3, §18.5, §18.9; Szegő 1939, ch. 5; Arfken et al. 2013, §18.3):

- Rodrigues formula: $L_n^{(\alpha)}(x) = \frac{x^{-\alpha}e^{x}}{n!}\frac{d^n}{dx^n}(e^{-x}x^{n+\alpha})$.
- Explicit sum: $L_n^{(\alpha)}(x) = \sum_{k=0}^{n}(-1)^k \binom{n+\alpha}{n-k}\frac{x^k}{k!}$.
- Orthogonality on $[0,\infty)$: $\int_0^\infty x^\alpha\, e^{-x} L_m^{(\alpha)} L_n^{(\alpha)}\, dx = \frac{\Gamma(n+\alpha+1)}{n!}\,\delta_{mn}$.
- Three-term recurrence: $(n+1)L_{n+1}^{(\alpha)} = (2n+1+\alpha-x)L_n^{(\alpha)} - (n+\alpha)L_{n-1}^{(\alpha)}$, $L_0 = 1$, $L_1 = 1+\alpha-x$.
- Derivative: $x\,\frac{d}{dx}L_n^{(\alpha)} = nL_n^{(\alpha)} - (n+\alpha)L_{n-1}^{(\alpha)}$.
- Generating function: $\sum_n L_n^{(\alpha)}(x)t^n = (1-t)^{-\alpha-1}e^{-xt/(1-t)}$.
- Zeros: n real simple zeros in $(0, 4n+2\alpha+2)$; they are the nodes of Gauss–Laguerre quadrature.



Laguerre polynomials and the orthonormal Laguerre functions $e^{-x/2}L_n(x)$.

1.2 The physics: hydrogen atom

For $V = -Z/r$ the separation $\psi = R_{nl}(r)Y_{lm}(\theta,\phi)$, $u = rR$, gives

$$u'' = \left[\frac{l(l+1)}{r^2} - \frac{2Z}{r} - 2E\right]u, \qquad u(0) = 0,\ u(\infty) = 0.$$

Writing $u = \rho^{l+1}e^{-\rho/2}f(\rho)$, $\rho = 2Zr/n$, turns this into the Laguerre equation with $\alpha = 2l+1$; normalisability forces f to be a polynomial, which quantises the energy (Bethe & Salpeter 1957, §1–3; Landau & Lifshitz 1977, §36; Griffiths & Schroeter 2018, §4.2; DLMF §18.39):

$$E_n = -\frac{Z^2}{2n^2}, \qquad R_{nl}(r) = \sqrt{\left(\frac{2Z}{n}\right)^3 \frac{(n-l-1)!}{2n\,(n+l)!}}\; e^{-\rho/2}\rho^{l}\, L_{n-l-1}^{(2l+1)}(\rho), \quad n = l+1, l+2, \ldots$$

Exact expectation values used for verification: $\langle r\rangle = 1/2Z\,[3n^2 - l(l+1)]$, $\langle r^2\rangle = n^2/2Z^2\,[5n^2 + 1 - 3l(l+1)]$, $\langle 1/r\rangle = Z/n^2$, virial theorem $\langle V\rangle = 2E$.

1.3 Analytical scheme

Part A — computing $L_n^{(\alpha)}$.

1. *Recurrence* (stable: L_n is a dominant solution) from L_0, L_1.
2. *Explicit sum* — for comparison only: its terms alternate in sign and grow like $x^k/k!$, so for large x the sum suffers catastrophic cancellation.
3. *Gauss–Laguerre nodes*: Newton iteration $z \leftarrow z - L_n/L_n{}'$ with the Stroud–Secrest initial guesses (as in Numerical Recipes, Press et al. 2007, §4.6); weights $w_i = \frac{\Gamma(n+\alpha+1)\,x_i}{n!\,(n+1)^2\left[L_{n+1}^{(\alpha)}(x_i)\right]^2}$. Check: Golub–Welsch eigenvalues of the Jacobi matrix with diagonal $2k+1+\alpha$ and off-diagonal $-\sqrt{k(k+\alpha)}$.

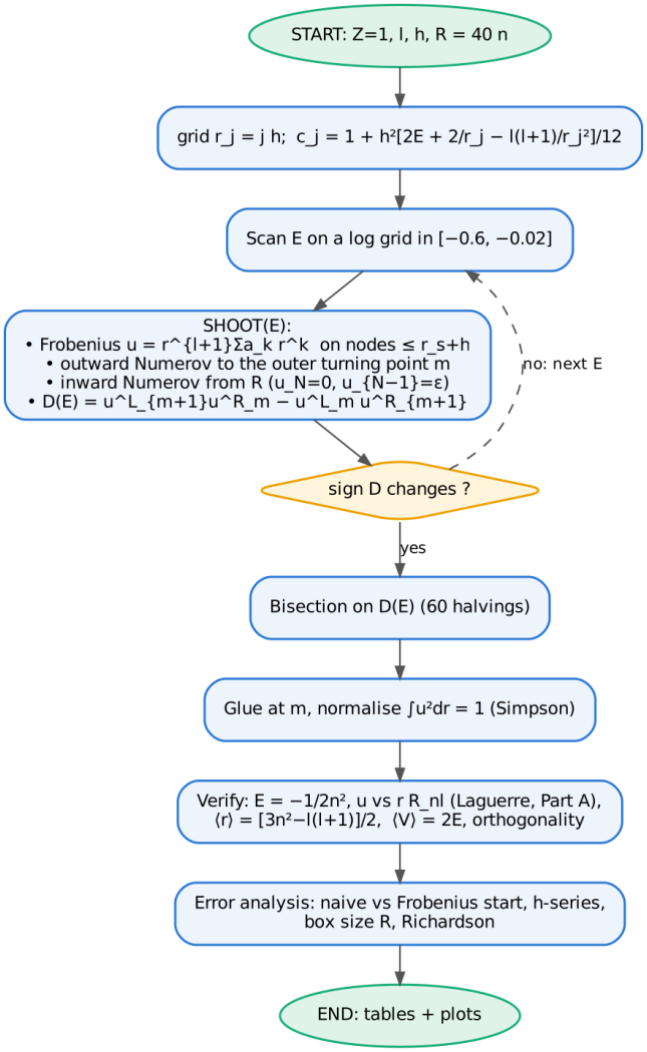
Part B — Numerov for hydrogen.

- Grid $r_j = jh$, $j = 0..N$, box $R = 40n$ (the tail is $\propto r^n e^{-r/n}$).
- $k_j^2 = 2E + 2/r_j - l(l+1)/r_j^2$, $c_j = 1 + h^2 k_j^2/12$.
- **Frobenius start**: $u = r^{l+1}\sum_k a_k\, r^k$ with $a_0 = 1$, $a_1 = -2/(2l+2)$,

  $$k(k+2l+1)\,a_k = -2a_{k-1} - 2E\,a_{k-2},$$

  evaluated on all nodes $r_j \le r_s + h$, $r_s = 0.25$. For comparison: the **naive start** $u_0 = 0$, $u_1 = h^{l+1}$.
- **Two-sided shooting** (as for the oscillator of the first course): outward Numerov up to the node m nearest the *outer* classical turning point $r_t = \left[1 + \sqrt{1 + 2El(l+1)}\right]/(-2E)$; inward Numerov from R with $u_N = 0$, $u_{N-1} = 10^{-30}$.
- Matching function: discrete Wronskian $D(E) = u_{m+1}^L u_m^R - u_m^L u_{m+1}^R$; scan E on a logarithmic grid, refine by bisection (60 halvings).
- Normalise with Simpson; compare with rR_{nl} built from the **Part A recurrence** (so Part A is verified by Part B and vice versa).

1.4 Logic chart



Logic chart of the hydrogen program (Part B). Part A follows the generic charts of Part 0.

1.5 Python 3 code

File: `ch1_laguerre_hydrogen.py`

```
#!/usr/bin/env python3
"""
Chapter 1 -- LAGUERRE POLYNOMIALS and the HYDROGEN ATOM (Numerov)
================================================================
Part A : computation of the generalised Laguerre polynomials L_n^(a)(x)
         * three-term recurrence  (stable)               DLMF §18.9
         * explicit finite sum    (cancellation!)          DLMF §18.5
         * Gauss-Laguerre nodes/weights: Newton on L_n^(a), and Golub-Welsch
         * verification: orthogonality by quadrature, mpmath reference
         * error analysis: sum vs recurrence, quadrature convergence
Part B : hydrogen atom, radial equation (hbar = m_e = e = 4 pi eps0 = 1, Z = 1)
         u'' = [ l(l+1)/r^2 - 2/r - 2E ] u ,   u = r R(r)
         exact  E_n = -1/(2n^2),   R_nl ~ e^{-r/n} (2r/n)^l L_{n-l-1}^{(2l+1)}(2r/n)
         Numerov, two-sided shooting, Frobenius start at r_s, Wronskian matching
         * verification: normalisation, comparison with the Laguerre-based
           exact R_nl (computed with Part A), <r>, virial <V> = 2E, orthogonality
         * error analysis: start at first grid nodes vs fixed r_s, h^4 law
Outputs: tables + ch1_functions.png, ch1_partA_errors.png, ch1_hydrogen.png,
         ch1_convergence.png
"""
import numpy as np
from math import lgamma, exp, factorial
import mpmath as mp
from scipy.linalg import eigh_tridiagonal
from scipy.special import eval_genlaguerre
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#7a5cd6"]
mp.mp.dps = 50


def style(ax):
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")


# ============================================================ PART A
def laguerre_rec(n, a, x):
    """L_0..L_n^(a)(x) by the three-term recurrence (returns array shape (n+1, len(x)))
       (k+1) L_{k+1} = (2k+1+a-x) L_k - (k+a) L_{k-1}"""
    x = np.asarray(x, float)
    L = np.zeros((n + 1,) + x.shape)
    L[0] = 1.0
    if n >= 1:
        L[1] = 1.0 + a - x
    for k in range(1, n):
        L[k + 1] = ((2 * k + 1 + a - x) * L[k] - (k + a) * L[k - 1]) / (k + 1)
    return L


def laguerre_sum(n, a, x):
    """explicit sum  L_n^(a)(x) = sum_k (-1)^k C(n+a, n-k) x^k / k!"""
    x = np.asarray(x, float)
    s = np.zeros_like(x)
    for k in range(n + 1):
        binom = exp(lgamma(n + a + 1) - lgamma(n - k + 1) - lgamma(a + k + 1))
        s += (-1) ** k * binom * x ** k / factorial(k)
    return s


def gauss_laguerre_newton(n, a=0.0, tol=1e-15):
    """nodes & weights of n-point Gauss-Laguerre rule: Newton on L_n^(a)
       with the Stroud-Secrest / Numerical Recipes initial guesses."""
    x = np.zeros(n); w = np.zeros(n)
    for i in range(n):
        if i == 0:
            z = (1 + a) * (3 + 0.92 * a) / (1 + 2.4 * n + 1.8 * a)
        elif i == 1:
            z = x[0] + (15 + 6.25 * a) / (1 + 0.9 * a + 2.5 * n)
        else:
            ai = i - 1
            z = x[i - 1] + ((1 + 2.55 * ai) / (1.9 * ai) + 1.26 * ai * a / (1 + 3.5 * ai)) \
                * (x[i - 1] - x[i - 2]) / (1 + 0.3 * a)
        for _ in range(100):
            L = laguerre_rec(n, a, np.array([z]))[:, 0]
            dL = (n * L[n] - (n + a) * L[n - 1]) / z       # derivative, DLMF §18.9
            dz = L[n] / dL
            z -= dz
            if abs(dz) < tol * max(1.0, z):
                break
        x[i] = z
        Lp1 = laguerre_rec(n + 1, a, np.array([z]))[n + 1, 0]
        w[i] = exp(lgamma(n + a + 1) - lgamma(n + 1)) * z / ((n + 1) ** 2 * Lp1 ** 2)
    return x, w


def gauss_laguerre_gw(n, a=0.0):
    """Golub-Welsch: eigenvalues of the symmetric Jacobi matrix"""
    k = np.arange(n)
    d = 2 * k + 1 + a
    e = -np.sqrt((k[1:]) * (k[1:] + a))
    x, V = eigh_tridiagonal(d, e)
    w = exp(lgamma(a + 1)) * V[0] ** 2
    return x, w


def partA():
    print("=" * 78)
    print("PART A  Generalised Laguerre polynomials L_n^(a)(x)")
    print("=" * 78)
    # --- 1. values vs high-precision reference
    xs = np.array([0.5, 5.0, 20.0, 60.0])
    print("Recurrence vs explicit sum vs mpmath (50 digits), n = 30, a = 1")
    print(f"{'x':>6} {'recurrence':>24} {'mpmath':>24} {'rel.err rec':>12} {'rel.err sum':>12}")
    for x in xs:
        ref = float(mp.laguerre(30, 1, x))
```

```
        r = laguerre_rec(30, 1, np.array([x]))[30, 0]
        s = laguerre_sum(30, 1, np.array([x]))[0]
        print(f"{x:6.1f} {r:24.15e} {ref:24.15e} {abs(r-ref)/abs(ref):12.2e} {abs(s-ref)/abs(ref):12.2e}")
    # --- 2. Gauss-Laguerre rules
    xn, wn = gauss_laguerre_newton(20)
    xg, wg = gauss_laguerre_gw(20)
    print(f"\n20-point Gauss-Laguerre: max|x_newton - x_GW| = {np.max(abs(xn-xg)):.2e}, "
          f"max|w_newton - w_GW|/w = {np.max(abs(wn-wg)/wg):.2e}, sum w = {wn.sum():.15f}")
    # --- 3. orthogonality by quadrature  int e^-x x^a L_m L_n = Gamma(n+a+1)/n! delta
    a = 2.0
    xq, wq = gauss_laguerre_newton(40, a)
    L = laguerre_rec(12, a, xq)
    G = (L * wq) @ L.T
    hn = np.array([exp(lgamma(n + a + 1) - lgamma(n + 1)) for n in range(13)])
    print(f"orthogonality (a=2, n,m<=12, 40-pt rule): max |G_mn/h_n - delta| = "
          f"{np.max(abs(G/np.sqrt(np.outer(hn,hn)) - np.eye(13))):.2e}")
    print(f"agreement with scipy eval_genlaguerre (n<=12, 40 nodes): "
          f"{np.max(abs(L - np.array([eval_genlaguerre(n, a, xq) for n in range(13)]))/(1+abs(L))):.2e}")

    # --- error analysis figure data
    xx = np.linspace(0.1, 100, 300)
    ref = np.array([float(mp.laguerre(30, 1, x)) for x in xx])
    e_rec = abs(laguerre_rec(30, 1, xx)[30] - ref) / abs(ref)
    e_sum = abs(laguerre_sum(30, 1, xx) - ref) / abs(ref)
    # quadrature convergence
    ns = np.arange(2, 41, 2)
    f1 = lambda x: np.cos(x)                    # int e^-x cos x = 1/2
    f2 = lambda x: 1 / (1 + x)                  # int e^-x/(1+x) = e E1(1)
    f3 = lambda x: np.sqrt(x)                   # int e^-x sqrt x = sqrt(pi)/2
    ex2 = float(mp.e * mp.e1(1))
    err = {k: [] for k in ("cos", "1/(1+x)", "sqrt")}
    for n in ns:
        xq, wq = gauss_laguerre_newton(n)
        err["cos"].append(abs(wq @ f1(xq) - 0.5))
        err["1/(1+x)"].append(abs(wq @ f2(xq) - ex2))
        err["sqrt"].append(abs(wq @ f3(xq) - np.sqrt(np.pi) / 2))
    print("\nGauss-Laguerre quadrature errors:")
    for i, n in enumerate(ns):
        if n in (2, 6, 10, 20, 30, 40):
            print(f"  n={n:2d}: cos {err['cos'][i]:.2e}   1/(1+x) {err['1/(1+x)'][i]:.2e}   sqrt(x)
{err['sqrt'][i]:.2e}")
    sl = np.polyfit(np.log(ns[5:]), np.log(err["sqrt"][5:]), 1)[0]
    print(f"  sqrt(x): algebraic decay ~ n^{sl:.2f}")

    # ---- figures
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    x = np.linspace(0, 15, 600)
    L = laguerre_rec(5, 0, x)
    for n in range(6):
        ax[0].plot(x, L[n], color=C[n], lw=2, label=f"$L_{n}$")
    ax[0].set(ylim=(-10, 20), xlabel="x", title=r"(a) Laguerre polynomials $L_n(x)$ ($\alpha=0$)")
    ax[0].legend(frameon=False, fontsize=8, ncol=3)
    for n in range(4):
        y = np.exp(-x / 2) * laguerre_rec(n, 0, x)[n]
        ax[1].plot(x, y, color=C[n], lw=2, label=f"$e^{{-x/2}}L_{n}$")
    ax[1].set(xlabel="x", title="(b) Laguerre functions (orthonormal on $[0,\\infty)$)")
    ax[1].legend(frameon=False, fontsize=8)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch1_functions.png", dpi=170)

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].semilogy(xx, e_sum + 1e-17, color=C[1], lw=1.5, label="explicit sum")
    ax[0].semilogy(xx, e_rec + 1e-17, color=C[0], lw=1.5, label="three-term recurrence")
    ax[0].set(xlabel="x", ylabel="relative error", title=r"(a) $L_{30}^{(1)}(x)$ vs 50-digit reference")
    ax[0].legend(frameon=False, fontsize=8)
    for i, (k, mk) in enumerate(zip(err, ("o-", "s-", "^-"))):
        ax[1].semilogy(ns, np.array(err[k]) + 1e-18, mk, color=C[i], lw=2, ms=4, label=fr"$\int_0^\infty e^{{-
x}}\,{k}\,dx$" if k != "sqrt" else r"$\int_0^\infty e^{-x}\sqrt{x}\,dx$")
    ax[1].set(xlabel="number of nodes n", ylabel="absolute error", title="(b) Gauss–Laguerre quadrature
convergence")
    ax[1].legend(frameon=False, fontsize=8)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch1_partA_errors.png", dpi=170)


# ============================================================ PART B
def R_exact(n, l, r):
    """normalised hydrogen radial function from the Laguerre recurrence (Part A)"""
    rho = 2 * r / n
    norm = np.sqrt((2.0 / n) ** 3 * exp(lgamma(n - l) - lgamma(n + l + 1)) / (2 * n))
    return norm * np.exp(-rho / 2) * rho ** l * laguerre_rec(n - l - 1, 2 * l + 1, rho)[n - l - 1]
```

```
def frobenius(r, l, E, nterm=40):
    """u(r) = r^{l+1} sum a_k r^k,  a_k k(k+2l+1) = -2 a_{k-1} - 2E a_{k-2}"""
    a = [1.0, -2.0 / (2 * l + 2)]
    for k in range(2, nterm):
        a.append((-2 * a[k - 1] - 2 * E * a[k - 2]) / (k * (k + 2 * l + 1)))
    s = np.zeros_like(r); p = np.ones_like(r)
    for ak in a:
        s += ak * p; p = p * r
    return r ** (l + 1) * s


def simpson(y, h):
    return h / 3 * (y[0] + y[-1] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum())


def shoot(E, l, h, R, rs):
    """returns D(E), assembled u, matching index.  rs = start radius (0 -> first nodes)"""
    N = int(round(R / h)); r = h * np.arange(N + 1)
    k2 = np.zeros(N + 1)
    k2[1:] = 2 * E + 2 / r[1:] - l * (l + 1) / r[1:] ** 2
    c = 1 + h * h * k2 / 12
    naive = rs < 0                                          # textbook start u0=0, u1=h^{l+1}
    js = 0 if naive else max(1, int(rs / h + 0.5))
    # outer classical turning point  E = -1/r + l(l+1)/(2r^2)
    rt = (1 + np.sqrt(max(1 + 2 * E * l * (l + 1), 0))) / (-2 * E)
    m = int(min(max(round(rt / h), js + 3), N - 3))
    pL = np.zeros(N + 1)
    if naive:
        pL[1] = h ** (l + 1)                                # c_0 u_0 taken as 0 (wrong for l<=1)
    else:
        pL[:js + 2] = frobenius(r[:js + 2], l, E)          # series up to the start nodes
    for n in range(js + 1, m + 1):
        pL[n + 1] = ((12 - 10 * c[n]) * pL[n] - c[n - 1] * pL[n - 1]) / c[n + 1]
    pR = np.zeros(N + 1); pR[N - 1] = 1e-30
    for n in range(N - 1, m - 1, -1):
        pR[n - 1] = ((12 - 10 * c[n]) * pR[n] - c[n + 1] * pR[n + 1]) / c[n - 1]
    sL = np.max(abs(pL[:m + 2])); sR = np.max(abs(pR[m - 1:]))
    pL /= sL; pR /= sR
    D = pL[m + 1] * pR[m] - pL[m] * pR[m + 1]
    u = np.concatenate([pL[:m + 1], pR[m + 1:] * pL[m] / pR[m]])
    return D, u, r


def eigen(n, l, h, R=None, rs=0.25):
    """bisection on D(E) in a bracket of +-15% around a scan-located root"""
    Ex = -0.5 / n ** 2
    R = 40.0 * n if R is None else R                        # box grows with n (u ~ r^n e^{-r/n})
    a, b = Ex * 1.25, Ex * 0.75
    fa = shoot(a, l, h, R, rs)[0]; fb = shoot(b, l, h, R, rs)[0]
    assert np.sign(fa) != np.sign(fb), (n, l, h)
    for _ in range(60):
        mid = 0.5 * (a + b); fm = shoot(mid, l, h, R, rs)[0]
        if np.sign(fm) == np.sign(fa): a, fa = mid, fm
        else: b = mid
    E = 0.5 * (a + b)
    _, u, r = shoot(E, l, h, R, rs)
    u /= np.sqrt(simpson(u * u, h))
    ex = r * R_exact(n, l, r)
    if np.dot(u, ex) < 0: u = -u
    return E, u, r


def scan_levels(l, h=0.01, R=160.0, rs=0.25):
    """locate all levels of given l with E < -0.02 by scanning D(E) on a log grid"""
    Eg = -np.exp(np.linspace(np.log(0.6), np.log(0.02), 500))
    D = np.array([shoot(E, l, h, R, rs)[0] for E in Eg])
    return [0.5 * (Eg[i] + Eg[i + 1]) for i in np.where(np.sign(D[:-1]) != np.sign(D[1:]))[0]]


def partB():
    print("\n" + "=" * 78)
    print("PART B  Hydrogen atom: radial Schroedinger equation, Numerov, h = 0.01, R = 40 n")
    print("=" * 78)
    for l in (0, 1, 2):
        lev = scan_levels(l)
        print(f"  scan l={l}: sign changes near E = " + ", ".join(f"{E:.4f}" for E in lev))
    print(f"\n{'n':>2} {'l':>2} {'E Numerov':>17} {'E exact':>13} {'|dE|':>9} {'norm':>15} {'max|u-u_ex|':>12}"
          f" {'<r>':>9} {'<r>exact':>9} {'<V>/2E':>10}")
    states = {}
    for (n, l) in ((1, 0), (2, 0), (2, 1), (3, 0), (3, 1), (3, 2), (4, 3)):
        E, u, r = eigen(n, l, 0.01)
```

```
        h = r[1]
        ex = r * R_exact(n, l, r)
        rexp = simpson(u * u * r, h)
        Vexp = -simpson(np.concatenate([[0.0], u[1:] ** 2 / r[1:]]), h)        # <V> = -<1/r>
        states[n, l] = (u, r)
        print(f"{n:2d} {l:2d} {E:17.12f} {-0.5/n**2:13.10f} {abs(E+0.5/n**2):9.1e} {simpson(u*u,h):15.12f} "
              f"{np.max(abs(u-ex)):12.1e} {rexp:9.5f} {(3*n*n-l*(l+1))/2:9.5f} {Vexp/(2*E):10.7f}")
    for l in (0, 1):
        ns = [n for (n, ll) in states if ll == l]
        def ov(a, b):
            ua, ub = states[a, l][0], states[b, l][0]; k = min(len(ua), len(ub))
            return simpson(ua[:k - (k + 1) % 2] * ub[:k - (k + 1) % 2], 0.01)
        O = max(abs(ov(a, b)) for a in ns for b in ns if a < b)
        print(f"  orthogonality l={l}: max |<u_n|u_n'>| (n != n') = {O:.1e}")

    # ---------------- convergence: start strategies
    hs = np.array([0.08, 0.04, 0.02, 0.01, 0.005])
    cases = [(1, 0), (2, 1), (3, 2)]
    err = {}
    for (n, l) in cases:
        for rs, lab in ((-1.0, "naive"), (0.25, "Frobenius")):
            err[n, l, lab] = [abs(eigen(n, l, h, rs=rs)[0] + 0.5 / n ** 2) for h in hs]
    print("\nEigenvalue error vs h  (naive start u0=0,u1=h^(l+1)  |  Frobenius start)")
    print(f"{'h':>8}" + "".join(f"  {'(%d,%d) naive' % c:>14} {'(%d,%d) Frob.' % c:>13}" for c in cases))
    for i, h in enumerate(hs):
        print(f"{h:8.4f}" + "".join(f"  {err[c[0],c[1],'naive'][i]:14.2e} {err[c[0],c[1],'Frobenius'][i]:13.2e}"
for c in cases))
    for c in cases:
        for lab in ("naive", "Frobenius"):
            e = np.array(err[c[0], c[1], lab]); ok = e > 2e-12
            p = np.polyfit(np.log(hs[ok][:5]), np.log(e[ok][:5]), 1)[0]
            print(f"  (n,l)={c}: {lab:12s} observed order {p:.2f}")
    # Richardson
    E1 = eigen(2, 1, 0.04)[0]; E2 = eigen(2, 1, 0.02)[0]
    print(f"\nRichardson (2p): E(0.04)={E1:.12f}  E(0.02)={E2:.12f}  E_R={(16*E2-E1)/15:.12f}  err={abs((16*E2-
E1)/15+0.125):.1e}")
    # box truncation
    print("Box truncation (h=0.01), n=3,l=0:")
    for R in (25, 30, 40, 50, 60, 80, 100, 120):
        print(f"  R={R:3d}: |dE| = {abs(eigen(3, 0, 0.01, R=float(R))[0] + 0.5/9):.2e}")

    # ---------------- figures
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.9))
    for i, (n, l) in enumerate(((1, 0), (2, 0), (2, 1), (3, 0), (3, 1), (3, 2))):
        u, r = states[n, l]; sel = r < 25
        ax[0].plot(r[sel], u[sel] ** 2, color=C[i], lw=2, label=f"{n}{'spd'[l]}")
        ax[0].plot(r[sel][::60], (r * R_exact(n, l, r))[sel][::60] ** 2, "o", ms=2.5, color="#333333")
    ax[0].set(xlim=(0, 25))
    ax[0].set(xlabel="r (Bohr radii)", ylabel=r"$|u_{nl}(r)|^2=r^2|R_{nl}|^2$",
              title="(a) Radial densities (lines: Numerov, dots: Laguerre exact)")
    ax[0].legend(frameon=False, fontsize=8, ncol=2)
    for i, (n, l) in enumerate(((1, 0), (2, 1), (3, 2), (3, 0))):
        u, r = states[n, l]
        ax[1].semilogy(r, abs(u - r * R_exact(n, l, r)) + 1e-18, color=C[i], lw=1.2, label=f"{n}{'spd'[l]}")
    ax[1].set(xlim=(0, 60), ylim=(1e-15, 1e-8), xlabel="r", ylabel=r"$|u_{num}-u_{exact}|$", title="(b) Pointwise
error, h = 0.01")
    ax[1].legend(frameon=False, fontsize=8, ncol=4)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch1_hydrogen.png", dpi=170)

    fig, ax = plt.subplots(figsize=(6.6, 5.4))
    for i, c in enumerate(cases):
        ax.loglog(hs, err[c[0], c[1], "naive"], "s--", color=C[i], lw=1.5, ms=5, label=f"{c[0]}{'spd'[c[1]]}:
naive start $u_1=h^{{l+1}}$")
        ax.loglog(hs, err[c[0], c[1], "Frobenius"], "o-", color=C[i], lw=2, ms=5, label=f"{c[0]}{'spd'[c[1]]}:
Frobenius start")
    ax.loglog(hs, err[1, 0, "Frobenius"][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax.loglog(hs, err[1, 0, "naive"][0] * (hs / hs[0]) ** 2, "-.", color="#888888", label=r"$\propto h^2$")
    ax.set(xlabel="h", ylabel=r"$|E_{nl}+1/2n^2|$", title="Hydrogen: convergence and the effect of the start")
    ax.legend(frameon=False, fontsize=7, loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=2); style(ax)
    fig.tight_layout(); fig.savefig("ch1_convergence.png", dpi=170)


if __name__ == "__main__":
    partA(); partB()
```

1.6 Fortran 2008 code

File: `ch1_laguerre_hydrogen.f90`

```
!======================================================================
! Chapter 1 -- LAGUERRE POLYNOMIALS and the HYDROGEN ATOM  (Fortran 2008)
! Part A: L_n^(a)(x) by recurrence and by explicit sum, Gauss-Laguerre
!         nodes/weights by Newton, orthogonality check, quadrature errors
! Part B: radial Schroedinger equation for hydrogen (atomic units, Z=1)
!         u'' = [l(l+1)/r^2 - 2/r - 2E] u ,   E_n = -1/(2n^2)
!         Numerov, two-sided shooting, Frobenius start, Wronskian matching
! Output: screen tables + ch1_f_states.dat, ch1_f_conv.dat
! Compile: gfortran -O2 -o ch1 ch1_laguerre_hydrogen.f90 && ./ch1
!======================================================================
module laguerre_mod
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.14159265358979323 8_dp
contains
  ! L_0..L_n^(a)(x) by (k+1)L_{k+1} = (2k+1+a-x)L_k - (k+a)L_{k-1}
  subroutine lag_rec(n, a, x, L)
    integer,  intent(in)  :: n
    real(dp), intent(in)  :: a, x
    real(dp), intent(out) :: L(0:n)
    integer :: k
    L(0) = 1.0_dp
    if (n >= 1) L(1) = 1.0_dp + a - x
    do k = 1, n-1
       L(k+1) = ((2*k + 1 + a - x)*L(k) - (k + a)*L(k-1))/(k + 1)
    end do
  end subroutine lag_rec

  real(dp) function lag(n, a, x)
    integer,  intent(in) :: n
    real(dp), intent(in) :: a, x
    real(dp) :: L(0:max(n,1))
    call lag_rec(max(n,1), a, x, L); lag = L(n)
  end function lag

  ! explicit sum  sum_k (-1)^k C(n+a,n-k) x^k/k!   (suffers cancellation)
  real(dp) function lag_sum(n, a, x)
    integer,  intent(in) :: n
    real(dp), intent(in) :: a, x
    integer :: k
    lag_sum = 0.0_dp
    do k = 0, n
       lag_sum = lag_sum + (-1)**k*exp(log_gamma(n+a+1) - log_gamma(real(n-k+1,dp)) &
                 - log_gamma(a+k+1) - log_gamma(real(k+1,dp)))*x**k
    end do
  end function lag_sum

  ! Gauss-Laguerre: Newton on L_n^(a) with Stroud-Secrest initial guesses
  subroutine gauss_laguerre(n, a, x, w)
    integer,  intent(in)  :: n
    real(dp), intent(in)  :: a
    real(dp), intent(out) :: x(n), w(n)
    real(dp) :: z, dz, L(0:n+1), dL
    integer  :: i, it, ai
    do i = 1, n
       if (i == 1) then
          z = (1 + a)*(3 + 0.92_dp*a)/(1 + 2.4_dp*n + 1.8_dp*a)
       else if (i == 2) then
          z = x(1) + (15 + 6.25_dp*a)/(1 + 0.9_dp*a + 2.5_dp*n)
       else
          ai = i - 2
          z = x(i-1) + ((1 + 2.55_dp*ai)/(1.9_dp*ai) + 1.26_dp*ai*a/(1 + 3.5_dp*ai)) &
              *(x(i-1) - x(i-2))/(1 + 0.3_dp*a)
       end if
       do it = 1, 100
          call lag_rec(n, a, z, L(0:n))
          dL = (n*L(n) - (n + a)*L(n-1))/z
          dz = L(n)/dL; z = z - dz
          if (abs(dz) < 1.0d-15*max(1.0_dp, z)) exit
       end do
       x(i) = z
       call lag_rec(n+1, a, z, L)
       w(i) = exp(log_gamma(n+a+1) - log_gamma(real(n+1,dp)))*z/((n+1)**2*L(n+1)**2)
    end do
  end subroutine gauss_laguerre
end module laguerre_mod

module hydrogen_mod
  use laguerre_mod
  implicit none
contains
  ! normalised exact radial function u = r R_nl, using the Laguerre recurrence
```

```
real(dp) function u_exact(n, l, r)
  integer,  intent(in) :: n, l
  real(dp), intent(in) :: r
  real(dp) :: rho, nrm
  rho = 2*r/n
  nrm = sqrt((2.0_dp/n)**3*exp(log_gamma(real(n-l,dp)) - log_gamma(real(n+l+1,dp)))/(2*n))
  u_exact = r*nrm*exp(-rho/2)*rho**l*lag(n-l-1, real(2*l+1,dp), rho)
end function u_exact

! Frobenius series  u = r^{l+1} sum a_k r^k
real(dp) function frob(r, l, E)
  real(dp), intent(in) :: r, E
  integer,  intent(in) :: l
  real(dp) :: a(0:39), s, p
  integer  :: k
  a(0) = 1.0_dp; a(1) = -2.0_dp/(2*l + 2)
  do k = 2, 39
     a(k) = (-2*a(k-1) - 2*E*a(k-2))/(k*(k + 2*l + 1))
  end do
  s = 0.0_dp; p = 1.0_dp
  do k = 0, 39
     s = s + a(k)*p; p = p*r
  end do
  frob = r**(l+1)*s
end function frob

real(dp) function simpson(y, h, n)
  integer,  intent(in) :: n
  real(dp), intent(in) :: y(0:n), h
  simpson = h/3*(y(0) + y(n) + 4*sum(y(1:n-1:2)) + 2*sum(y(2:n-2:2)))
end function simpson

! two-sided shooting; naive = .true. -> textbook start u0=0, u1=h^(l+1)
subroutine shoot(E, l, h, N, naive, D, u)
  real(dp), intent(in)  :: E, h
  integer,  intent(in)  :: l, N
  logical,  intent(in)  :: naive
  real(dp), intent(out) :: D, u(0:N)
  real(dp) :: r(0:N), c(0:N), pL(0:N), pR(0:N), rt, sL, sR
  integer  :: j, m, js
  r = [(j*h, j=0,N)]
  c(0) = 1.0_dp
  c(1:N) = 1 + h*h*(2*E + 2/r(1:N) - l*(l+1)/r(1:N)**2)/12
  rt = (1 + sqrt(max(1 + 2*E*l*(l+1), 0.0_dp)))/(-2*E)
  m = min(max(nint(rt/h), nint(0.25_dp/h)+4), N-3)
  pL = 0.0_dp
  if (naive) then
     js = 0; pL(1) = h**(l+1)
  else
     js = max(1, nint(0.25_dp/h))                    ! Frobenius start at r_s = 0.25
     do j = 0, js+1
        pL(j) = frob(r(j), l, E)
     end do
  end if
  do j = js+1, m
     pL(j+1) = ((12 - 10*c(j))*pL(j) - c(j-1)*pL(j-1))/c(j+1)
  end do
  pR = 0.0_dp; pR(N-1) = 1.0d-30
  do j = N-1, m, -1
     pR(j-1) = ((12 - 10*c(j))*pR(j) - c(j+1)*pR(j+1))/c(j-1)
  end do
  sL = maxval(abs(pL(0:m+1))); sR = maxval(abs(pR(m-1:N)))
  pL = pL/sL; pR = pR/sR
  D = pL(m+1)*pR(m) - pL(m)*pR(m+1)
  u(0:m) = pL(0:m); u(m+1:N) = pR(m+1:N)*pL(m)/pR(m)
end subroutine shoot

subroutine eigen(n, l, h, naive, E, u, Nout)
  integer,  intent(in)  :: n, l
  real(dp), intent(in)  :: h
  logical,  intent(in)  :: naive
  real(dp), intent(out) :: E
  real(dp), allocatable, intent(out) :: u(:)
  integer,  intent(out) :: Nout
  real(dp) :: a, b, fa, fm, mid, Ex, D
  integer  :: it
  Ex = -0.5_dp/n**2
  Nout = nint(40.0_dp*n/h); Nout = Nout + mod(Nout, 2)
  allocate(u(0:Nout))
  a = 1.25_dp*Ex; b = 0.75_dp*Ex
  call shoot(a, l, h, Nout, naive, fa, u)
  do it = 1, 60
```

```
      mid = 0.5_dp*(a + b)
      call shoot(mid, l, h, Nout, naive, fm, u)
      if (sign(1.0_dp, fm) == sign(1.0_dp, fa)) then
         a = mid; fa = fm
      else
         b = mid
      end if
    end do
    E = 0.5_dp*(a + b)
    call shoot(E, l, h, Nout, naive, D, u)
    u = u/sqrt(simpson(u*u, h, Nout))
  end subroutine eigen
end module hydrogen_mod

program ch1
  use laguerre_mod
  use hydrogen_mod
  implicit none
  real(dp) :: xq(40), wq(40), Lm(0:12), G(0:12,0:12), hn, maxo, E, x
  real(dp), allocatable :: u(:)
  real(dp) :: hs(5) = [0.08_dp, 0.04_dp, 0.02_dp, 0.01_dp, 0.005_dp], errn(5), errf(5)
  real(dp) :: rexp, vexp, mx, rr
  integer  :: i, j, k, n, l, ng, cases(2,3)
  integer  :: nlist(6) = [1, 2, 2, 3, 3, 3], llist(6) = [0, 0, 1, 0, 1, 2]
  real(dp) :: xs(4) = [0.5_dp, 5.0_dp, 20.0_dp, 60.0_dp]

  print '(a)', repeat('=',78)
  print '(a)', 'PART A  Laguerre polynomials (Fortran)'
  print '(a)', repeat('=',78)
  print '(a)', 'L_30^(1)(x): recurrence vs explicit sum (difference shows the cancellation of the sum)'
  do i = 1, 4
     x = xs(i)
     print '(a,f5.1,a,es24.15,a,es24.15,a,es9.2)', ' x=', x, '  rec=', lag(30, 1.0_dp, x), &
          '  sum=', lag_sum(30, 1.0_dp, x), '  rel.diff=', abs(lag_sum(30,1.0_dp,x)/lag(30,1.0_dp,x) - 1)
  end do
  call gauss_laguerre(40, 2.0_dp, xq, wq)
  G = 0.0_dp
  do k = 1, 40
     call lag_rec(12, 2.0_dp, xq(k), Lm)
     do i = 0, 12
        do j = 0, 12
           G(i,j) = G(i,j) + wq(k)*Lm(i)*Lm(j)
        end do
     end do
  end do
  maxo = 0.0_dp
  do i = 0, 12
     do j = 0, 12
        hn = sqrt(exp(log_gamma(i+3.0_dp) - log_gamma(i+1.0_dp))*exp(log_gamma(j+3.0_dp) - log_gamma(j+1.0_dp)))
        maxo = max(maxo, abs(G(i,j)/hn - merge(1.0_dp, 0.0_dp, i == j)))
     end do
  end do
  print '(a,es9.2)', ' orthogonality a=2, n,m<=12, 40-point Gauss-Laguerre: max|G/h - delta| = ', maxo
  print '(a)', ' Gauss-Laguerre errors for int e^-x cos x = 1/2 and int e^-x sqrt(x) = sqrt(pi)/2:'
  do n = 4, 40, 6
     call gauss_laguerre(n, 0.0_dp, xq(1:n), wq(1:n))
     print '(a,i3,a,es10.2,a,es10.2)', '   n=', n, '  cos: ', abs(sum(wq(1:n)*cos(xq(1:n))) - 0.5_dp), &
          '   sqrt: ', abs(sum(wq(1:n)*sqrt(xq(1:n))) - sqrt(pi)/2)
  end do

  print '(/a)', repeat('=',78)
  print '(a)', 'PART B  Hydrogen atom, Numerov h = 0.01, R = 40 n, Frobenius start'
  print '(a)', repeat('=',78)
  print '(a2,a3,a17,a14,a10,a16,a12,a10,a10,a11)', 'n', 'l', 'E Numerov', 'E exact', '|dE|', 'norm', &
       'max|u-ue|', '<r>', '<r>ex', '<V>/2E'
  open(10, file='ch1_f_states.dat', status='replace')
  do i = 1, 6
     n = nlist(i); l = llist(i)
     call eigen(n, l, 0.01_dp, .false., E, u, ng)
     mx = 0.0_dp
     if (sum(u*[(u_exact(n,l,k*0.01_dp), k=0,ng)]) < 0) u = -u     ! fix arbitrary sign
     do j = 0, ng
        rr = j*0.01_dp
        mx = max(mx, abs(u(j) - u_exact(n, l, rr)))
     end do
     rexp = simpson(u*u*[(j*0.01_dp, j=0,ng)], 0.01_dp, ng)
     vexp = -simpson([0.0_dp, u(1:ng)**2/[(j*0.01_dp, j=1,ng)]], 0.01_dp, ng)
     print '(i2,i3,f17.12,f14.10,es10.1,f16.12,es12.1,f10.5,f10.5,f11.7)', n, l, E, -0.5_dp/n**2, &
          abs(E + 0.5_dp/n**2), simpson(u*u, 0.01_dp, ng), mx, rexp, (3*n*n - l*(l+1))/2.0_dp, vexp/(2*E)
     write(10,'(2i3,2es24.15)') n, l, E, abs(E + 0.5_dp/n**2)
     deallocate(u)
  end do
```

```
  close(10)

  cases = reshape([1,0, 2,1, 3,2], [2,3])
  open(11, file='ch1_f_conv.dat', status='replace')
  print '(/a)', 'Eigenvalue error vs h: naive start u1 = h^(l+1)  |  Frobenius start'
  do k = 1, 3
     n = cases(1,k); l = cases(2,k)
     do i = 1, 5
        call eigen(n, l, hs(i), .true.,  E, u, ng); errn(i) = abs(E + 0.5_dp/n**2); deallocate(u)
        call eigen(n, l, hs(i), .false., E, u, ng); errf(i) = abs(E + 0.5_dp/n**2); deallocate(u)
        write(11,'(2i3,3es16.7)') n, l, hs(i), errn(i), errf(i)
     end do
     print '(a,i1,a,i1,a,5es10.2)', ' (', n, ',', l, ') naive    : ', errn
     print '(a,i1,a,i1,a,5es10.2)', ' (', n, ',', l, ') Frobenius: ', errf
  end do
  close(11)
end program ch1
```

1.7 Results, normalisation and verification

Part A. $L_{30}^{(1)}(x)$ against a 50-digit reference:

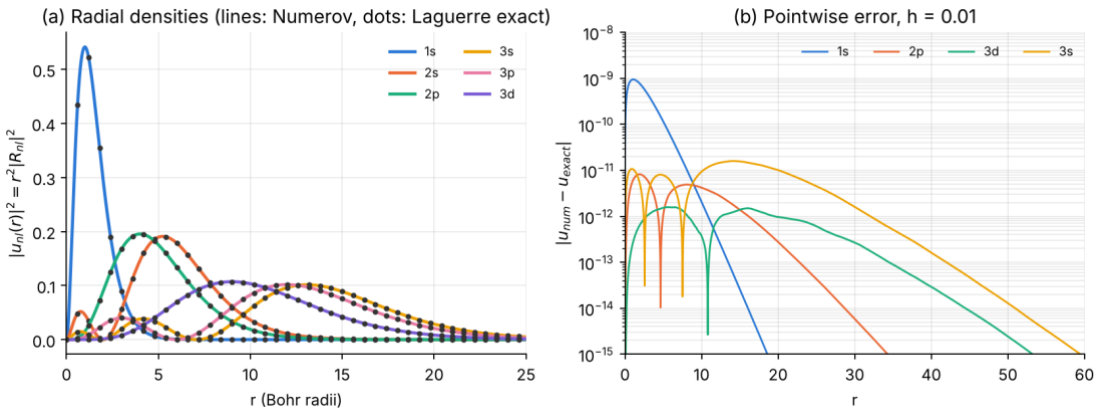
x	recurrence	rel. error recurrence	rel. error explicit sum
0.5	2.163585262160450	1.2e-15	1.9e-12
5	−4.516468178217739	5.9e-16	2.7e-07
20	−1624.339169078655	1.4e-16	6.9e-02
60	4.650827778222133e+11	2.6e-16	2.3e-01

Further checks: the 20-point Gauss–Laguerre nodes from Newton and from Golub–Welsch agree to 3.6×10^{-14}, $\sum w_i = 1$ to 10^{-14}; orthogonality of $L_0^{(2)} \dots L_{12}^{(2)}$ with a 40-point rule holds to 2.8×10^{-14}; agreement with SciPy `eval_genlaguerre` to 1.4×10^{-14}. A 20-point rule integrates $x^{39}e^{-x}$ exactly (4×10^{-16}) but not $x^{40}e^{-x}$ (7×10^{-12}), as the theory of Gaussian quadrature predicts.

Part B. Hydrogen, $h = 0.01$, Frobenius start:

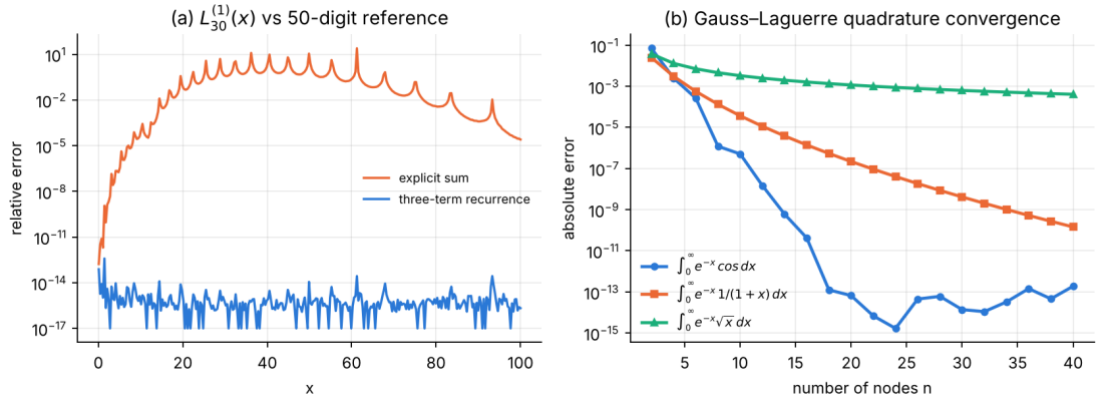
n	l	E Numerov	E exact	|ΔE|	∫u²dr	max|u − u_ex|	⟨r⟩	⟨r⟩ exact	⟨V⟩/2E
1	0	−0.500000000093	−0.5	9.3e-11	1.000000000000	9.4e-10	1.50000	1.5	1.0000000
2	0	−0.125000000001	−0.125	1.4e-12	1.000000000000	7.1e-11	6.00000	6	1.0000000
2	1	−0.124999999999	−0.125	1.5e-12	1.000000000000	8.2e-12	5.00000	5	1.0000000
3	0	−0.055555555556	−1/18	1.9e-13	1.000000000000	1.6e-11	13.50000	13.5	1.0000000
3	1	−0.055555555555	−1/18	5.2e-13	1.000000000000	8.3e-12	12.50000	12.5	1.0000000
3	2	−0.055555555556	−1/18	1.8e-13	1.000000000000	1.6e-12	10.50000	10.5	1.0000000
4	3	−0.031250000000	−1/32	1.0e-13	1.000000000000	3.1e-12	18.00000	18	1.0000000

The energy scan finds, for each l, exactly the levels $n = l+1, l+2, \dots$ — the l**-degeneracy** of the Coulomb problem is reproduced. States of equal l are orthogonal (below 10^{-9}), and the virial theorem holds to 7 digits.



(a) Radial probability densities u_{nl}^2 (lines: Numerov; dots: exact functions built from the Laguerre recurrence). (b) Pointwise error at $h = 0.01$.

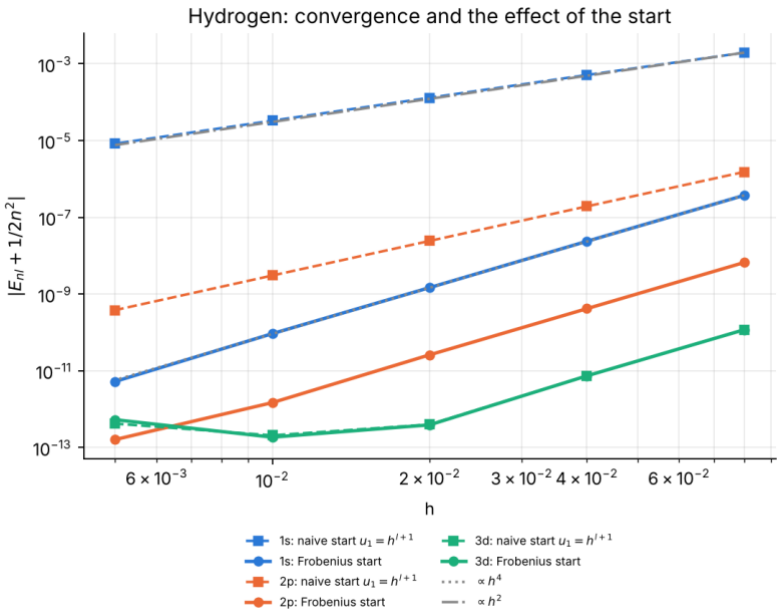
1.8 Error analysis and convergence



(a) $L_{30}^{(1)}$: the explicit sum loses all accuracy for $x \gtrsim 15$, the recurrence stays at round-off level. (b) Gauss–Laguerre quadrature: exponential convergence for the entire function $\cos x$, slower for $1/(1+x)$ (pole at $x = -1$), and only algebraic ($\propto n^{-1.5}$) for $\sqrt{x}$, whose derivative is singular at the end point.

Discretisation error and the start (Part B).

h	1s naive	1s Frobenius	2p naive	2p Frobenius	3d naive	3d Frobenius
0.08	1.96e-03	3.73e-07	1.52e-06	6.63e-09	1.17e-10	1.17e-10
0.04	5.11e-04	2.37e-08	1.92e-07	4.15e-10	7.34e-12	7.34e-12
0.02	1.31e-04	1.47e-09	2.42e-08	2.59e-11	3.97e-13	3.87e-13
0.01	3.30e-05	9.25e-11	3.03e-09	1.47e-12	2.06e-13	1.85e-13
0.005	8.29e-06	5.23e-12	3.80e-10	1.60e-13	4.17e-13	5.28e-13
order	**1.97**	**4.02**	**2.99**	**4.00**	3.99	3.99



Hydrogen: convergence of the energy for the naive and the Frobenius start.

Interpretation (Part 0): u is analytic, so the stencil itself is fine; but the naive start sets $c_0 u_0 = 0$ although $\lim_{r\to 0} k^2 u = 2Zu'(0) \neq 0$ for $l = 0$ and $= -l(l+1)\,u''(0)/2 \neq 0$ for $l = 1$. The resulting defect excites the irregular solution with relative amplitude $h^{2l+1} \times \delta$, giving order 2 for s states and 3 for p states. For $l \geq 2$ the limit vanishes and the naive start is harmless. With the Frobenius start the order is 4 for all l, until the round-off floor ($\sim 2 \times 10^{-13}$) is reached.

Richardson extrapolation (2p): $E(0.04) = -0.124999999585$, $E(0.02) = -0.124999999974$, $E_R = -0.125000000000$ (error 8×10^{-15}).

Box truncation ($3s$, $h = 0.01$): replacing $u(\infty) = 0$ by $u(R) = 0$ gives $|\Delta E| = 9.6 \times 10^{-4}$ ($R = 25$), 1.3×10^{-6} ($R = 40$), 3.3×10^{-11} ($R = 60$), 2×10^{-13} ($R \geq 80$). The decay is $\propto R^{2n} e^{-2R/n}$, which motivates the choice $R = 40n$.

1.9 Exercises

1. **Exactness of Gauss–Laguerre.** Show that an n-point rule integrates $x^k e^{-x}$ exactly for $k \leq 2n - 1$. Verify numerically for $n = 20$. *Answer:* relative error 4×10^{-16} for $k = 39$ and 7×10^{-12} for $k = 40$.

2. **Cancellation in the explicit sum.** Estimate the largest term of the explicit sum of $L_{30}^{(1)}(x)$ and explain why the relative error at $x = 60$ is of order 10^{-1}. *Hint:* the largest term is $\approx x^k/k! \cdot \binom{31}{30-k}$ near $k \approx x$; compare with $\left|L_{30}^{(1)}(60)\right| \approx 4.7 \times 10^{11}$ and machine precision.

3. $\langle r^2 \rangle$**.** Compute $\langle r^2 \rangle$ for the $3d$ state with the program and compare with $\frac{n^2}{2}[5n^2 + 1 - 3l(l+1)]$. *Answer:* 125.999999999 vs 126.

4. **Hydrogen-like ions.** Generalise the code to nuclear charge Z (change $2/r \to 2Z/r$ in k^2, in the Frobenius recurrence and in R_{nl}). Verify $E_n = -Z^2/2n^2$ for He^+ ($Z = 2$), and explain why the box can be taken Z times smaller.

5. **Screened Coulomb (Yukawa) potential** $V = -e^{-r/D}/r$, $D = 10$. There is no closed form. Use the Coulomb Frobenius series with $E \to E + 1/D$ (why?) and compute the ground state for $h = 0.02$

and 0.01, then extrapolate. *Answer:* $E(0.02) = -0.40706410$, $E(0.01) = -0.40705880$, Richardson $E_0 = -0.4070585$ (compare: -0.5 unscreened; first-order perturbation $-0.5 + 1/D = -0.4$).

6. **Naive start, theory.** Show that $\lim_{r\to 0}(k^2 u)$ equals $2Z$ for $l = 0$ (with $u'(0) = 1$) and $-2a$ for $l = 1$ with $u = ar^2 + \cdots$, and use the argument of Part 0 to predict orders 2 and 3.

Chapter 2 — Legendre functions and the angular Schrödinger equation

2.1 The function

The **associated Legendre equation** (DLMF §14.2)

$$(1-x^2)\,y'' - 2x\,y' + \left[l(l+1) - \frac{m^2}{1-x^2}\right] y = 0$$

has, for integer $0 \le m \le l$, the solution bounded on $[-1,1]$

$$P_l^m(x) = (-1)^m (1-x^2)^{m/2} \frac{d^m}{dx^m} P_l(x), \qquad P_l(x) = \frac{1}{2^l l!}\frac{d^l}{dx^l}(x^2-1)^l \quad \text{(Rodrigues)}$$

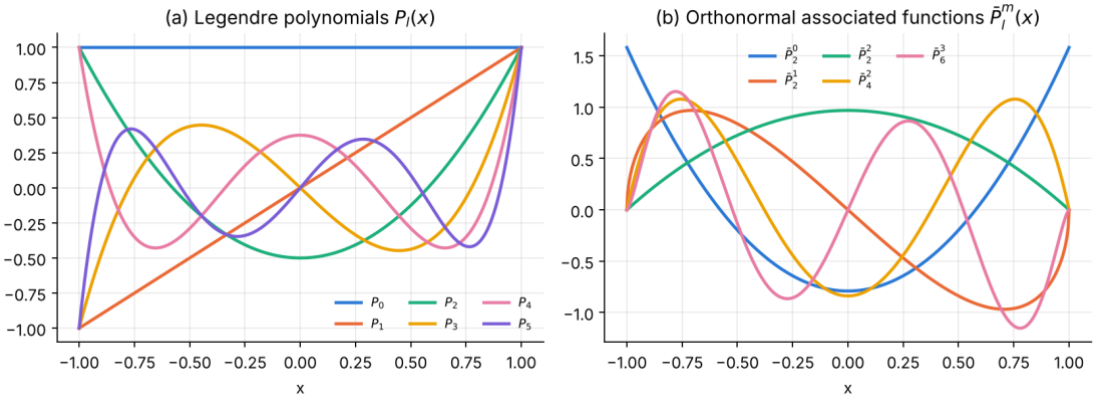
(the factor $(-1)^m$ is the Condon–Shortley phase, used by the DLMF, mpmath and SciPy). Main properties (DLMF §14.7, §14.10, §14.17, §18.3, §18.9; Arfken et al. 2013, ch. 15; Lebedev 1965, ch. 7):

- Bonnet recurrence: $(l+1)P_{l+1} = (2l+1)xP_l - lP_{l-1}$, and $P_l'(x) = l\,[xP_l - P_{l-1}]/(x^2-1)$.
- Orthogonality: $\int_{-1}^{1} P_l^m\, P_{l'}^m\, dx = \frac{2}{2l+1}\frac{(l+m)!}{(l-m)!}\,\delta_{ll'}$.
- Explicit sum: $P_l(x) = 2^{-l}\sum_k (-1)^k \binom{l}{k}\binom{2l-2k}{l} x^{l-2k}$.
- Generating function: $(1-2xt+t^2)^{-1/2} = \sum_l P_l\,(x)t^l$.
- Spherical harmonics: $Y_{lm}(\theta,\phi) = \bar{P}_l^m(\cos\theta)\,e^{im\phi}/\sqrt{2\pi}$ with the orthonormal functions $\bar{P}_l^m = \sqrt{2l+1/2\,(l-m)!/(l+m)!}\;P_l^m$ (DLMF §14.30).

Overflow-free normalised recurrence (used in both codes): with $s = \sqrt{1-x^2}$,

$$\bar{P}_0^0 = 1/\sqrt{2}, \quad \bar{P}_m^m = -\sqrt{2m+1/2m}\;s\,\bar{P}_{m-1}^{m-1}, \quad \bar{P}_{m+1}^m = \sqrt{2m+3}\,x\,\bar{P}_m^m, \quad \bar{P}_l^m = a_l\left(x\bar{P}_{l-1}^m - \frac{\bar{P}_{l-2}^m}{a_{l-1}}\right),\; a_l = \sqrt{\frac{4l^2-1}{l^2-m^2}}.$$

No factorials appear, so arbitrarily large l, m can be handled (the textbook form starts with $(2m-1)!!$, which overflows for $m \gtrsim 150$).



Legendre polynomials and orthonormal associated Legendre functions.

2.2 The physics: the angular equation

For any central potential, $\psi = R(r)\Theta(\theta)e^{im\phi}$ and the polar part satisfies

$$-\frac{1}{\sin\theta}\frac{d}{d\theta}\left(\sin\theta\frac{d\Theta}{d\theta}\right)+\frac{m^2}{\sin^2\theta}\Theta=\lambda\,\Theta,$$

i.e. the eigenvalue problem of $\hat{L}^2$. Regularity at both poles quantises $\lambda = l(l+1)$, $l = |m|, |m|+1, \ldots$, with $\Theta = \bar{P}_l^m(\cos\theta)$ (Messiah 1961, App. B; Sakurai & Napolitano 2017, §3.6). The same equation is the Schrödinger equation of the **rigid rotor** (diatomic molecule) with $E = \lambda/2I$. Exact checks: $\langle\cos^2\theta\rangle = \frac{2l(l+1)-2m^2-1}{(2l-1)(2l+3)}$, parity $\Theta(\pi-\theta) = (-1)^{l+m}\Theta(\theta)$.

2.3 Analytical scheme

Part A. Bonnet recurrence (stable forwards); explicit sum for comparison; normalised recurrence for $\bar{P}_l^m$; Gauss–Legendre nodes by Newton from Tricomi's guesses $\cos\big(\pi(i-1/4)/(n+1/2)\big)$, weights $w_i = 2/\big[(1-x_i^2)P_n'(x_i)^2\big]$.

Part B. Liouville transform $u = \sqrt{\sin\theta}\,\Theta$:

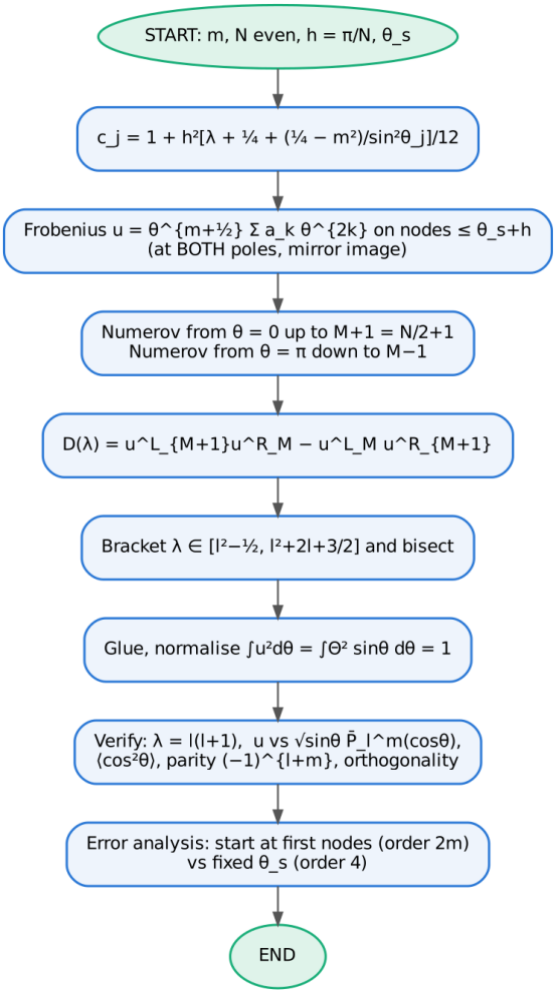
$$u''+\left[\lambda+1/4+\frac{1/4-m^2}{\sin^2\theta}\right]u=0,\qquad u\sim\theta^{m+1/2}\ (\theta\to0),\qquad \int_0^\pi u^2\,d\theta=\int_0^\pi\Theta^2\sin\theta\,d\theta.$$

- Grid $\theta_j = j\pi/N$, N even; $c_j = 1 + h^2k_j^2/12$.
- Frobenius series $u = \theta^{m+1/2}\sum_k a_k\,\theta^{2k}$ with $1/\sin^2\theta = \sum_j b_j\,\theta^{2j-2}$ ($b = 1, 1/3, 1/15, 2/189, 1/675, \ldots$):

$$2k(2k+2m)\,a_k = -(\lambda+1/4)\,a_{k-1} - (1/4-m^2)\sum_{j\geq1} b_j\,a_{k-j}.$$

- Two-sided shooting from **both poles** (the problem is symmetric), matched at $\theta = \pi/2$ with the discrete Wronskian $D(\lambda)$; bracket $[l^2-1/2,\ l^2+2l+3/2]$, 64 bisections.
- Start either at the first two nodes or at fixed $\theta_s = 0.4$.

2.4 Logic chart



Logic chart of the angular-equation program.

2.5 Python 3 code

File: `ch2_legendre_rotor.py`

```
#!/usr/bin/env python3
"""
Chapter 2 -- LEGENDRE FUNCTIONS and the ANGULAR SCHROEDINGER EQUATION (Numerov)
================================================================================
Part A : Legendre polynomials P_l(x) and associated functions P_l^m(x)
         * Bonnet recurrence for P_l                      DLMF §14.10, §18.9
         * normalised associated functions  Pbar_l^m  by the stable
           recurrence in l (no factorials, no overflow)    DLMF §14.10, §14.7
         * explicit sum for P_l                           DLMF §18.5 (cancellation)
         * Gauss-Legendre nodes and weights (Newton)       DLMF §3.5(v)
         * verification: orthonormality, mpmath reference, scipy lpmv
         * error analysis: sum vs recurrence, unnormalised overflow,
           Gauss-Legendre vs trapezoid convergence
Part B : angular equation (rigid rotor, hbar = I = 1, E = lambda/2)
         -(1/sin t) d/dt (sin t dTheta/dt) + m^2/sin^2 t Theta = lambda Theta
         Liouville transform  u = sqrt(sin t) Theta :
         u'' + [ lambda + 1/4 + (1/4 - m^2)/sin^2 t ] u = 0 ,  u ~ t^{m+1/2}
         exact lambda = l(l+1), Theta = Pbar_l^m(cos t)
         Numerov, two-sided shooting from both poles, Frobenius start,
         Wronskian matching at t = pi/2
         * verification: normalisation int u^2 dt = 1, comparison with Pbar_l^m,
           <cos^2 t>, parity, orthogonality
         * error analysis: start at first nodes (order 2m!) vs fixed t_s (order 4)
Outputs: tables + ch2_functions.png, ch2_partA_errors.png, ch2_rotor.png, ch2_convergence.png
"""
import numpy as np
from math import lgamma, exp, comb
import mpmath as mp
```

```
from scipy.special import lpmv
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#7a5cd6"]
mp.mp.dps = 50


def style(ax):
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")


# ============================================================ PART A
def legendre_rec(L, x):
    """P_0..P_L(x) by Bonnet:  (l+1) P_{l+1} = (2l+1) x P_l - l P_{l-1}"""
    x = np.asarray(x, float)
    P = np.zeros((L + 1,) + x.shape); P[0] = 1.0
    if L >= 1: P[1] = x
    for l in range(1, L):
        P[l + 1] = ((2 * l + 1) * x * P[l] - l * P[l - 1]) / (l + 1)
    return P


def legendre_sum(l, x):
    """explicit sum  P_l = 2^-l sum_k (-1)^k C(l,k) C(2l-2k,l) x^{l-2k}"""
    x = np.asarray(x, float)
    return sum((-1) ** k * comb(l, k) * comb(2 * l - 2 * k, l) * x ** (l - 2 * k)
               for k in range(l // 2 + 1)) / 2.0 ** l


def plm_bar(L, m, x):
    """orthonormal associated Legendre functions Pbar_l^m(x), l = m..L
       (int_{-1}^{1} Pbar^2 dx = 1, Condon-Shortley phase included)
       Pbar_m^m     = -sqrt((2m+1)/(2m)) sqrt(1-x^2) Pbar_{m-1}^{m-1},  Pbar_0^0 = 1/sqrt 2
       Pbar_{m+1}^m = sqrt(2m+3) x Pbar_m^m
       Pbar_l^m     = a_l ( x Pbar_{l-1}^m - Pbar_{l-2}^m / a_{l-1} ),
                      a_l = sqrt((4l^2-1)/(l^2-m^2))"""
    x = np.asarray(x, float)
    s = np.sqrt(np.maximum(1 - x * x, 0.0))
    p = np.full_like(x, 1 / np.sqrt(2.0))
    for k in range(1, m + 1):
        p = -np.sqrt((2 * k + 1) / (2 * k)) * s * p
    out = np.zeros((L + 1,) + x.shape)
    out[m] = p
    if L > m:
        out[m + 1] = np.sqrt(2 * m + 3) * x * p
    a = lambda l: np.sqrt((4 * l * l - 1) / (l * l - m * m))
    for l in range(m + 2, L + 1):
        out[l] = a(l) * (x * out[l - 1] - out[l - 2] / a(l - 1))
    return out


def plm_unnorm(L, m, x):
    """textbook unnormalised recurrence (starts with (2m-1)!!: overflows for large m)"""
    x = np.asarray(x, float)
    with np.errstate(all="ignore"):
        dfact = np.prod(np.arange(1, 2 * m, 2, dtype=float)) if m > 0 else 1.0
        out = np.zeros((L + 1,) + x.shape)
        out[m] = (-1) ** m * dfact * (1 - x * x) ** (m / 2)
        if L > m: out[m + 1] = x * (2 * m + 1) * out[m]
        for l in range(m + 1, L):
            out[l + 1] = ((2 * l + 1) * x * out[l] - (l + m) * out[l - 1]) / (l - m + 1)
    return out


def gauss_legendre(n, tol=1e-15):
    """Newton on P_n with the Tricomi initial guesses cos(pi (i-1/4)/(n+1/2))"""
    x = np.zeros(n); w = np.zeros(n)
    for i in range(n):
        z = np.cos(np.pi * (i + 0.75) / (n + 0.5))
        for _ in range(100):
            P = legendre_rec(n, np.array([z]))[:, 0]
            dP = n * (z * P[n] - P[n - 1]) / (z * z - 1)
            dz = P[n] / dP; z -= dz
            if abs(dz) < tol: break
        P = legendre_rec(n, np.array([z]))[:, 0]
        dP = n * (z * P[n] - P[n - 1]) / (z * z - 1)
        x[i], w[i] = z, 2 / ((1 - z * z) * dP * dP)
    return x[::-1], w[::-1]
```

```
def partA():
    print("=" * 78)
    print("PART A  Legendre polynomials and associated Legendre functions")
    print("=" * 78)
    print("P_l(x): Bonnet recurrence vs explicit sum vs mpmath (50 digits)")
    print(f"{'l':>4} {'x':>6} {'recurrence':>24} {'mpmath':>24} {'rel.err rec':>12} {'rel.err sum':>12}")
    for l, x in ((10, 0.3), (40, 0.3), (60, 0.05), (100, 0.9)):
        ref = float(mp.legendre(l, x))
        r = legendre_rec(l, np.array([x]))[l, 0]; s = legendre_sum(l, np.array([x]))[0]
        print(f"{l:4d} {x:6.2f} {r:24.15e} {ref:24.15e} {abs(r-ref)/abs(ref):12.2e} {abs(s-ref)/abs(ref):12.2e}")
    print("\nNormalised Pbar_l^m vs unnormalised recurrence vs mpmath, x = 0.5")
    for l, m in ((5, 2), (30, 10), (120, 100), (200, 180)):
        nrm = mp.sqrt((2 * l + 1) / mp.mpf(2) * mp.factorial(l - m) / mp.factorial(l + m))
        ref = nrm * mp.legenp(l, m, 0.5)                    # mpmath includes (-1)^m
        pb = plm_bar(l, m, np.array([0.5]))[l, 0]
        pu = plm_unnorm(l, m, np.array([0.5]))[l, 0]
        pu_n = float(nrm) * pu if np.isfinite(pu) else np.nan
        print(f"  l={l:3d} m={m:3d}: Pbar = {pb: .15e}  mpmath = {float(ref): .15e}  rel.err {abs(pb-
float(ref))/abs(float(ref)):.1e}"
              f"   unnormalised: {'overflow (inf)' if not np.isfinite(pu) else f'{pu_n: .6e}'}")
    xg, wg = gauss_legendre(30)
    P = plm_bar(20, 3, xg)[3:]
    G = (P * wg) @ P.T
    print(f"\northonormality of Pbar_l^3, l=3..20, 30-point Gauss-Legendre: max|G-I| = {np.max(abs(G-
np.eye(len(G)))):.1e}")
    ls = np.arange(3, 21)
    scip = np.array([lpmv(3, l, xg) * np.sqrt((2 * l + 1) / 2 * exp(lgamma(l - 2) - lgamma(l + 4))) for l in ls])
    print(f"agreement with scipy lpmv (normalised): {np.max(abs(P - scip)):.1e}")
    print(f"30-point rule: sum w = {wg.sum():.15f}, int x^58 dx exact? err = {abs(wg@xg**58 - 2/59):.1e}")

    # error data
    xx = np.linspace(-0.999, 0.999, 400)
    ref40 = np.array([float(mp.legendre(40, x)) for x in xx])
    e_rec = abs(legendre_rec(40, xx)[40] - ref40) / (abs(ref40) + 1e-3)
    e_sum = abs(legendre_sum(40, xx) - ref40) / (abs(ref40) + 1e-3)
    ns = np.arange(2, 41, 2)
    tests = {r"$e^x$": (np.exp, np.e - 1 / np.e), r"$1/(1+25x^2)$": (lambda x: 1 / (1 + 25 * x * x), 0.4 *
np.arctan(5)),
             r"$|x|^3$": (lambda x: abs(x) ** 3, 0.5)}
    eg = {k: [] for k in tests}; et = []
    for n in ns:
        xq, wq = gauss_legendre(n)
        for k, (f, ex) in tests.items():
            eg[k].append(abs(wq @ f(xq) - ex) + 1e-18)
        xt = np.linspace(-1, 1, n); ht = xt[1] - xt[0]
        ft = np.exp(xt); et.append(abs(ht * (ft.sum() - 0.5 * (ft[0] + ft[-1])) - (np.e - 1 / np.e)))
    print("\nGauss-Legendre quadrature errors (n nodes):")
    for i, n in enumerate(ns):
        if n in (2, 4, 8, 12, 20, 40):
            print(f"  n={n:2d}: " + "  ".join(f"{k.strip('$')}: {eg[k][i]:.1e}" for k in tests) + f"   trapezoid
e^x: {et[i]:.1e}")

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    x = np.linspace(-1, 1, 500)
    P = legendre_rec(5, x)
    for l in range(6):
        ax[0].plot(x, P[l], color=C[l], lw=2, label=f"$P_{l}$")
    ax[0].set(xlabel="x", title=r"(a) Legendre polynomials $P_l(x)$")
    ax[0].legend(frameon=False, fontsize=8, ncol=3)
    for i, (l, m) in enumerate(((2, 0), (2, 1), (2, 2), (4, 2), (6, 3))):
        ax[1].plot(x, plm_bar(l, m, x)[l], color=C[i], lw=2, label=fr"$\bar P_{l}^{{{m}}}$")
    ax[1].set(xlabel="x", title=r"(b) Orthonormal associated functions $\bar P_l^m(x)$")
    ax[1].legend(frameon=False, fontsize=8, ncol=3)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch2_functions.png", dpi=170)

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].semilogy(xx, e_sum + 1e-18, color=C[1], lw=1.2, label="explicit sum")
    ax[0].semilogy(xx, e_rec + 1e-18, color=C[0], lw=1.2, label="Bonnet recurrence")
    ax[0].set(xlabel="x", ylabel="error / (|P|+10$^{-3}$)", title=r"(a) $P_{40}(x)$ vs 50-digit reference")
    ax[0].legend(frameon=False, fontsize=8)
    for i, k in enumerate(tests):
        ax[1].semilogy(ns, eg[k], "o-", color=C[i], lw=2, ms=4, label=f"Gauss-Legendre, {k}")
    ax[1].semilogy(ns, et, "s--", color="#888888", lw=1.5, ms=4, label=r"trapezoid, $e^x$")
    ax[1].set(ylim=(1e-17, 1), xlabel="number of nodes n", ylabel="absolute error", title=r"(b) $\int_{-1}^{1}
f\,dx$: quadrature convergence")
    ax[1].legend(frameon=False, fontsize=7)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch2_partA_errors.png", dpi=170)


# ============================================================ PART B
```

```
INV_SIN2 = [1.0, 1 / 3, 1 / 15, 2 / 189, 1 / 675, 2 / 10395, 1382 / 58046625, 4 / 1403325]


def frobenius(t, m, lam, nterm=24):
    """u = t^{m+1/2} sum a_k t^{2k};  1/sin^2 t = sum b_j t^{2j-2}
       2k(2k+2m) a_k = -(1/4-m^2) sum_{j>=1} b_j a_{k-j} - (lam+1/4) a_{k-1}"""
    q = 0.25 - m * m; L = lam + 0.25
    b = INV_SIN2 + [0.0] * nterm   # higher b_j negligible for t <= 0.5 at double precision
    a = [1.0]
    for k in range(1, nterm):
        s = -L * a[k - 1] - q * sum(b[j] * a[k - j] for j in range(1, k + 1))
        a.append(s / (2 * k * (2 * k + 2 * m)))
    t = np.asarray(t, float)
    return t ** (m + 0.5) * sum(ak * t ** (2 * k) for k, ak in enumerate(a))


def simpson(y, h):
    return h / 3 * (y[0] + y[-1] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum())


def shoot(lam, m, N, ts):
    """N even; grid t_j = j pi/N.  ts = start angle (0 -> first nodes)."""
    h = np.pi / N; t = h * np.arange(N + 1); t[N] = np.pi
    k2 = np.zeros(N + 1)
    k2[1:N] = lam + 0.25 + (0.25 - m * m) / np.sin(t[1:N]) ** 2
    c = 1 + h * h * k2 / 12
    js = max(1, int(ts / h + 0.5))
    M = N // 2
    pL = np.zeros(N + 1); pL[:js + 2] = frobenius(t[:js + 2], m, lam)
    for n in range(js + 1, M + 1):
        pL[n + 1] = ((12 - 10 * c[n]) * pL[n] - c[n - 1] * pL[n - 1]) / c[n + 1]
    pR = np.zeros(N + 1); pR[N - js - 1:] = frobenius(np.maximum(np.pi - t[N - js - 1:], 0.0), m, lam)
    for n in range(N - js - 1, M - 1, -1):
        pR[n - 1] = ((12 - 10 * c[n]) * pR[n] - c[n + 1] * pR[n + 1]) / c[n - 1]
    sL = np.max(abs(pL[:M + 2])); sR = np.max(abs(pR[M - 1:]))
    pL /= sL; pR /= sR
    D = pL[M + 1] * pR[M] - pL[M] * pR[M + 1]
    # glue at M+1 or M, whichever is larger in |pR| (odd states vanish at pi/2)
    j = M if abs(pR[M]) > abs(pR[M + 1]) else M + 1
    u = np.concatenate([pL[:j + 1], pR[j + 1:] * pL[j] / pR[j]])
    return D, u, t


def eigen(l, m, N, ts=0.4):
    lex = l * (l + 1)
    a, b = lex - l - 0.5, lex + l + 1.5            # brackets only lambda = l(l+1)
    fa = shoot(a, m, N, ts)[0]
    assert np.sign(fa) != np.sign(shoot(b, m, N, ts)[0])
    for _ in range(64):
        mid = 0.5 * (a + b); fm = shoot(mid, m, N, ts)[0]
        if np.sign(fm) == np.sign(fa): a, fa = mid, fm
        else: b = mid
    lam = 0.5 * (a + b)
    _, u, t = shoot(lam, m, N, ts)
    h = t[1]
    u /= np.sqrt(simpson(u * u, h))
    ex = np.sqrt(np.abs(np.sin(t))) * plm_bar(l, m, np.cos(t))[l]
    if np.dot(u, ex) < 0: u = -u
    return lam, u, t, ex


def partB():
    print("\n" + "=" * 78)
    print("PART B  Angular equation / rigid rotor: Numerov in theta, N = 400 (h = pi/400), t_s = 0.4")
    print("=" * 78)
    # scan to show the spectrum for m = 1
    N = 400
    lg = np.linspace(0.05, 45, 900)
    D = np.array([shoot(L, 1, N, 0.4)[0] for L in lg])
    roots = [0.5 * (lg[i] + lg[i + 1]) for i in np.where(np.sign(D[:-1]) != np.sign(D[1:]))[0]]
    print("  scan m=1: sign changes of D(lambda) near " + ", ".join(f"{r:.2f}" for r in roots) + "  (exact 2, 6,
12, 20, 30, 42)")
    print(f"\n{'l':>2} {'m':>2} {'lambda Numerov':>18} {'l(l+1)':>7} {'|dlam|':>9} {'norm':>15} {'max|u-
u_ex|':>12} {'<cos^2>':>10} {'exact':>10}")
    store = {}
    for (l, m) in ((0, 0), (1, 0), (2, 0), (1, 1), (2, 1), (3, 1), (2, 2), (3, 2), (5, 3)):
        lam, u, t, ex = eigen(l, m, N)
        h = t[1]
        c2 = simpson(u * u * np.cos(t) ** 2, h)
        c2ex = (2 * l * (l + 1) - 2 * m * m - 1) / ((2 * l - 1) * (2 * l + 3))
        store[l, m] = (u, t, ex)
        print(f"{l:2d} {m:2d} {lam:18.12f} {l*(l+1):7d} {abs(lam-l*(l+1)):9.1e} {simpson(u*u,h):15.12f}
```

```
{np.max(abs(u-ex)):12.1e} {c2:10.7f} {c2ex:10.7f}")
    ov = max(abs(simpson(store[a, 1][0] * store[b, 1][0], np.pi / N)) for a in (1, 2, 3) for b in (1, 2, 3) if a <
b)
    print(f"  orthogonality m=1: max |<u_l|u_l'>| = {ov:.1e};   parity u(pi-t) = (-1)^(l+m) u(t): "
          f"max err {max(np.max(abs(store[l,m][0][::-1] - (-1)**(l+m)*store[l,m][0])) for (l,m) in store):.1e}")

    Ns = np.array([20, 40, 80, 160, 320, 640, 1280])
    hs = np.pi / Ns
    err = {}
    for m, l in ((0, 2), (1, 2), (2, 3)):
        for ts, lab in ((0.0, "first nodes"), (0.4, "t_s = 0.4")):
            err[m, lab] = [abs(eigen(l, m, N, ts)[0] - l * (l + 1)) for N in Ns]
    print("\nEigenvalue error vs h: Frobenius start at first two grid nodes  vs  at fixed t_s = 0.4")
    print(f"{'N':>6} {'h':>8}" + "".join(f" {'m=%d nodes' % m:>12} {'m=%d t_s' % m:>11}" for m in (0, 1, 2)))
    for i, N in enumerate(Ns):
        print(f"{N:6d} {hs[i]:8.5f}" + "".join(f" {err[m,'first nodes'][i]:12.2e} {err[m,'t_s = 0.4'][i]:11.2e}"
for m in (0, 1, 2)))
    for m in (0, 1, 2):
        for lab in ("first nodes", "t_s = 0.4"):
            e = np.array(err[m, lab]); ok = e > 1e-12
            print(f"  m={m} {lab:12s}: observed order {np.polyfit(np.log(hs[ok][1:6]), np.log(e[ok][1:6]),
1)[0]:.2f}")

    fig, ax = plt.subplots(1, 2, figsize=(10, 4.2), gridspec_kw={"width_ratios": [1.5, 1]})
    for i, (l, m) in enumerate(((0, 0), (1, 0), (2, 0), (2, 1), (3, 2))):
        u, t, ex = store[l, m]
        ax[0].plot(t, u, color=C[i], lw=2, label=f"l={l}, m={m}")
        ax[0].plot(t[::16], ex[::16], "o", ms=2.5, color="#333333")
    ax[0].set(xlabel=r"$\theta$", ylabel=r"$u(\theta)=\sqrt{\sin\theta}\,\Theta(\theta)$",
              title=r"(a) Numerov (lines) vs $\sqrt{\sin\theta}\,\bar P_l^m(\cos\theta)$ (dots)")
    ax[0].legend(frameon=False, fontsize=7, ncol=2)
    th = np.linspace(0, 2 * np.pi, 400)
    for i, (l, m) in enumerate(((1, 0), (2, 0), (2, 1), (3, 2))):
        rr = abs(plm_bar(l, m, np.cos(th))[l]) ** 2
        ax[1].plot(rr * np.sin(th), rr * np.cos(th), color=C[i], lw=2, label=fr"$|Y_{{{l}{m}}}|^2$")
    ax[1].set_aspect("equal"); ax[1].set(xlabel="x-z plane", title=r"(b) Polar plots of $|\bar
P_l^m(\cos\theta)|^2$")
    ax[1].legend(frameon=False, fontsize=7, loc="center left", bbox_to_anchor=(1.0, 0.5))
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch2_rotor.png", dpi=170)

    fig, ax = plt.subplots(figsize=(6.6, 4.2))
    for i, m in enumerate((0, 1, 2)):
        ax.loglog(hs, err[m, "first nodes"], "s--", color=C[i], lw=1.5, ms=5, label=f"m={m}: start at first
nodes")
        ax.loglog(hs, err[m, "t_s = 0.4"], "o-", color=C[i], lw=2, ms=5, label=fr"m={m}: Frobenius start at
$\theta_s$=0.4")
    ax.loglog(hs, err[1, "t_s = 0.4"][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax.loglog(hs, err[1, "first nodes"][0] * (hs / hs[0]) ** 2, "-.", color="#888888", label=r"$\propto h^2$")
    ax.set(xlabel=r"$h=\pi/N$", ylabel=r"$|\lambda-l(l+1)|$", title="Angular equation: the order of convergence is
$\\min(4, 2m)$\nunless the start is moved away from the pole")
    ax.legend(frameon=False, fontsize=7); style(ax)
    fig.tight_layout(); fig.savefig("ch2_convergence.png", dpi=170)


if __name__ == "__main__":
    partA(); partB()
```

2.6 Fortran 2008 code

File: `ch2_legendre_rotor.f90`

```
!=======================================================================
! Chapter 2 -- LEGENDRE FUNCTIONS and the ANGULAR EQUATION  (Fortran 2008)
! Part A: P_l by Bonnet recurrence vs explicit sum, orthonormal associated
!         functions Pbar_l^m by the stable l-recurrence, Gauss-Legendre
!         rule by Newton, orthonormality and quadrature-error checks
! Part B: u = sqrt(sin t) Theta,  u'' + [lam + 1/4 + (1/4-m^2)/sin^2 t] u = 0
!         Numerov from both poles, Frobenius start at t_s (or first nodes),
!         Wronskian matching at t = pi/2, lam = l(l+1)
! Output: screen tables + ch2_f_conv.dat
! Compile: gfortran -O2 -o ch2 ch2_legendre_rotor.f90 && ./ch2
!=======================================================================
module legendre_mod
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.141592653589793238_dp
contains
  subroutine leg_rec(L, x, P)
    integer,  intent(in)  :: L
```

```
    real(dp), intent(in)  :: x
    real(dp), intent(out) :: P(0:L)
    integer :: k
    P(0) = 1.0_dp
    if (L >= 1) P(1) = x
    do k = 1, L-1
       P(k+1) = ((2*k + 1)*x*P(k) - k*P(k-1))/(k + 1)
    end do
  end subroutine leg_rec

  real(dp) function leg_sum(l, x)                ! explicit sum (cancellation)
    integer,  intent(in) :: l
    real(dp), intent(in) :: x
    integer :: k
    leg_sum = 0.0_dp
    do k = 0, l/2
       leg_sum = leg_sum + (-1)**k*exp(log_gamma(l+1.0_dp) - log_gamma(k+1.0_dp) - log_gamma(l-k+1.0_dp) &
                 + log_gamma(2*l-2*k+1.0_dp) - log_gamma(l+1.0_dp) - log_gamma(l-2*k+1.0_dp))*x**(l-2*k)
    end do
    leg_sum = leg_sum/2.0_dp**l
  end function leg_sum

  ! orthonormal associated Legendre functions Pbar_l^m, l = m..L
  subroutine plm_bar(L, m, x, P)
    integer,  intent(in)  :: L, m
    real(dp), intent(in)  :: x
    real(dp), intent(out) :: P(0:L)
    real(dp) :: s, p0, al, alm1
    integer  :: k
    P = 0.0_dp
    s = sqrt(max(1.0_dp - x*x, 0.0_dp))
    p0 = 1.0_dp/sqrt(2.0_dp)
    do k = 1, m
       p0 = -sqrt((2*k + 1.0_dp)/(2*k))*s*p0
    end do
    P(m) = p0
    if (L > m) P(m+1) = sqrt(2*m + 3.0_dp)*x*p0
    do k = m+2, L
       al   = sqrt((4.0_dp*k*k - 1)/(real(k,dp)**2 - m*m))
       alm1 = sqrt((4.0_dp*(k-1)**2 - 1)/(real(k-1,dp)**2 - m*m))
       P(k) = al*(x*P(k-1) - P(k-2)/alm1)
    end do
  end subroutine plm_bar

  subroutine gauss_legendre(n, x, w)
    integer,  intent(in)  :: n
    real(dp), intent(out) :: x(n), w(n)
    real(dp) :: z, dz, P(0:n), dPn
    integer  :: i, it
    do i = 1, n
       z = cos(pi*(i - 0.25_dp)/(n + 0.5_dp))
       do it = 1, 100
          call leg_rec(n, z, P)
          dPn = n*(z*P(n) - P(n-1))/(z*z - 1)
          dz = P(n)/dPn; z = z - dz
          if (abs(dz) < 1.0d-15) exit
       end do
       call leg_rec(n, z, P)
       dPn = n*(z*P(n) - P(n-1))/(z*z - 1)
       x(n+1-i) = z; w(n+1-i) = 2/((1 - z*z)*dPn*dPn)
    end do
  end subroutine gauss_legendre
end module legendre_mod

module rotor_mod
  use legendre_mod
  implicit none
  real(dp), parameter :: b(0:7) = [1.0_dp, 1/3.0_dp, 1/15.0_dp, 2/189.0_dp, 1/675.0_dp, &
                                   2/10395.0_dp, 1382/58046625.0_dp, 4/1403325.0_dp]
contains
  real(dp) function frob(t, m, lam)              ! u = t^{m+1/2} sum a_k t^{2k}
    real(dp), intent(in) :: t, lam
    integer,  intent(in) :: m
    real(dp) :: a(0:23), s, q, LL
    integer  :: k, j
    q = 0.25_dp - m*m; LL = lam + 0.25_dp
    a(0) = 1.0_dp
    do k = 1, 23
       s = -LL*a(k-1)
       do j = 1, min(k, 7)
          s = s - q*b(j)*a(k-j)
       end do
```

```
      a(k) = s/(2*k*(2*k + 2*m))
    end do
    s = 0.0_dp
    do k = 0, 23
       s = s + a(k)*t**(2*k)
    end do
    frob = t**(m + 0.5_dp)*s
  end function frob

  real(dp) function simpson(y, h, n)
    integer,  intent(in) :: n
    real(dp), intent(in) :: y(0:n), h
    simpson = h/3*(y(0) + y(n) + 4*sum(y(1:n-1:2)) + 2*sum(y(2:n-2:2)))
  end function simpson

  subroutine shoot(lam, m, N, ts, D, u)
    real(dp), intent(in)  :: lam, ts
    integer,  intent(in)  :: m, N
    real(dp), intent(out) :: D, u(0:N)
    real(dp) :: h, t(0:N), c(0:N), pL(0:N), pR(0:N), sL, sR
    integer  :: j, js, Mm, jg
    h = pi/N; t = [(j*h, j=0,N)]; t(N) = pi
    c = 1.0_dp
    c(1:N-1) = 1 + h*h*(lam + 0.25_dp + (0.25_dp - m*m)/sin(t(1:N-1))**2)/12
    js = max(1, nint(ts/h)); Mm = N/2
    pL = 0.0_dp; pR = 0.0_dp
    do j = 0, js+1
       pL(j) = frob(t(j), m, lam); pR(N-j) = frob(t(j), m, lam)
    end do
    do j = js+1, Mm
       pL(j+1) = ((12 - 10*c(j))*pL(j) - c(j-1)*pL(j-1))/c(j+1)
    end do
    do j = N-js-1, Mm, -1
       pR(j-1) = ((12 - 10*c(j))*pR(j) - c(j+1)*pR(j+1))/c(j-1)
    end do
    sL = maxval(abs(pL(0:Mm+1))); sR = maxval(abs(pR(Mm-1:N)))
    pL = pL/sL; pR = pR/sR
    D = pL(Mm+1)*pR(Mm) - pL(Mm)*pR(Mm+1)
    jg = Mm; if (abs(pR(Mm+1)) > abs(pR(Mm))) jg = Mm + 1
    u(0:jg) = pL(0:jg); u(jg+1:N) = pR(jg+1:N)*pL(jg)/pR(jg)
  end subroutine shoot

  subroutine eigen(l, m, N, ts, lam, u)
    integer,  intent(in)  :: l, m, N
    real(dp), intent(in)  :: ts
    real(dp), intent(out) :: lam, u(0:N)
    real(dp) :: a, bb, fa, fm, mid, D
    integer  :: it
    a = l*(l+1) - l - 0.5_dp; bb = l*(l+1) + l + 1.5_dp
    call shoot(a, m, N, ts, fa, u)
    do it = 1, 64
       mid = 0.5_dp*(a + bb)
       call shoot(mid, m, N, ts, fm, u)
       if (sign(1.0_dp, fm) == sign(1.0_dp, fa)) then
          a = mid; fa = fm
       else
          bb = mid
       end if
    end do
    lam = 0.5_dp*(a + bb)
    call shoot(lam, m, N, ts, D, u)
    u = u/sqrt(simpson(u*u, pi/N, N))
  end subroutine eigen
end module rotor_mod

program ch2
  use legendre_mod
  use rotor_mod
  implicit none
  integer, parameter :: NN = 400
  real(dp) :: P(0:200), xq(30), wq(30), G(18,18), Pq(0:20), u(0:NN), ex(0:NN), lam, c2, c2ex, mx, t
  real(dp), allocatable :: uu(:)
  integer  :: i, j, k, l, m, Ns(7) = [20, 40, 80, 160, 320, 640, 1280]
  integer  :: ll(9) = [0,1,2,1,2,3,2,3,5], mm(9) = [0,0,0,1,1,1,2,2,3], lconv(3) = [2, 2, 3]
  real(dp) :: e1(7), e2(7), xA(3) = [0.3_dp, 0.3_dp, 0.9_dp]
  integer  :: lA(3) = [10, 40, 100]

  print '(a)', repeat('=',78)
  print '(a)', 'PART A  Legendre functions (Fortran)'
  print '(a)', repeat('=',78)
  do k = 1, 3
     l = lA(k); t = xA(k)
```

```
      call leg_rec(l, t, P(0:l))
      print '(a,i4,a,f4.1,a,es24.15,a,es24.15,a,es9.2)', ' l=', l, ' x=', t, '  Bonnet=', P(l), '  sum=', &
           leg_sum(l, t), '  rel.diff=', abs(leg_sum(l, t)/P(l) - 1)
   end do
   call plm_bar(200, 180, 0.5_dp, P)
   print '(a,es24.15,a)', ' Pbar_200^180(0.5) = ', P(200), '   (mpmath: 7.231608361615227e-02)'
   call gauss_legendre(30, xq, wq)
   G = 0.0_dp
   do k = 1, 30
      call plm_bar(20, 3, xq(k), Pq)
      do i = 3, 20
         do j = 3, 20
            G(i-2,j-2) = G(i-2,j-2) + wq(k)*Pq(i)*Pq(j)
         end do
      end do
   end do
   do i = 1, 18
      G(i,i) = G(i,i) - 1.0_dp
   end do
   print '(a,es9.2)', ' orthonormality Pbar_l^3 (l=3..20), 30-pt Gauss-Legendre: max|G-I| = ', maxval(abs(G))
   print '(a)', ' Gauss-Legendre errors, int_{-1}^{1} e^x dx and int 1/(1+25x^2) dx:'
   do k = 2, 30, 4
      call gauss_legendre(k, xq(1:k), wq(1:k))
      print '(a,i3,2es11.2)', '   n=', k, abs(sum(wq(1:k)*exp(xq(1:k))) - (exp(1.0_dp) - exp(-1.0_dp))), &
           abs(sum(wq(1:k)/(1 + 25*xq(1:k)**2)) - 0.4_dp*atan(5.0_dp))
   end do

   print '(/a)', repeat('=',78)
   print '(a)', 'PART B  Angular equation, Numerov, N = 400, t_s = 0.4'
   print '(a)', repeat('=',78)
   print '(a2,a3,a18,a7,a10,a16,a12,a10,a10)', 'l', 'm', 'lambda', 'l(l+1)', '|dlam|', 'norm', 'max|u-ue|', &
        '<cos^2>', 'exact'
   do k = 1, 9
      l = ll(k); m = mm(k)
      call eigen(l, m, NN, 0.4_dp, lam, u)
      do j = 0, NN
         t = j*pi/NN
         call plm_bar(l, m, cos(t), P(0:l))
         ex(j) = sqrt(abs(sin(t)))*P(l)
      end do
      if (sum(u*ex) < 0) u = -u
      mx = maxval(abs(u - ex))
      c2 = simpson(u*u*cos([(j*pi/NN, j=0,NN)])**2, pi/NN, NN)
      c2ex = (2.0_dp*l*(l+1) - 2*m*m - 1)/((2*l - 1)*(2*l + 3))
      print '(i2,i3,f18.12,i7,es10.1,f16.12,es12.1,2f10.7)', l, m, lam, l*(l+1), abs(lam - l*(l+1)), &
           simpson(u*u, pi/NN, NN), mx, c2, c2ex
   end do

   open(10, file='ch2_f_conv.dat', status='replace')
   print '(/a)', 'Error of lambda vs N: Frobenius start at first nodes | at t_s = 0.4'
   do m = 0, 2
      do i = 1, 7
         allocate(uu(0:Ns(i)))
         call eigen(lconv(m+1), m, Ns(i), 0.0_dp, lam, uu); e1(i) = abs(lam - lconv(m+1)*(lconv(m+1)+1))
         call eigen(lconv(m+1), m, Ns(i), 0.4_dp, lam, uu); e2(i) = abs(lam - lconv(m+1)*(lconv(m+1)+1))
         write(10,'(i3,i6,3es16.7)') m, Ns(i), pi/Ns(i), e1(i), e2(i)
         deallocate(uu)
      end do
      print '(a,i1,a,7es9.1)', ' m=', m, ' first nodes:', e1
      print '(a,i1,a,7es9.1)', ' m=', m, ' t_s = 0.4  :', e2
   end do
   close(10)
end program ch2
```

2.7 Results, normalisation and verification

Part A.

l	x	Bonnet recurrence	rel. error recurrence	rel. error explicit sum
10	0.30	0.2514763495160157	2.2e-16	4.4e-16
40	0.30	0.1251158458557079	2.2e-16	3.0e-12
60	0.05	−0.1019617243819477	0	8.2e-16
100	0.90	0.1022658205587189	4.1e-16	6.7e+17

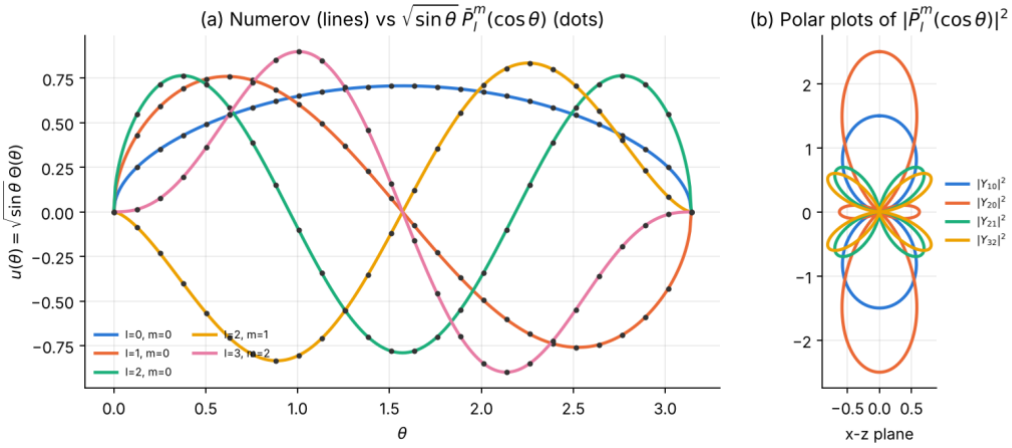
For $l = 100$, $x = 0.9$ the explicit sum returns a number of order 10^{17} instead of 0.10. The normalised recurrence gives $\bar{P}_{200}^{180}(0.5) = 0.07231608361615147$ (relative error 1.1×10^{-14}) where the

unnormalised one overflows. Orthonormality of $\bar{P}_3^3 \ldots \bar{P}_{20}^3$ with a 30-point rule: 7×10^{-16}; agreement with SciPy `lpmv`: 4×10^{-15}.

Part B ($N = 400$, $\theta_s = 0.4$). A scan of $D(\lambda)$ for $m = 1$ finds sign changes near 2.02, 5.97, 11.98, 19.98, 29.98, 41.98 — exactly $l(l+1)$ for $l = 1 \ldots 6$ and no spurious root.

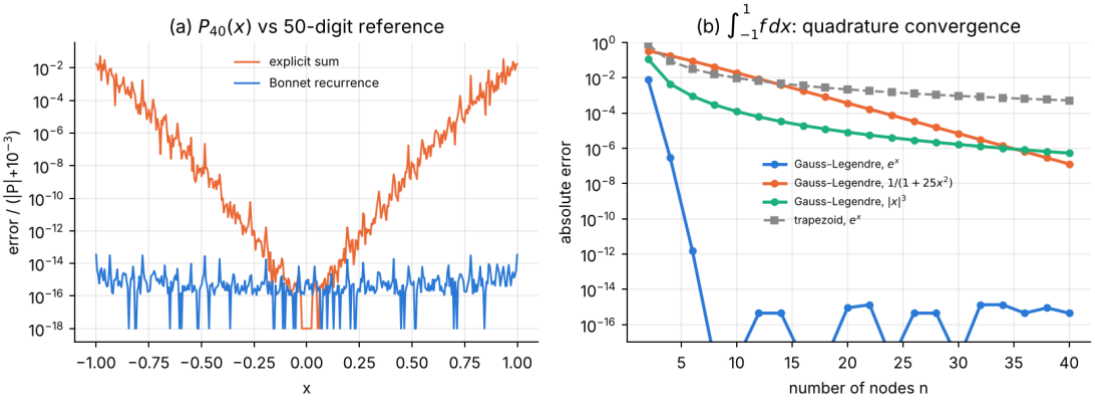
l	m	λ Numerov	l(l+1)	|Δλ|	∫u²dθ	max|u − u_ex|	⟨cos²θ⟩	exact
0	0	−0.000000002115	0	2.1e-09	1.000000000000	7.8e-09	0.3333333	0.3333333
1	0	1.999999994594	2	5.4e-09	1.000000000000	1.4e-08	0.6000000	0.6000000
2	0	5.999999991221	6	8.8e-09	1.000000000000	1.7e-08	0.5238095	0.5238095
1	1	2.000000000168	2	1.7e-10	1.000000000000	1.2e-10	0.2000000	0.2000000
2	1	5.999999998116	6	1.9e-09	1.000000000000	4.5e-10	0.4285714	0.4285714
3	1	11.999999979338	12	2.1e-08	1.000000000000	1.2e-09	0.4666667	0.4666667
2	2	5.999999999185	6	8.1e-10	1.000000000000	5.6e-11	0.1428571	0.1428571
3	2	11.999999989533	12	1.0e-08	1.000000000000	4.0e-10	0.3333333	0.3333333
5	3	29.999999892684	30	1.1e-07	1.000000000000	4.7e-09	0.3504273	0.3504274

Orthogonality for $m = 1$: 7×10^{-10}; parity $(-1)^{l+m}$ satisfied to 8×10^{-12}.



(a) Numerov solutions $u(\theta)$ and the exact $\sqrt{sin\theta}\,\bar{P}_l^m(cos\theta)$. (b) Polar diagrams of $|\bar{P}_l^m(cos\theta)|^2$ (the familiar orbital shapes).

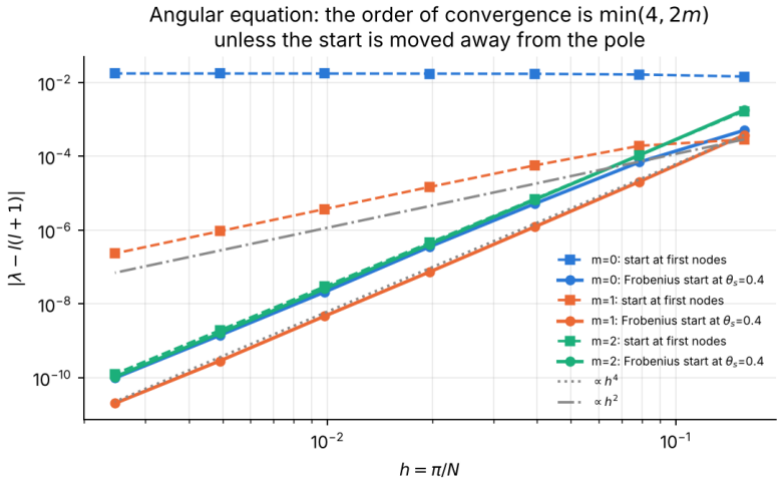
2.8 Error analysis and convergence



(a) $P_{40}(x)$: the explicit sum loses accuracy away from $x = 0$, the Bonnet recurrence does not. (b) Gauss–Legendre converges exponentially for e^x (machine precision with 8 nodes) and for the Runge function $1/(1+25x^2)$ (more slowly, because of the poles at $\pm i/5$), algebraically for $|x|^3$; the trapezoid rule converges only as n^{-2}.

N	h	m=0 first nodes	m=0 θ_s=0.4	m=1 first nodes	m=1 θ_s=0.4	m=2 first nodes	m=2 θ_s=0.4

N	h	m=0 first nodes	m=0 θ_s=0.4	m=1 first nodes	m=1 θ_s=0.4	m=2 first nodes	m=2 θ_s=0.4
20	0.157	1.44e-02	5.01e-04	2.84e-04	3.74e-04	1.65e-03	1.78e-03
40	0.0785	1.63e-02	6.88e-05	1.90e-04	2.03e-05	1.06e-04	1.06e-04
80	0.0393	1.70e-02	5.10e-06	5.60e-05	1.21e-06	6.93e-06	6.56e-06
160	0.0196	1.72e-02	3.49e-07	1.46e-05	7.30e-08	4.53e-07	4.09e-07
320	0.00982	1.73e-02	2.08e-08	3.69e-06	4.64e-09	2.95e-08	2.56e-08
640	0.00491	1.73e-02	1.40e-09	9.26e-07	2.83e-10	1.92e-09	1.59e-09
1280	0.00245	1.74e-02	9.75e-11	2.32e-07	2.00e-11	1.26e-10	1.04e-10
order		**0.0**	**3.9**	**1.9**	**4.0**	**3.9**	**4.0**



The start fault in the angular equation: with the start at the first grid nodes the order is $\min(4,2m)$ — no convergence at all for $m = 0$; the Frobenius start at fixed θ_s gives order 4 for every m.

This table is the clearest illustration of the start fault of Part 0: for $m = 0$ the error is 1.7×10^{-2} *whatever the step* — a calculation that looks perfectly converged in h and is wrong in the second digit. Only a comparison with the exact result or a change of start strategy reveals it.

2.9 Exercises

1. **Gauss–Legendre exactness.** Show that the n-point rule is exact for degree $2n - 1$; verify that the 30-point rule integrates x^{58} exactly. *Answer:* error 3.5×10^{-17}.

2. **Addition theorem.** Verify numerically $\sum_{m=-l}^{l} |Y_{lm}(\theta, \phi)|^2 = (2l + 1)/4\pi$ for $l = 5$, $\theta = 0.7$ using `plm_bar`. *Answer:* 0.8753521870054248 vs 0.8753521870054244.

3. **Why does the Runge function converge slowly?** Relate the convergence rate of Gauss–Legendre for $1/(1 + 25x^2)$ to the Bernstein ellipse through the poles $\pm i/5$, and estimate the rate ρ^{-2n} with $\rho = 0.2 + \sqrt{1.04} = 1.2198$. *Answer:* $\rho^{-80} = 1.3 \times 10^{-7}$, in agreement with the computed error 1.3×10^{-7} at $n = 40$.

4. **Rotor in an electric field (Stark effect / hindered rotor).** Add $\beta\cos\theta$ to the angular Hamiltonian ($m = 0$): $u'' + [\lambda + 1/4 + 1/4\sin^2\theta - \beta\cos\theta]u = 0$. (a) Show that second-order perturbation theory gives $\lambda_0 \approx -\beta^2/6$. (b) Compute λ_0 for $\beta = 0.1$ and 1. (c) **Trap:** the Frobenius series must now include the expansion of $\cos\theta$ about each pole (different signs at the two poles); a series that ignores it gives a wrong answer that converges perfectly in h. *Answer:* $\lambda_0 = -0.0016656495$ ($\beta = 0.1$; perturbation theory -0.0016667), $\lambda_0 = -0.157663483$ ($\beta = 1$), confirmed by diagonalising the Hamiltonian in the basis $\bar{P}_l^0$, $l < 40$, where $\langle l|\cos\theta|l + 1\rangle = (l + 1)/\sqrt{(2l + 1)(2l + 3)}$.

5. **The $m = 0$ fault, theory.** Show that for $m = 0$ the irregular solution behaves as $\theta^{1/2}\ln\theta$ and explain why the eigenvalue error does not tend to zero: the ratio of regular to irregular solution at $\theta \approx h$ is $1/|\ln h|$ instead of a positive power of h, and the contamination amplitude also involves the h-independent defect δ. The table shows that the error in fact settles at a constant, $\approx 1.7 \times 10^{-2}$.

Chapter 3 — Cylindrical Bessel functions and the circular well

3.1 The function

Bessel's equation of order ν (DLMF §10.2; Watson 1944, ch. 3)

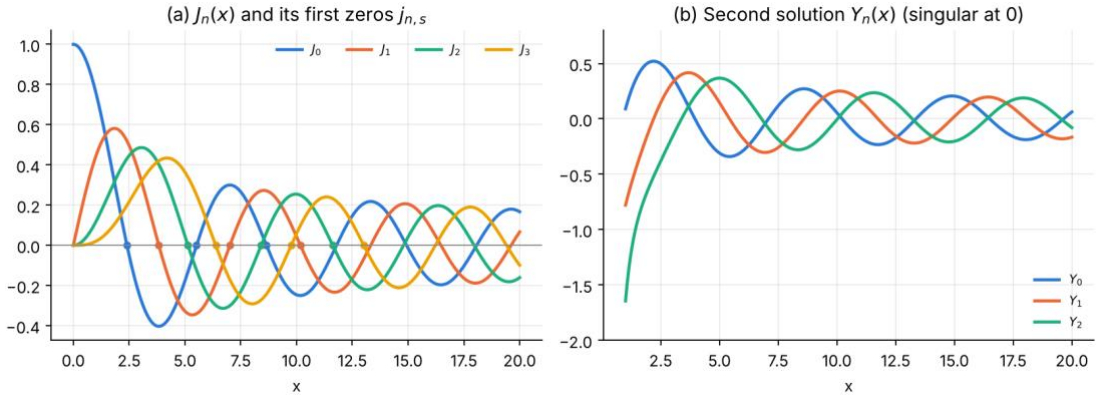
$$x^2y'' + x\,y' + (x^2 - \nu^2)\,y = 0$$

has the solution regular at the origin

$$J_\nu(x) = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!\;\Gamma(k+\nu+1)} \left(\frac{x}{2}\right)^{2k+\nu}$$

and a second solution Y_ν that is singular at $x = 0$ ($Y_0 \sim 2/\pi \ln x$, $Y_n \sim -(n-1)!/\pi\,(2/x)^n$). For integer n (DLMF §10.6, §10.9, §10.12, §10.17, §10.21; Watson 1944; Arfken et al. 2013, ch. 14):

- Recurrences: $J_{n-1} + J_{n+1} = 2n/x\,J_n,\;\; J_{n-1} - J_{n+1} = 2J_n{}'$.
- Integral representation (Bessel): $J_n(x) = \frac{1}{\pi}\int_0^\pi \cos(n\tau - x\sin\tau)\,d\tau$.
- Generating function and Jacobi–Anger expansion: $e^{ix\sin\phi} = \sum_n J_n(x)e^{in\phi}$, hence $1 = J_0 + 2\sum_{k\geq 1} J_{2k}$ and $\cos x = J_0 - 2J_2 + 2J_4 - \cdots$
- Asymptotics: $J_n(x) \approx \sqrt{2/\pi x}\cos(x - n\pi/2 - \pi/4)$ for $x \gg n$; $J_n(x) \approx (x/2)^n/n!$ for $x \ll \sqrt{n+1}$.
- Zeros $j_{n,s}$ (McMahon): $j_{n,s} \approx \beta - \frac{\mu-1}{8\beta} - \frac{4(\mu-1)(7\mu-31)}{3(8\beta)^3}$, $\beta = (s + n/2 - 1/4)\pi$, $\mu = 4n^2$.
- Orthogonality on $[0,1]$: $\int_0^1 J_n(j_{n,s}r)J_n(j_{n,s'}r)\,r\,dr = 1/2\,J_{n+1}(j_{n,s})^2\,\delta_{ss'}$.



J_n with its first zeros, and the singular second solution Y_n.

3.2 The physics: circular infinite well

A particle confined to a disc of radius $a = 1$ (a "quantum corral", a cylindrical waveguide, a drum) obeys $-1/2\,\nabla^2\psi = E\psi$ with $\psi = 0$ at $r = 1$. Separation $\psi = R(r)e^{im\phi}$ gives Bessel's equation in kr, $k = \sqrt{2E}$; regularity at the origin selects J_m and the boundary condition gives

$$E_{m,s} = \frac{j_{m,s}^2}{2}, \qquad R_{m,s}(r) = \frac{\sqrt{2}\,J_m(j_{m,s}r)}{|J_{m+1}(j_{m,s})|} \quad \left(\int_0^1 R^2\;r\,dr = 1\right).$$

States with $m \neq 0$ are doubly degenerate ($\pm m$). Exact check: $\langle r^2 \rangle = 1/3 + 2(m^2 - 1)/3j_{m,s}^2$ (follows from the Bessel-function integrals of Watson 1944, §5.11).

3.3 Analytical scheme

Part A — computing $J_n(x)$.

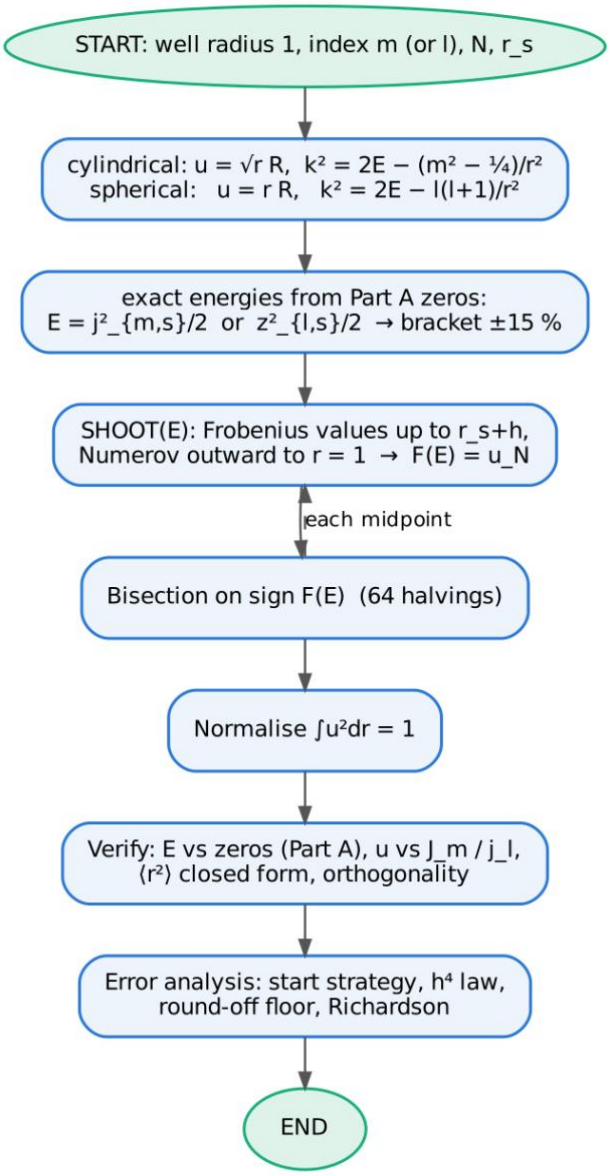
1. Power series — accurate for small x, but its terms reach $\sim e^x/\sqrt{x}$ while the sum is $O\left(x^{-1/2}\right)$: cancellation for large x.
2. Forward recurrence from J_0, J_1 — **unstable** for $n > x$ (J_n is minimal).
3. **Miller's algorithm** (recommended): choose even $M > \max(n, x) + \sqrt{64\max(n, x)} + 10$, set $f_{M+1} = 0$, $f_M = 10^{-30}$, recur downwards with rescaling, normalise by $f_0 + 2\sum f_{2k}$.
4. Trapezoid rule on the Bessel integral: the integrand is smooth and periodic, so the trapezoid rule converges **geometrically** once the number of panels exceeds $\approx x + n$.
5. Zeros: McMahon start, Newton with $J_n' = (J_{n-1} - J_{n+1})/2$.

Part B — Numerov. $u = \sqrt{r}\, R$:

$$u'' + \left[2E - \frac{m^2 - 1/4}{r^2}\right] u = 0, \quad u(1) = 0, \quad u = r^{m+1/2} \sum_k a_k\, r^{2k}, \;\; 2k(2k + 2m)\, a_k = -2E\, a_{k-1}.$$

Outward shooting from the Frobenius values on $[0, r_s + h]$ ($r_s = 1/8$, or the first two nodes for comparison) to $r = 1$; bisection on the sign of $F(E) = u_N(E)$ in $[0.85, 1.15] \times j_{m,s}^2/2$, with the zeros from Part A.

3.4 Logic chart



Logic chart common to Chapters 3 and 4 (Part B).

3.5 Python 3 code

File: `ch3_bessel_circular.py`

```
#!/usr/bin/env python3
"""
Chapter 3 -- BESSEL FUNCTIONS J_n (cylindrical) and the CIRCULAR INFINITE WELL
================================================================================
Part A : computation of J_n(x)
         * power series                                  DLMF §10.2  (cancellation)
         * forward recurrence  J_{n+1} = (2n/x) J_n - J_{n-1}   DLMF §10.6 (unstable n > x)
         * Miller backward recurrence + normalisation
           J_0 + 2 sum J_{2k} = 1                        DLMF §3.6(iii), §10.12
         * trapezoid on the Bessel integral              DLMF §10.9 (geometric convergence)
         * zeros j_{n,s}: McMahon asymptotics + Newton   DLMF §10.21
         * verification: mpmath 50-digit reference, Jacobi-Anger identity
           cos x = J_0 - 2 J_2 + 2 J_4 - ...  (DLMF §10.12), scipy.special.jv
Part B : particle in a circular infinite well of radius 1 (hbar = m = 1)
         psi = R(r) e^{i m phi},  u = sqrt(r) R :
         u'' + [ 2E - (m^2 - 1/4)/r^2 ] u = 0 ,  u(1) = 0 ,  u ~ r^{m+1/2}
         exact  E_{m,s} = j_{m,s}^2 / 2 ,  R = J_m(j r) sqrt(2)/|J_{m+1}(j)|
         Numerov outward shooting with Frobenius start at r_s, bisection on u(1)
         * verification: normalisation int R^2 r dr = 1, comparison with J_m,
           orthogonality, energies vs zeros computed in Part A
```

```
        * error analysis: start at first nodes (order 2m, no convergence for m=0)
          vs fixed r_s (order 4); Richardson
Outputs: tables + ch3_functions.png, ch3_partA_errors.png, ch3_well.png, ch3_convergence.png
"""
import numpy as np
from math import factorial, lgamma, exp
import mpmath as mp
from scipy.special import jv
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#7a5cd6"]
mp.mp.dps = 50


def style(ax):
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")


# ============================================================ PART A
def J_series(n, x, kmax=200):
    """J_n(x) = sum_k (-1)^k (x/2)^{2k+n} / (k! (k+n)!)"""
    x = float(x); s = 0.0
    term = (x / 2) ** n / factorial(n)
    for k in range(kmax):
        s += term
        term *= -(x / 2) ** 2 / ((k + 1) * (k + 1 + n))
        if abs(term) < 1e-17 * abs(s) and k > x: break
    return s


def J_forward(nmax, x, J0, J1):
    J = np.zeros(nmax + 1); J[0], J[1] = J0, J1
    for n in range(1, nmax):
        J[n + 1] = 2 * n / x * J[n] - J[n - 1]
    return J


def miller_start(nmax, x, digits=16):
    """starting index for Miller's algorithm (Numerical Recipes style + margin)"""
    return 2 * ((max(nmax, int(x)) + int(np.sqrt(digits * 4 * max(nmax, int(x), 1))) + 10) // 2)


def J_miller(nmax, x, M=None):
    """J_0..J_nmax(x) by backward recurrence from M (even), normalised with
       J_0 + 2 (J_2 + J_4 + ...) = 1.  Valid for x > 0."""
    if M is None: M = miller_start(nmax, x)
    M += M % 2
    f = np.zeros(M + 2); f[M + 1] = 0.0; f[M] = 1e-30
    for n in range(M, 0, -1):
        f[n - 1] = 2 * n / x * f[n] - f[n + 1]
        if abs(f[n - 1]) > 1e250:                       # rescale to avoid overflow
            f[n - 1:] *= 1e-250
    norm = f[0] + 2 * f[2:M + 1:2].sum()
    return f[:nmax + 1] / norm


def J_trapezoid(n, x, K):
    """J_n(x) = (1/pi) int_0^pi cos(n t - x sin t) dt with K-panel trapezoid"""
    t = np.linspace(0, np.pi, K + 1)
    f = np.cos(n * t - x * np.sin(t))
    return (f.sum() - 0.5 * (f[0] + f[-1])) / K


def J(n, x):
    """recommended: Miller for x>0, J_n(0)=delta"""
    if x == 0: return 1.0 if n == 0 else 0.0
    return J_miller(n + 1, x)[n]


def dJ(n, x):
    Jm = J_miller(n + 2, x)
    return (Jm[n - 1] - Jm[n + 1]) / 2 if n > 0 else -Jm[1]


def bessel_zeros(n, S):
    """first S positive zeros of J_n: McMahon start, Newton refinement"""
    z = []
    mu = 4 * n * n
    for s in range(1, S + 1):
        b = (s + n / 2 - 0.25) * np.pi
```

```
        x = b - (mu - 1) / (8 * b) - 4 * (mu - 1) * (7 * mu - 31) / (3 * (8 * b) ** 3)
        for _ in range(50):
            Jm = J_miller(n + 2, x)
            d = (Jm[n - 1] - Jm[n + 1]) / 2 if n > 0 else -Jm[1]
            dx = Jm[n] / d; x -= dx
            if abs(dx) < 1e-15 * x: break
        z.append(x)
    return np.array(z)


def partA():
    print("=" * 78)
    print("PART A  Bessel functions of the first kind J_n(x)")
    print("=" * 78)
    print("J_n(x): series / Miller / mpmath")
    print(f"{'n':>3} {'x':>6} {'Miller':>24} {'mpmath':>24} {'rel.err Miller':>15} {'rel.err series':>15}")
    for n, x in ((0, 1.0), (5, 10.0), (0, 30.0), (3, 50.0), (20, 5.0), (50, 1.0)):
        ref = float(mp.besselj(n, x))
        jm = J(n, x); js = J_series(n, x)
        print(f"{n:3d} {x:6.1f} {jm:24.15e} {ref:24.15e} {abs(jm-ref)/abs(ref):15.2e} {abs(js-
ref)/abs(ref):15.2e}")
    # forward recurrence instability at x = 1
    x = 1.0
    Jm = J_miller(30, x)
    Jf = J_forward(30, x, Jm[0], Jm[1])
    ref = np.array([float(mp.besselj(n, x)) for n in range(31)])
    print("\nForward vs backward recurrence at x = 1 (relative error):")
    for n in (2, 5, 10, 15, 20, 25, 30):
        print(f"  n={n:2d}: forward {abs(Jf[n]-ref[n])/abs(ref[n]):9.2e}   Miller {abs(Jm[n]-
ref[n])/abs(ref[n]):9.2e}   J_n = {ref[n]:.3e}")
    # Jacobi-Anger identity check
    for x in (0.5, 7.3, 25.0):
        Jm = J_miller(80, x)
        s = Jm[0] + 2 * sum((-1) ** k * Jm[2 * k] for k in range(1, 40))
        print(f"  Jacobi-Anger x={x:5.1f}: J0 - 2J2 + 2J4 - ... - cos x = {s-np.cos(x):.1e}")
    xs = np.linspace(0.1, 40, 50)
    print(f"  max |J_miller - scipy jv| for n<=10, 0.1<x<40: "
          f"{max(np.max(abs(J_miller(10, x) - jv(np.arange(11), x))) for x in xs):.1e}")
    z = {n: bessel_zeros(n, 5) for n in range(4)}
    for n in range(4):
        refz = [float(mp.besseljzero(n, s)) for s in range(1, 6)]
        print(f"  zeros J_{n}: " + ", ".join(f"{v:.12f}" for v in z[n]) + f"   max err {max(abs(z[n]-refz)):.1e}")

    # error-analysis data
    xx = np.linspace(0.5, 60, 240)
    ref0 = np.array([float(mp.besselj(0, x)) for x in xx])
    e_ser = [abs(J_series(0, x) - r) for x, r in zip(xx, ref0)]
    e_mil = [abs(J(0, x) - r) for x, r in zip(xx, ref0)]
    Ms = np.arange(2, 62, 2)
    x = 10.0; r5 = float(mp.besselj(5, x))
    e_M = [abs(J_miller(5, x, M=M)[5] - r5) / abs(r5) if M > 5 else np.nan for M in Ms]
    Ks = np.arange(2, 41, 1)
    e_K = [abs(J_trapezoid(5, x, K) - r5) / abs(r5) + 1e-18 for K in Ks]
    nn = np.arange(31)
    e_f = abs(Jf - ref) / abs(ref); e_b = abs(J_miller(30, 1.0) - ref) / abs(ref)
    print("\nMiller start index M (x=10, n=5): " + "  ".join(f"M={M}:{e:.1e}" for M, e in zip(Ms, e_M) if M in
(10, 16, 20, 26, 30, 40)))
    print("Trapezoid panels K (x=10, n=5):   " + "  ".join(f"K={K}:{e:.1e}" for K, e in zip(Ks, e_K) if K in (5,
10, 15, 20, 25, 30)))

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    xg = np.linspace(0.001, 20, 600)
    for n in range(4):
        ax[0].plot(xg, [J(n, v) for v in xg], color=C[n], lw=2, label=f"$J_{n}$")
        ax[0].plot(z[n][:3], 0 * z[n][:3], "o", ms=4, color=C[n])
    ax[0].axhline(0, color="#888888", lw=.8)
    ax[0].set(xlabel="x", title=r"(a) $J_n(x)$ and its first zeros $j_{n,s}$")
    ax[0].legend(frameon=False, fontsize=8, ncol=4)
    from scipy.special import yv
    for n in range(3):
        ax[1].plot(xg[30:], yv(n, xg[30:]), color=C[n], lw=2, label=f"$Y_{n}$")
    ax[1].set(ylim=(-2, 0.8), xlabel="x", title=r"(b) Second solution $Y_n(x)$ (singular at 0)")
    ax[1].legend(frameon=False, fontsize=8)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch3_functions.png", dpi=170)

    fig, ax = plt.subplots(1, 3, figsize=(13, 3.7))
    ax[0].semilogy(xx, np.array(e_ser) + 1e-18, color=C[1], lw=1.5, label="power series")
    ax[0].semilogy(xx, np.array(e_mil) + 1e-18, color=C[0], lw=1.5, label="Miller")
    ax[0].set(xlabel="x", ylabel="absolute error", title=r"(a) $J_0(x)$: series cancellation")
    ax[0].legend(frameon=False, fontsize=8)
    ax[1].semilogy(nn[1:], e_f[1:] + 1e-18, "s-", color=C[1], lw=1.5, ms=4, label="forward recurrence")
```

```
    ax[1].semilogy(nn[1:], e_b[1:] + 1e-18, "o-", color=C[0], lw=1.5, ms=4, label="backward (Miller)")
    ax[1].set(xlabel="order n", ylabel="relative error", title="(b) Recurrence direction, x = 1")
    ax[1].legend(frameon=False, fontsize=8)
    ax[2].semilogy(Ms, e_M, "o-", color=C[0], lw=1.5, ms=4, label="Miller, start index M")
    ax[2].semilogy(Ks, e_K, "s-", color=C[2], lw=1.5, ms=4, label="trapezoid, K panels")
    ax[2].set(xlabel="M or K", ylabel="relative error", title=r"(c) Convergence for $J_5(10)$")
    ax[2].legend(frameon=False, fontsize=8)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch3_partA_errors.png", dpi=170)
    return z


# ============================================================ PART B
def frobenius(r, m, E, nterm=40):
    """u = r^{m+1/2} sum a_k r^{2k},  2k(2k+2m) a_k = -2E a_{k-1}"""
    r = np.asarray(r, float)
    s = np.zeros_like(r); a = 1.0; p = np.ones_like(r)
    for k in range(nterm):
        s += a * p
        a = -2 * E * a / ((2 * k + 2) * (2 * k + 2 + 2 * m)); p = p * r * r
    return r ** (m + 0.5) * s


def simpson(y, h):
    return h / 3 * (y[0] + y[-1] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum())


def shoot(E, m, N, rs):
    h = 1.0 / N; r = h * np.arange(N + 1)
    k2 = np.zeros(N + 1); k2[1:] = 2 * E - (m * m - 0.25) / r[1:] ** 2
    c = 1 + h * h * k2 / 12
    js = max(1, int(rs / h + 0.5))
    u = np.zeros(N + 1); u[:js + 2] = frobenius(r[:js + 2], m, E)
    for n in range(js + 1, N):
        u[n + 1] = ((12 - 10 * c[n]) * u[n] - c[n - 1] * u[n - 1]) / c[n + 1]
    return u, r


def eigen(m, s, N, rs=0.125, zeros=None):
    Ex = zeros[m][s - 1] ** 2 / 2
    a, b = Ex * 0.85, Ex * 1.15
    fa = shoot(a, m, N, rs)[0][-1]
    for _ in range(64):
        mid = 0.5 * (a + b); fm = shoot(mid, m, N, rs)[0][-1]
        if np.sign(fm) == np.sign(fa): a, fa = mid, fm
        else: b = mid
    E = 0.5 * (a + b)
    u, r = shoot(E, m, N, rs)
    u /= np.sqrt(simpson(u * u, r[1]))
    j = zeros[m][s - 1]
    ex = np.sqrt(r) * np.array([J(m, j * v) for v in r]) * np.sqrt(2) / abs(J(m + 1, j))
    if np.dot(u, ex) < 0: u = -u
    return E, u, r, ex


def partB(z):
    print("\n" + "=" * 78)
    print("PART B  Circular infinite well, radius 1:  Numerov in u = sqrt(r) R,  N = 400, r_s = 0.125")
    print("=" * 78)
    # scan, m = 0
    Eg = np.linspace(0.5, 80, 800)
    F = np.array([shoot(E, 0, 400, 0.125)[0][-1] for E in Eg])
    roots = [0.5 * (Eg[i] + Eg[i + 1]) for i in np.where(np.sign(F[:-1]) != np.sign(F[1:]))[0]]
    print("  scan m=0: sign changes of u(1;E) near " + ", ".join(f"{v:.2f}" for v in roots)
          + "   (exact j_{0,s}^2/2 = " + ", ".join(f"{v**2/2:.2f}" for v in z[0][:4]) + ")")
    print(f"\n{'m':>2} {'s':>2} {'E Numerov':>17} {'j^2/2 (Part A)':>17} {'|dE|':>9} {'norm':>15} {'max|u-
u_ex|':>12} {'<r^2>':>10} {'exact':>10}")
    store = {}
    for (m, s) in ((0, 1), (0, 2), (0, 3), (1, 1), (1, 2), (2, 1), (3, 1)):
        E, u, r, ex = eigen(m, s, 400, zeros=z)
        Ex = z[m][s - 1] ** 2 / 2
        store[m, s] = (u, r, ex)
        print(f"{m:2d} {s:2d} {E:17.12f} {Ex:17.12f} {abs(E-Ex):9.1e} {simpson(u*u,r[1]):15.12f} {np.max(abs(u-
ex)):12.1e} {simpson(u*u*r*r, r[1]):10.6f} {1/3+2*(m*m-1)/(3*z[m][s-1]**2):10.6f}")
    ov = max(abs(simpson(store[0, a][0] * store[0, b][0], 1 / 400)) for a in (1, 2, 3) for b in (1, 2, 3) if a <
b)
    print(f"  orthogonality (m=0, s=1,2,3): max |<u_s|u_s'>| = {ov:.1e}")

    Ns = np.array([20, 40, 80, 160, 320, 640, 1280])
    hs = 1.0 / Ns
    err = {}
    for m in (0, 1, 2):
```

```
        Ex = z[m][0] ** 2 / 2
        for rs, lab in ((0.0, "first nodes"), (0.125, "r_s")):
            err[m, lab] = [abs(eigen(m, 1, N, rs, z)[0] - Ex) for N in Ns]
    print("\nError of E_{m,1} vs h: Frobenius start at first nodes  vs  at fixed r_s = 0.125")
    print(f"{'N':>6} {'h':>9}" + "".join(f" {'m=%d nodes' % m:>12} {'m=%d r_s' % m:>11}" for m in (0, 1, 2)))
    for i, N in enumerate(Ns):
        print(f"{N:6d} {hs[i]:9.6f}" + "".join(f" {err[m,'first nodes'][i]:12.2e} {err[m,'r_s'][i]:11.2e}" for m
in (0, 1, 2)))
    for m in (0, 1, 2):
        for lab in ("first nodes", "r_s"):
            e = np.array(err[m, lab]); ok = e > 1e-12
            print(f"  m={m} {lab:12s}: observed order {np.polyfit(np.log(hs[ok][1:6]), np.log(e[ok][1:6]),
1)[0]:.2f}")
    E1 = eigen(0, 1, 160, 0.125, z)[0]; E2 = eigen(0, 1, 320, 0.125, z)[0]
    ER = (16 * E2 - E1) / 15
    print(f"Richardson m=0: E(1/160)={E1:.12f} E(1/320)={E2:.12f} E_R={ER:.12f} err={abs(ER - z[0][0]**2/2):.1e}")

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.9))
    for i, (m, s) in enumerate(((0, 1), (0, 2), (1, 1), (2, 1), (1, 2))):
        u, r, ex = store[m, s]
        R = np.zeros_like(u); R[1:] = u[1:] / np.sqrt(r[1:]); R[0] = R[1] if m == 0 else 0
        ax[0].plot(r, R, color=C[i], lw=2, label=f"m={m}, s={s}")
        ax[0].plot(r[1::16], ex[1::16] / np.sqrt(r[1::16]), "o", ms=2.5, color="#333333")
    ax[0].set(xlabel="r", ylabel="R(r)", title=r"(a) Radial functions (dots: $\sqrt{2}J_m(jr)/|J_{m+1}(j)|$)")
    ax[0].legend(frameon=False, fontsize=7, ncol=2)
    # 2D density of (m=1,s=2) cos(phi) standing wave
    xg = np.linspace(-1, 1, 241); X, Y = np.meshgrid(xg, xg)
    Rr = np.hypot(X, Y); Ph = np.arctan2(Y, X)
    j = z[1][1]
    psi = np.where(Rr <= 1, jv(1, j * Rr) * np.cos(Ph), np.nan)
    im = ax[1].imshow(psi ** 2, extent=(-1, 1, -1, 1), origin="lower", cmap="Blues")
    ax[1].set(title=r"(b) $|\psi|^2$ for m=1, s=2  ($\cos\phi$ standing wave)", xlabel="x", ylabel="y")
    style(ax[0])
    fig.tight_layout(); fig.savefig("ch3_well.png", dpi=170)

    fig, ax = plt.subplots(figsize=(6.6, 4.2))
    for i, m in enumerate((0, 1, 2)):
        ax.loglog(hs, err[m, "first nodes"], "s--", color=C[i], lw=1.5, ms=5, label=f"m={m}: start at first
nodes")
        ax.loglog(hs, err[m, "r_s"], "o-", color=C[i], lw=2, ms=5, label=fr"m={m}: Frobenius start at $r_s$=1/8")
    ax.loglog(hs, err[0, "r_s"][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax.loglog(hs, err[1, "first nodes"][0] * (hs / hs[0]) ** 2, "-.", color="#888888", label=r"$\propto h^2$")
    ax.set(xlabel="h = 1/N", ylabel=r"$|E_{m,1}-j_{m,1}^2/2|$", title="Circular well: the start fault (order
$\\min(4,2m)$)\nand its cure")
    ax.legend(frameon=False, fontsize=7); style(ax)
    fig.tight_layout(); fig.savefig("ch3_convergence.png", dpi=170)


if __name__ == "__main__":
    z = partA(); partB(z)
```

3.6 Fortran 2008 code

File: `ch3_bessel_circular.f90`

```
!=======================================================================
! Chapter 3 -- BESSEL FUNCTIONS J_n and the CIRCULAR INFINITE WELL (F2008)
! Part A: J_n(x) by Miller's backward recurrence (normalised with
!         J0 + 2 sum J_2k = 1), forward recurrence, power series,
!         zeros by McMahon + Newton; independent check with the Fortran
!         2008 intrinsic bessel_jn and the Jacobi-Anger identity
! Part B: u = sqrt(r) R,  u'' + [2E - (m^2-1/4)/r^2] u = 0,  u(1) = 0
!         Numerov outward shooting + bisection, Frobenius start
! Output: screen tables + ch3_f_conv.dat
! Compile: gfortran -O2 -o ch3 ch3_bessel_circular.f90 && ./ch3
!=======================================================================
module bessel_mod
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.14159265358979323_dp
contains
  ! J_0..J_nmax(x), x > 0, Miller backward recurrence from even index M
  subroutine j_miller(nmax, x, Jv, Mstart)
    integer,  intent(in)  :: nmax
    real(dp), intent(in)  :: x
    real(dp), intent(out) :: Jv(0:nmax)
    integer,  intent(in), optional :: Mstart
    real(dp), allocatable :: f(:)
    real(dp) :: nrm
    integer  :: M, k
```

```
    if (present(Mstart)) then
       M = Mstart
    else
       M = 2*((max(nmax, int(x)) + int(sqrt(64.0_dp*max(nmax, int(x), 1))) + 10)/2)
    end if
    M = M + mod(M, 2)
    allocate(f(0:M+1))
    f = 0.0_dp; f(M) = 1.0d-30
    do k = M, 1, -1
       f(k-1) = 2*k/x*f(k) - f(k+1)
       if (abs(f(k-1)) > 1.0d250) f(k-1:M+1) = f(k-1:M+1)*1.0d-250
    end do
    nrm = f(0) + 2*sum(f(2:M:2))
    Jv = 0.0_dp
    Jv(0:min(nmax, M)) = f(0:min(nmax, M))/nrm
  end subroutine j_miller

  real(dp) function J(n, x)
    integer,  intent(in) :: n
    real(dp), intent(in) :: x
    real(dp) :: Jv(0:n+1)
    if (x == 0.0_dp) then
       J = merge(1.0_dp, 0.0_dp, n == 0); return
    end if
    call j_miller(n+1, x, Jv); J = Jv(n)
  end function J

  real(dp) function j_series(n, x)
    integer,  intent(in) :: n
    real(dp), intent(in) :: x
    real(dp) :: term
    integer  :: k
    term = (x/2)**n/gamma(n + 1.0_dp); j_series = 0.0_dp
    do k = 0, 300
       j_series = j_series + term
       term = -term*(x/2)**2/((k + 1)*(k + 1 + n))
       if (abs(term) < 1.0d-17*abs(j_series) .and. k > x) exit
    end do
  end function j_series

  subroutine bessel_zeros(n, S, z)
    integer,  intent(in)  :: n, S
    real(dp), intent(out) :: z(S)
    real(dp) :: bb, mu, x, dx, Jv(0:n+2), d
    integer  :: k, it
    mu = 4.0_dp*n*n
    do k = 1, S
       bb = (k + n/2.0_dp - 0.25_dp)*pi
       x = bb - (mu - 1)/(8*bb) - 4*(mu - 1)*(7*mu - 31)/(3*(8*bb)**3)
       do it = 1, 50
          call j_miller(n+2, x, Jv)
          if (n > 0) then
             d = (Jv(n-1) - Jv(n+1))/2
          else
             d = -Jv(1)
          end if
          dx = Jv(n)/d; x = x - dx
          if (abs(dx) < 1.0d-15*x) exit
       end do
       z(k) = x
    end do
  end subroutine bessel_zeros
end module bessel_mod

module well_mod
  use bessel_mod
  implicit none
contains
  real(dp) function frob(r, m, E)
    real(dp), intent(in) :: r, E
    integer,  intent(in) :: m
    real(dp) :: a, s, p
    integer  :: k
    s = 0.0_dp; a = 1.0_dp; p = 1.0_dp
    do k = 0, 39
       s = s + a*p
       a = -2*E*a/((2*k + 2)*(2*k + 2 + 2*m)); p = p*r*r
    end do
    frob = r**(m + 0.5_dp)*s
  end function frob

  real(dp) function simpson(y, h, n)
```

```
    integer,  intent(in) :: n
    real(dp), intent(in) :: y(0:n), h
    simpson = h/3*(y(0) + y(n) + 4*sum(y(1:n-1:2)) + 2*sum(y(2:n-2:2)))
  end function simpson

  subroutine shoot(E, m, N, rs, u)
    real(dp), intent(in)  :: E, rs
    integer,  intent(in)  :: m, N
    real(dp), intent(out) :: u(0:N)
    real(dp) :: h, r(0:N), c(0:N)
    integer  :: j, js
    h = 1.0_dp/N; r = [(j*h, j=0,N)]
    c(0) = 1.0_dp; c(1:N) = 1 + h*h*(2*E - (m*m - 0.25_dp)/r(1:N)**2)/12
    js = max(1, nint(rs/h))
    u = 0.0_dp
    do j = 0, js+1
       u(j) = frob(r(j), m, E)
    end do
    do j = js+1, N-1
       u(j+1) = ((12 - 10*c(j))*u(j) - c(j-1)*u(j-1))/c(j+1)
    end do
  end subroutine shoot

  subroutine eigen(m, Ex, N, rs, E, u)
    integer,  intent(in)  :: m, N
    real(dp), intent(in)  :: Ex, rs
    real(dp), intent(out) :: E, u(0:N)
    real(dp) :: a, b, fa, mid
    integer  :: it
    a = 0.85_dp*Ex; b = 1.15_dp*Ex
    call shoot(a, m, N, rs, u); fa = u(N)
    do it = 1, 64
       mid = 0.5_dp*(a + b)
       call shoot(mid, m, N, rs, u)
       if (sign(1.0_dp, u(N)) == sign(1.0_dp, fa)) then
          a = mid; fa = u(N)
       else
          b = mid
       end if
    end do
    E = 0.5_dp*(a + b)
    call shoot(E, m, N, rs, u)
    u = u/sqrt(simpson(u*u, 1.0_dp/N, N))
  end subroutine eigen
end module well_mod

program ch3
  use bessel_mod
  use well_mod
  implicit none
  integer, parameter :: NN = 400
  real(dp) :: Jv(0:80), Jf(0:30), z(0:3,5), x, s, E, u(0:NN), uex(0:NN), r, Eex, e1(7), e2(7), maxd
  real(dp), allocatable :: uu(:)
  integer  :: i, k, m, n, sidx, Ns(7) = [20, 40, 80, 160, 320, 640, 1280]
  real(dp) :: xja(3) = [0.5_dp, 7.3_dp, 25.0_dp]
  integer  :: mlist(7) = [0,0,0,1,1,2,3], slist(7) = [1,2,3,1,2,1,1]

  print '(a)', repeat('=',78)
  print '(a)', 'PART A  Bessel functions J_n (Fortran); reference = intrinsic bessel_jn'
  print '(a)', repeat('=',78)
  maxd = 0.0_dp
  do i = 1, 400
     x = 0.1_dp*i
     call j_miller(10, x, Jv(0:10))
     do n = 0, 10
        maxd = max(maxd, abs(Jv(n) - bessel_jn(n, x)))
     end do
  end do
  print '(a,es9.2)', ' max |J_miller - bessel_jn| for n<=10, 0.1<=x<=40: ', maxd
  print '(a)', ' power series vs Miller (relative difference):'
  print '(a,2es24.15,es10.2)', '   J_0(30): ', j_series(0, 30.0_dp), J(0, 30.0_dp),
abs(j_series(0,30.0_dp)/J(0,30.0_dp)-1)
  print '(a,2es24.15,es10.2)', '   J_3(50): ', j_series(3, 50.0_dp), J(3, 50.0_dp),
abs(j_series(3,50.0_dp)/J(3,50.0_dp)-1)
  call j_miller(30, 1.0_dp, Jv(0:30))
  Jf(0) = Jv(0); Jf(1) = Jv(1)
  do n = 1, 29
     Jf(n+1) = 2*n/1.0_dp*Jf(n) - Jf(n-1)
  end do
  print '(a)', ' forward recurrence at x = 1, relative error vs Miller:'
  do n = 5, 30, 5
     print '(a,i3,a,es10.2,a,es10.2)', '   n=', n, '  forward ', abs(Jf(n)/Jv(n) - 1), &
```

```
          '   Miller vs bessel_jn ', abs(Jv(n)/bessel_jn(n, 1.0_dp) - 1)
  end do
  do k = 1, 3
     x = xja(k)
     call j_miller(80, x, Jv)
     s = Jv(0)
     do i = 1, 40
        s = s + 2*(-1)**i*Jv(2*i)
     end do
     print '(a,f5.1,a,es9.1)', ' Jacobi-Anger x=', x, ':  J0 - 2J2 + 2J4 - ... - cos x = ', s - cos(x)
  end do
  do m = 0, 3
     call bessel_zeros(m, 5, z(m,:))
     print '(a,i1,a,5f16.12)', ' zeros J_', m, ':', z(m,:)
  end do
  print '(a,4es9.1)', ' check |bessel_jn(m, j_m1)|:', (abs(bessel_jn(m, z(m,1))), m=0,3)

  print '(/a)', repeat('=',78)
  print '(a)', 'PART B  Circular infinite well, Numerov, N = 400, r_s = 0.125'
  print '(a)', repeat('=',78)
  print '(a2,a3,a17,a17,a10,a16,a12,a10,a10)', 'm', 's', 'E Numerov', 'j^2/2', '|dE|', 'norm', 'max|u-ue|',
'<r^2>', 'exact'
  do k = 1, 7
     m = mlist(k); sidx = slist(k); Eex = z(m,sidx)**2/2
     call eigen(m, Eex, NN, 0.125_dp, E, u)
     do i = 0, NN
        r = i/real(NN,dp)
        uex(i) = sqrt(r)*bessel_jn(m, z(m,sidx)*r)*sqrt(2.0_dp)/abs(bessel_jn(m+1, z(m,sidx)))
     end do
     if (sum(u*uex) < 0) u = -u
     print '(i2,i3,2f17.12,es10.1,f16.12,es12.1,2f10.6)', m, sidx, E, Eex, abs(E - Eex), &
          simpson(u*u, 1.0_dp/NN, NN), maxval(abs(u - uex)), &
          simpson(u*u*[((i/real(NN,dp))**2, i=0,NN)], 1.0_dp/NN, NN), 1/3.0_dp + 2*(m*m - 1)/(3*z(m,sidx)**2)
  end do

  open(10, file='ch3_f_conv.dat', status='replace')
  print '(/a)', 'Error of E_{m,1} vs N: start at first nodes | at r_s = 1/8'
  do m = 0, 2
     Eex = z(m,1)**2/2
     do i = 1, 7
        allocate(uu(0:Ns(i)))
        call eigen(m, Eex, Ns(i), 0.0_dp, E, uu);   e1(i) = abs(E - Eex)
        call eigen(m, Eex, Ns(i), 0.125_dp, E, uu); e2(i) = abs(E - Eex)
        write(10,'(i3,i6,3es16.7)') m, Ns(i), 1.0_dp/Ns(i), e1(i), e2(i)
        deallocate(uu)
     end do
     print '(a,i1,a,7es9.1)', ' m=', m, ' first nodes:', e1
     print '(a,i1,a,7es9.1)', ' m=', m, ' r_s = 1/8  :', e2
  end do
  close(10)
end program ch3
```

3.7 Results, normalisation and verification

Part A.

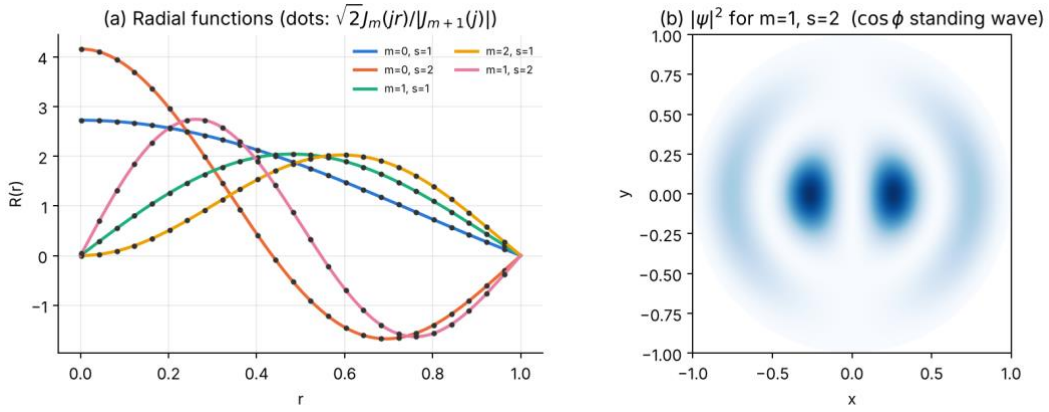
n	x	Miller	rel. error Miller	rel. error series
0	1	0.7651976865579666	0	0
5	10	−0.2340615281867937	3.6e-16	3.9e-14
0	30	−0.08636798358104024	3.2e-16	1.4e-04
3	50	0.09273480406163438	6.0e-16	2.9e+04
20	5	2.770330052128941e-11	3.5e-16	2.3e-16
50	1	2.906004948173239e-80	1.3e-16	1.3e-16

Forward recurrence at $x = 1$ (started from exact J_0, J_1): relative error 1.9×10^{-11} at $n = 5$, 8.2 at $n = 10$, 10^{64} at $n = 30$; Miller stays at 4×10^{-16}. Independent checks: Jacobi–Anger identity satisfied to 4×10^{-16} for $x = 0.5, 7.3, 25$; agreement with SciPy `jv` (5×10^{-16}) and with the Fortran intrinsic `bessel_jn` (3×10^{-16}, $n \le 10$, $0.1 \le x \le 40$). Zeros $j_{n,s}$ ($n \le 3$, $s \le 5$) agree with mpmath to $\le 2 \times 10^{-15}$; e.g. $j_{0,1} = 2.404825557696$, $j_{1,1} = 3.831705970208$.

Part B ($N = 400$, $r_s = 1/8$):

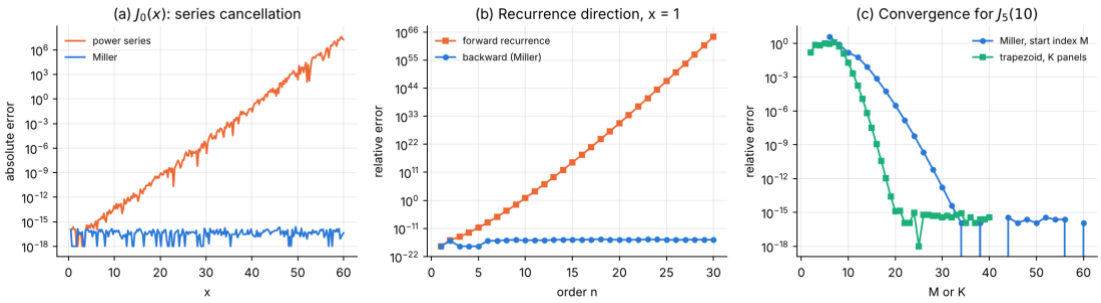
m	s	E Numerov	j²/2 (Part A)	|ΔE|	∫u²dr	max|u − u_ex|	⟨r²⟩	exact
0	1	2.891592973150	2.891592981473	8.3e-09	1.000000000000	1.7e-09	0.218057	0.218057
0	2	15.235631154365	15.235631171831	1.7e-08	1.000000000000	2.2e-09	0.311455	0.311455
0	3	37.443503350309	37.443503395348	4.5e-08	1.000000000000	1.9e-09	0.324431	0.324431
1	1	7.340985321312	7.340985321062	2.5e-10	1.000000000000	1.2e-10	0.333333	0.333333
1	2	24.609228154294	24.609228160847	6.6e-09	1.000000000000	4.3e-10	0.333333	0.333333
2	1	13.187308212838	13.187308213582	7.4e-10	1.000000000000	4.9e-11	0.409164	0.409164
3	1	20.353232907800	20.353232909100	1.3e-09	1.000000000000	1.8e-10	0.464353	0.464353

Orthogonality of the $m = 0$ states: 7.5×10^{-10}. (Note $\langle r^2 \rangle = 1/3$ exactly for every $m = 1$ state — a nice check of the formula.)



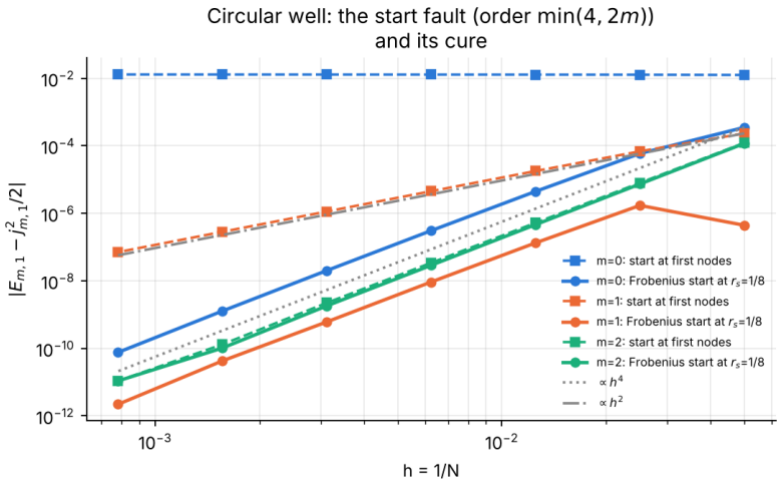
(a) Radial functions $R(r)$ (lines) and the exact $\sqrt{2}J_m(jr)/|J_{m+1}(j)|$ (dots). (b) Probability density of the real standing wave $J_1(j_{1,2}r)\cos\phi$.

3.8 Error analysis and convergence



(a) The power series for J_0 loses accuracy like e^x. (b) Forward recurrence explodes for $n > x$; Miller's backward recurrence does not. (c) Convergence of Miller's algorithm with the start index M and of the trapezoid rule with the number of panels K for $J_5(10)$: both geometric; the trapezoid rule reaches machine precision with $K \approx 20$.

N	m=0 first nodes	m=0 r_s=1/8	m=1 first nodes	m=1 r_s=1/8	m=2 first nodes	m=2 r_s=1/8
20	1.25e-02	3.51e-04	2.34e-04	4.42e-07	1.22e-04	1.17e-04
40	1.27e-02	5.90e-05	6.85e-05	1.71e-06	7.99e-06	7.35e-06
80	1.28e-02	4.44e-06	1.79e-05	1.33e-07	5.23e-07	4.60e-07
160	1.29e-02	3.06e-07	4.51e-06	9.23e-09	3.41e-08	2.88e-08
320	1.29e-02	2.01e-08	1.13e-06	6.07e-10	2.22e-09	1.81e-09
640	1.29e-02	1.27e-09	2.83e-07	4.25e-11	1.31e-10	1.02e-10
1280	1.30e-02	7.57e-11	7.08e-08	2.19e-12	1.08e-11	1.08e-11
order	**0.0**	**3.9**	**2.0**	**3.8**	**4.0**	**4.0**



The start fault for the circular well: identical behaviour to the angular equation (order $\min(4,2m)$), cured by the Frobenius start at a fixed radius.

At $N = 20$ the $m = 1$ entry with $r_s = 1/8$ (4.4×10^{-7}) is accidentally small (errors of opposite sign cancel); the asymptotic h^4 behaviour starts at $N = 40$. **Richardson** ($m = 0$): $E(1/160) = 2.891592675486$, $E(1/320) = 2.891592961368$, $E_R = 2.891592980427$, error 1.0×10^{-9} — a factor 20 better than the finer grid.

3.9 Exercises

1. **McMahon's expansion.** Compare the one-term (β) and two-term approximations with the exact zeros of J_0 for $s = 1,5,20$. *Answer:* one-term errors 4.9×10^{-2}, 8.4×10^{-3}, 2.0×10^{-3}; two-term errors 4.4×10^{-3}, 2.4×10^{-5}, 3.4×10^{-7} — an asymptotic expansion in $1/\beta$.
2. **Miller's starting index.** Find empirically the smallest M that gives $J_5(10)$ to 10^{-13}, and compare with the Numerical-Recipes rule $M \approx 2\left[\left(n + \sqrt{40n}\right)/2\right]$ (Press et al. 2007, §6.5). *Answer from the plot:* $M \approx 30$.
3. **Annular well** $0.5 < r < 1$ (both walls hard), $m = 0$. Show that the eigenvalue condition is $J_0(ka)Y_0(k) - J_0(k)Y_0(ka) = 0$, and that for a thin annulus $k_1 \approx \pi/(1-a)$. Then solve it with Numerov — no special start is needed, since $r = 0$ is excluded. *Answer:* $k_1 = 6.246061839$, $E = 19.5066442$ (thin-annulus estimate $k = 6.283$).
4. **Fault theory.** For $m = 1$, show that the ratio of the regular to the irregular solution at $r = h$ is h^2, and explain the observed order 2.
5. **Degeneracies.** List the first ten levels of the circular well with their degeneracy (1 for $m = 0$, 2 otherwise) and compare with the spectrum of a square box of the same area.

Chapter 4 — Spherical Bessel functions and the spherical well

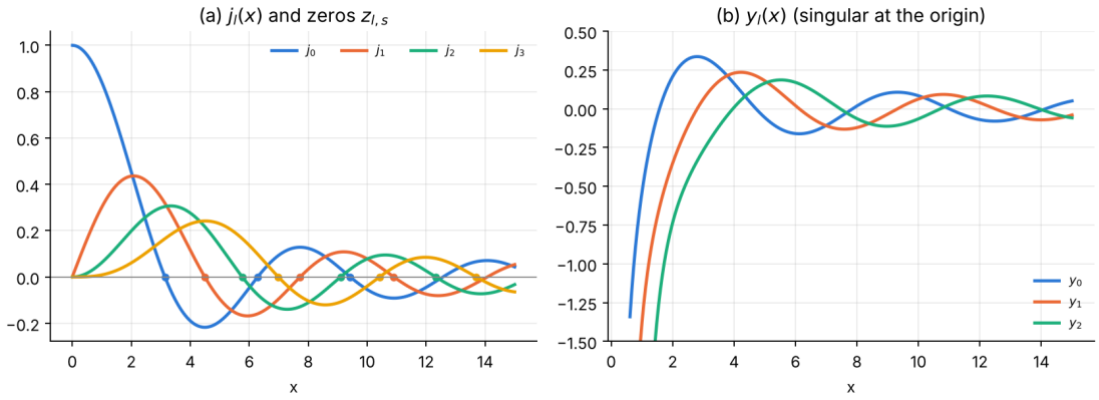
4.1 The function

The spherical Bessel functions solve (DLMF §10.47)

$$x^2 y'' + 2x\,y' + [x^2 - l(l+1)]\,y = 0, \qquad j_l(x) = \sqrt{\frac{\pi}{2x}}\,J_{l+1/2}(x), \quad y_l(x) = \sqrt{\frac{\pi}{2x}}\,Y_{l+1/2}(x).$$

Main properties (DLMF §10.49–§10.53, §10.60; Arfken et al. 2013, §14.7; Messiah 1961, App. B.6):

- Elementary closed forms: $j_0 = \frac{\sin x}{x}$, $j_1 = \frac{\sin x}{x^2} - \frac{\cos x}{x}$, $y_0 = -\frac{\cos x}{x}$, $y_1 = -\frac{\cos x}{x^2} - \frac{\sin x}{x}$; in general (Rayleigh) $j_l = (-x)^l (1/x\,d/dx)^l \sin x/x$.
- Recurrence (for both j_l and y_l): $f_{l+1} = \frac{2l+1}{x} f_l - f_{l-1}$; derivative $j_l' = j_{l-1} - l + 1/x\,j_l$.
- Small-x series: $j_l(x) = \frac{x^l}{(2l+1)!!}\left[1 - \frac{x^2}{2(2l+3)} + \cdots\right]$.
- Cross product: $j_l(x)\,y_{l-1}(x) - j_{l-1}(x)\,y_l(x) = x^{-2}$ (DLMF §10.50).
- Plane-wave expansion: $e^{ikz} = \sum_l i^l\,(2l+1) j_l(kr) P_l(\cos\theta)$ (DLMF §10.60) — the basis of partial-wave scattering theory.
- Zeros $z_{l,s}$ of j_l: $z_{0,s} = s\pi$; they interlace and determine the spectrum of the spherical well.



j_l with its first zeros, and the singular y_l.

4.2 The physics: spherical infinite well

For $V = 0$ inside a sphere of radius 1 and an infinite wall at $r = 1$, $\psi = R(r)Y_{lm}$, $u = rR$:

$$u'' + \left[2E - \frac{l(l+1)}{r^2}\right] u = 0, \qquad u(0) = u(1) = 0, \qquad E_{l,s} = \frac{z_{l,s}^2}{2}, \qquad R = \frac{\sqrt{2}\,j_l(z_{l,s}r)}{\left|j_{l+1}(z_{l,s})\right|}.$$

Each level is $(2l+1)$-fold degenerate. This is the simplest model of a nucleus or a quantum dot: the level order $1s, 1p, 1d, 2s, 1f, 2p, 1g, \ldots$ and the cumulative occupancies 2,8,18,20,34,40,58,68,90,92, … reproduce the first nuclear magic numbers 2, 8, 20 and, after the spin–orbit splitting is added, 28, 50, 82, 126 (Bohr & Mottelson 1969, vol. I, §2-4).

4.3 Analytical scheme

Part A.

1. Upward recurrence from the closed forms — **stable for** y_l, **unstable for** j_l **when** $l > x$ (j_l is minimal).
2. **Miller's algorithm for** j_l: start at $M = l + x + 20 + \sqrt{40(l+x)}$, recur downwards, normalise with $j_0 = \sin x/x$ (or with j_1 where j_0 is near a zero).
3. For $x < 10^{-3}$: the series — the closed forms suffer catastrophic cancellation (e.g. j_3 is a difference of terms of size x^{-4} whose result is $x^3/105$).
4. Zeros: scan with step 0.1, then Newton.

Part B. Numerov for $u = rR$ with Frobenius series $u = r^{l+1}\sum a_k r^{2k}$, $2k(2k+2l+1)a_k = -2Ea_{k-1}$, evaluated at the first nodes (here u is analytic, so this is sufficient), compared with the naive start $u_1 = h^{l+1}$; outward shooting and bisection as in Chapter 3.

4.4 Logic chart

See the logic chart of Section 3.4: the program is identical except for k^2, the Frobenius recurrence and the exact solution.

4.5 Python 3 code

File: `ch4_spherical_bessel_well.py`

```
#!/usr/bin/env python3
"""
Chapter 4 -- SPHERICAL BESSEL FUNCTIONS and the SPHERICAL INFINITE WELL
======================================================================
Part A : computation of j_l(x), y_l(x)
         * closed forms j_0 = sin x / x, j_1, y_0, y_1           DLMF §10.49
         * upward recurrence f_{l+1} = (2l+1)/x f_l - f_{l-1}   DLMF §10.51
           (stable for y_l, UNSTABLE for j_l when l > x)
         * Miller downward recurrence for j_l, normalised by j_0 = sin x / x
         * small-x series  j_l ~ x^l/(2l+1)!! (1 - x^2/(2(2l+3)) + ...)  DLMF §10.53
         * zeros z_{l,s}: bisection bracket + Newton, j_l' = j_{l-1} - (l+1) j_l / x
         * verification: cross product j_l y_{l-1} - j_{l-1} y_l = x^{-2} (DLMF §10.50),
           mpmath reference, scipy spherical_jn
Part B : particle in a spherical infinite well of radius 1 (hbar = m = 1)
         psi = R(r) Y_lm,  u = r R :
         u'' + [ 2E - l(l+1)/r^2 ] u = 0 ,  u(1) = 0 ,  u ~ r^{l+1}  (analytic!)
         exact  E_{l,s} = z_{l,s}^2/2,  R = sqrt(2) j_l(z r)/|j_{l+1}(z)|
         Numerov outward shooting, bisection on u(1)
         * verification: normalisation, comparison with j_l, orthogonality, <r^2>
         * error analysis: here the first-node start is harmless (u is analytic),
           in contrast with Chapters 2-3; h^4 law, Richardson
Outputs: tables + ch4_functions.png, ch4_partA_errors.png, ch4_well.png, ch4_convergence.png
"""
import numpy as np
import mpmath as mp
from scipy.special import spherical_jn
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#7a5cd6"]
mp.mp.dps = 50


def style(ax):
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")


def ref_j(l, x):
    return float(mp.sqrt(mp.pi / (2 * mp.mpf(x))) * mp.besselj(l + mp.mpf(1) / 2, x))
```

```
def ref_y(l, x):
    return float(mp.sqrt(mp.pi / (2 * mp.mpf(x))) * mp.bessely(l + mp.mpf(1) / 2, x))


# ============================================================ PART A
def j_up(L, x):
    j = np.zeros(L + 1); j[0] = np.sin(x) / x
    if L >= 1: j[1] = np.sin(x) / x ** 2 - np.cos(x) / x
    for l in range(1, L):
        j[l + 1] = (2 * l + 1) / x * j[l] - j[l - 1]
    return j


def y_up(L, x):
    y = np.zeros(L + 1); y[0] = -np.cos(x) / x
    if L >= 1: y[1] = -np.cos(x) / x ** 2 - np.sin(x) / x
    for l in range(1, L):
        y[l + 1] = (2 * l + 1) / x * y[l] - y[l - 1]
    return y


def j_series(l, x, nterm=30):
    df = np.prod(np.arange(1, 2 * l + 2, 2, dtype=float))
    s, t = 0.0, 1.0
    for k in range(nterm):
        s += t
        t *= -x * x / (2 * (k + 1) * (2 * l + 2 * k + 3))
    return x ** l / df * s


def j_miller(L, x, M=None):
    """downward recurrence from M, normalised with j_0 = sin x / x"""
    if x < 1e-3:                                        # tiny x: series is exact to round-off
        return np.array([j_series(l, x) for l in range(L + 1)])
    if M is None:
        M = L + int(x) + 20 + int(np.sqrt(40 * (L + x)))
    f = np.zeros(M + 2); f[M] = 1e-30
    for l in range(M, 0, -1):
        f[l - 1] = (2 * l + 1) / x * f[l] - f[l + 1]
        if abs(f[l - 1]) > 1e250: f[l - 1:] *= 1e-250
    j0 = np.sin(x) / x
    # normalise with the larger of j_0, j_1 (j_0 vanishes at x = k pi)
    j1 = np.sin(x) / x ** 2 - np.cos(x) / x
    scale = j0 / f[0] if abs(j0) > abs(j1) else j1 / f[1]
    return f[:L + 1] * scale


def jl(l, x):
    return j_miller(l + 1, x)[l]


def jl_zeros(l, S):
    """first S zeros: bracket by scanning (step 0.1), then Newton"""
    zs = []
    x = l + 0.5 if l > 0 else 1.0
    prev = jl(l, x)
    while len(zs) < S:
        x2 = x + 0.1; cur = jl(l, x2)
        if np.sign(cur) != np.sign(prev):
            z = 0.5 * (x + x2)
            for _ in range(50):
                jj = j_miller(l + 1, z)
                d = (jj[l - 1] - (l + 1) / z * jj[l]) if l > 0 else -jj[1]
                dz = jj[l] / d; z -= dz
                if abs(dz) < 1e-15 * z: break
            zs.append(z)
        x, prev = x2, cur
    return np.array(zs)


def partA():
    print("=" * 78)
    print("PART A  Spherical Bessel functions j_l, y_l")
    print("=" * 78)
    print(f"{'l':>3} {'x':>6} {'j_l Miller':>24} {'mpmath':>24} {'rel.err Miller':>15} {'rel.err upward':>15}")
    for l, x in ((2, 5.0), (5, 1.0), (10, 2.0), (20, 30.0), (30, 1.0), (3, 1e-4)):
        ref = ref_j(l, x); jm = jl(l, x); ju = j_up(l, x)[l]
        print(f"{l:3d} {x:6.0e} {jm:24.15e} {ref:24.15e} {abs(jm-ref)/abs(ref):15.2e} {abs(ju-
ref)/abs(ref):15.2e}")
    print("\ny_l by upward recurrence (stable):")
    for l, x in ((5, 1.0), (20, 3.0), (30, 1.0)):
```

```
        ref = ref_y(l, x); yu = y_up(l, x)[l]
        print(f"  l={l:2d} x={x}: y_l = {yu:.15e}  mpmath {ref:.15e}  rel.err {abs(yu-ref)/abs(ref):.1e}")
    print("\nCross product  j_l y_{l-1} - j_{l-1} y_l - 1/x^2  (independent check, DLMF §10.50):")
    for x in (0.5, 3.0, 20.0):
        jj = j_miller(25, x); yy = y_up(25, x)
        cp = max(abs((jj[l] * yy[l - 1] - jj[l - 1] * yy[l]) * x * x - 1) for l in range(1, 26))
        print(f"  x={x:5.1f}: max over l=1..25 of |x^2 (j_l y_(l-1) - j_(l-1) y_l) - 1| = {cp:.1e}")
    xs = np.linspace(0.05, 40, 80)
    print(f"  max |j_miller - scipy spherical_jn| (l<=15, 0.05<x<40) = "
          f"{max(np.max(abs(j_miller(15, x) - spherical_jn(np.arange(16), x))) for x in xs):.1e}")
    z = {l: jl_zeros(l, 4) for l in range(4)}
    for l in range(4):
        refz = [float(mp.findroot(lambda t: mp.sqrt(mp.pi/(2*t))*mp.besselj(l+0.5, t), v)) for v in z[l]]
        print(f"  zeros j_{l}: " + ", ".join(f"{v:.12f}" for v in z[l]) + f"   max err vs mpmath {max(abs(z[l]-
refz)):.1e}")

    # error data
    ls = np.arange(0, 31)
    ref1 = np.array([ref_j(l, 1.0) for l in ls])
    e_up = abs(j_up(30, 1.0) - ref1) / abs(ref1)
    e_dn = abs(j_miller(30, 1.0) - ref1) / abs(ref1)
    xx = np.logspace(-6, 0, 120)
    ref3 = np.array([ref_j(3, x) for x in xx])
    j3_closed = (15 / xx ** 4 - 6 / xx ** 2) * np.sin(xx) - (15 / xx ** 3 - 1 / xx) * np.cos(xx)  # DLMF §10.49
    e_cl = abs(j3_closed / xx * 1 - ref3) / abs(ref3)
    e_cl = abs(((15 / xx ** 3 - 6 / xx) * np.sin(xx) - (15 / xx ** 2 - 1) * np.cos(xx)) / xx - ref3) / abs(ref3)
    e_mi = np.array([abs(jl(3, x) - r) / abs(r) for x, r in zip(xx, ref3)])
    print("\nj_3 closed trigonometric form vs Miller at small x (relative error):")
    for x in (1e-1, 1e-2, 1e-3, 1e-4):
        i = np.argmin(abs(xx - x))
        print(f"  x={xx[i]:.0e}: closed form {e_cl[i]:.1e}   Miller/series {e_mi[i]:.1e}")

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    xg = np.linspace(0.01, 15, 500)
    for l in range(4):
        ax[0].plot(xg, [jl(l, v) for v in xg], color=C[l], lw=2, label=f"$j_{l}$")
        ax[0].plot(z[l][:3], 0 * z[l][:3], "o", ms=4, color=C[l])
    ax[0].axhline(0, color="#888888", lw=.8)
    ax[0].set(xlabel="x", title=r"(a) $j_l(x)$ and zeros $z_{l,s}$")
    ax[0].legend(frameon=False, fontsize=8, ncol=4)
    for l in range(3):
        ax[1].plot(xg[20:], [y_up(l, v)[l] for v in xg[20:]], color=C[l], lw=2, label=f"$y_{l}$")
    ax[1].set(ylim=(-1.5, 0.5), xlabel="x", title=r"(b) $y_l(x)$ (singular at the origin)")
    ax[1].legend(frameon=False, fontsize=8)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch4_functions.png", dpi=170)

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].semilogy(ls, e_up + 1e-18, "s-", color=C[1], lw=1.5, ms=4, label="upward recurrence")
    ax[0].semilogy(ls, e_dn + 1e-18, "o-", color=C[0], lw=1.5, ms=4, label="downward (Miller)")
    ax[0].set(xlabel="l", ylabel="relative error", title=r"(a) $j_l(1)$: direction of recurrence")
    ax[0].legend(frameon=False, fontsize=8)
    ax[1].loglog(xx, e_cl + 1e-18, color=C[1], lw=1.5, label="closed trigonometric form")
    ax[1].loglog(xx, e_mi + 1e-18, color=C[0], lw=1.5, label="Miller / series")
    ax[1].set(xlabel="x", ylabel="relative error", title=r"(b) $j_3(x)$ for small x: cancellation")
    ax[1].legend(frameon=False, fontsize=8)
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch4_partA_errors.png", dpi=170)
    return z


# ============================================================ PART B
def frobenius(r, l, E, nterm=40):
    """u = r^{l+1} sum a_k r^{2k},  2k(2k+2l+1) a_k = -2E a_{k-1}"""
    r = np.asarray(r, float); s = np.zeros_like(r); a = 1.0; p = np.ones_like(r)
    for k in range(nterm):
        s += a * p
        a = -2 * E * a / ((2 * k + 2) * (2 * k + 2 * l + 3)); p = p * r * r
    return r ** (l + 1) * s


def simpson(y, h):
    return h / 3 * (y[0] + y[-1] + 4 * y[1:-1:2].sum() + 2 * y[2:-1:2].sum())


def shoot(E, l, N, rs, naive=False):
    h = 1.0 / N; r = h * np.arange(N + 1)
    k2 = np.zeros(N + 1); k2[1:] = 2 * E - l * (l + 1) / r[1:] ** 2
    c = 1 + h * h * k2 / 12
    if naive:                                   # u0 = 0, u1 = h^{l+1}, c0 u0 = 0
        js = 0; u = np.zeros(N + 1); u[1] = h ** (l + 1)
    else:
```

```
        js = max(1, int(rs / h + 0.5))
        u = np.zeros(N + 1); u[:js + 2] = frobenius(r[:js + 2], l, E)
    for n in range(js + 1, N):
        u[n + 1] = ((12 - 10 * c[n]) * u[n] - c[n - 1] * u[n - 1]) / c[n + 1]
    return u, r


def eigen(l, s, N, z, rs=0.0, naive=False):
    Ex = z[l][s - 1] ** 2 / 2
    a, b = Ex * 0.85, Ex * 1.15
    fa = shoot(a, l, N, rs, naive)[0][-1]
    for _ in range(64):
        mid = 0.5 * (a + b); fm = shoot(mid, l, N, rs, naive)[0][-1]
        if np.sign(fm) == np.sign(fa): a, fa = mid, fm
        else: b = mid
    E = 0.5 * (a + b)
    u, r = shoot(E, l, N, rs, naive)
    u /= np.sqrt(simpson(u * u, r[1]))
    zz = z[l][s - 1]
    ex = r * np.array([jl(l, zz * v) if v > 0 else (1.0 if l == 0 else 0.0) * zz for v in r]) * np.sqrt(2) /
abs(jl(l + 1, zz))
    ex[0] = 0.0
    if np.dot(u, ex) < 0: u = -u
    return E, u, r, ex


def r2_exact(l, z):
    """<r^2> = int_0^1 j_l(zr)^2 r^4 dr / int_0^1 j_l(zr)^2 r^2 dr, by 50-digit quadrature"""
    f = lambda t: mp.besselj(l + mp.mpf(1) / 2, z * t) ** 2 / t       # = (2 z/pi) j_l^2 * t^... common factor
    num = mp.quad(lambda t: f(t) * t ** 4, [0, 1]); den = mp.quad(lambda t: f(t) * t ** 2, [0, 1])
    return float(num / den)


def order(hs, e):
    """observed order, fitted only on the range where the error still drops (>6x per halving)"""
    idx = [0]
    for i in range(len(e) - 1):
        if e[i] / e[i + 1] > 6: idx.append(i + 1)
        else: break
    idx = idx[1:] if len(idx) > 3 else idx
    return np.polyfit(np.log(np.array(hs)[idx]), np.log(np.array(e)[idx]), 1)[0]


def partB(z):
    print("\n" + "=" * 78)
    print("PART B  Spherical infinite well, radius 1: Numerov in u = r R, N = 400, Frobenius start at first
nodes")
    print("=" * 78)
    Eg = np.linspace(0.5, 80, 800)
    F = np.array([shoot(E, 1, 400, 0.0)[0][-1] for E in Eg])
    roots = [0.5 * (Eg[i] + Eg[i + 1]) for i in np.where(np.sign(F[:-1]) != np.sign(F[1:]))[0]]
    print("  scan l=1: sign changes of u(1;E) near " + ", ".join(f"{v:.2f}" for v in roots)
          + "  (exact " + ", ".join(f"{v**2/2:.2f}" for v in z[1][:4]) + ")")
    print(f"\n{'l':>2} {'s':>2} {'E Numerov':>17} {'z^2/2 (Part A)':>17} {'|dE|':>9} {'norm':>15} {'max|u-
u_ex|':>12} {'<r^2>':>9} {'exact':>9}")
    store = {}
    for (l, s) in ((0, 1), (0, 2), (1, 1), (1, 2), (2, 1), (3, 1)):
        E, u, r, ex = eigen(l, s, 400, z)
        Ex = z[l][s - 1] ** 2 / 2
        zz = z[l][s - 1]
        store[l, s] = (u, r, ex)
        print(f"{l:2d} {s:2d} {E:17.12f} {Ex:17.12f} {abs(E-Ex):9.1e} {simpson(u*u,r[1]):15.12f} {np.max(abs(u-
ex)):12.1e} "
              f"{simpson(u*u*r*r,r[1]):9.6f} {r2_exact(l, zz):9.6f}")
    ov = max(abs(simpson(store[0, 1][0] * store[0, 2][0], 1 / 400)), abs(simpson(store[1, 1][0] * store[1, 2][0],
1 / 400)))
    print(f"  orthogonality (same l, different s): max |<u|u'>| = {ov:.1e}")

    Ns = np.array([20, 40, 80, 160, 320, 640, 1280])
    hs = 1.0 / Ns
    err = {}
    for l in (0, 1, 2):
        Ex = z[l][0] ** 2 / 2
        err[l, "naive"] = [abs(eigen(l, 1, N, z, naive=True)[0] - Ex) for N in Ns]
        err[l, "Frobenius"] = [abs(eigen(l, 1, N, z)[0] - Ex) for N in Ns]
    print("\nError of E_{l,1} vs h: naive start (u0=0, u1=h^(l+1))  vs  Frobenius values at the first nodes")
    print(f"{'N':>6} {'h':>9}" + "".join(f" {'l=%d naive' % l:>12} {'l=%d Frob.' % l:>11}" for l in (0, 1, 2)))
    for i, N in enumerate(Ns):
        print(f"{N:6d} {hs[i]:9.6f}" + "".join(f" {err[l,'naive'][i]:12.2e} {err[l,'Frobenius'][i]:11.2e}" for l
in (0, 1, 2)))
    for l in (0, 1, 2):
        for lab in ("naive", "Frobenius"):
```

```
            print(f"  l={l} {lab:10s}: observed order {order(hs, err[l, lab]):.2f}")
    E1 = eigen(1, 1, 80, z)[0]; E2 = eigen(1, 1, 160, z)[0]; ER = (16 * E2 - E1) / 15
    print(f"Richardson l=1: E(1/80)={E1:.12f} E(1/160)={E2:.12f} E_R={ER:.12f} err={abs(ER-z[1][0]**2/2):.1e}")

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.9))
    for i, (l, s) in enumerate(((0, 1), (0, 2), (1, 1), (2, 1), (3, 1))):
        u, r, ex = store[l, s]
        ax[0].plot(r, u, color=C[i], lw=2, label=f"l={l}, s={s}")
        ax[0].plot(r[::16], ex[::16], "o", ms=2.5, color="#333333")
    ax[0].set(xlabel="r", ylabel="u(r) = r R(r)", title=r"(a) Numerov vs $\sqrt{2}\,r\,j_l(zr)/|j_{l+1}(z)|$
(dots)")
    ax[0].legend(frameon=False, fontsize=7, ncol=2)
    lev = sorted([(z[l][s] ** 2 / 2, l, s + 1) for l in range(4) for s in range(3) if z[l][s] ** 2 / 2 < 60])
    for E, l, s in lev:
        ax[1].hlines(E, l - 0.3, l + 0.3, color=C[l], lw=2.5)
        ax[1].text(l + 0.33, E, f"{s}{'spdf'[l]}", fontsize=7, va="center")
    ax[1].set(xticks=range(4), xticklabels=["s (l=0)", "p (l=1)", "d (l=2)", "f (l=3)"], ylabel="E",
              title="(b) Level scheme $E=z_{l,s}^2/2$ (cf. nuclear shell model)")
    for a_ in ax: style(a_)
    fig.tight_layout(); fig.savefig("ch4_well.png", dpi=170)

    fig, ax = plt.subplots(figsize=(6.6, 5.4))
    for i, l in enumerate((0, 1, 2)):
        ax.loglog(hs, err[l, "naive"], "s--", color=C[i], lw=1.5, ms=5, label=f"l={l}: naive start
$u_1=h^{{l+1}}$")
        ax.loglog(hs, err[l, "Frobenius"], "o-", color=C[i], lw=2, ms=5, label=f"l={l}: Frobenius values at first
nodes")
    ax.loglog(hs, err[0, "Frobenius"][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax.loglog(hs, err[1, "naive"][0] * (hs / hs[0]) ** 3, "-.", color="#888888", label=r"$\propto h^3$")
    ax.set(xlabel="h = 1/N", ylabel=r"$|E_{1,1}-z_{1,1}^2/2|$", title="Spherical well: analytic u = r R needs no
special start\n(only the naive $u_1$ for l = 1 is wrong)")
    ax.legend(frameon=False, fontsize=7, loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=2); style(ax)
    fig.tight_layout(); fig.savefig("ch4_convergence.png", dpi=170)


if __name__ == "__main__":
    z = partA(); partB(z)
```

4.6 Fortran 2008 code

File: `ch4_spherical_bessel_well.f90`

```
!=======================================================================
! Chapter 4 -- SPHERICAL BESSEL FUNCTIONS and the SPHERICAL WELL (F2008)
! Part A: j_l by Miller downward recurrence (normalised by j_0 or j_1),
!         y_l by the stable upward recurrence, small-x series, zeros;
!         checks: cross product x^2 (j_l y_{l-1} - j_{l-1} y_l) = 1 and
!         j_l(x) = sqrt(pi/2x) J_{l+1/2}(x) for l = 0 (sin x / x)
! Part B: u = r R,  u'' + [2E - l(l+1)/r^2] u = 0,  u(1) = 0
!         Numerov + bisection; naive start vs Frobenius values
! Output: screen tables + ch4_f_conv.dat
! Compile: gfortran -O2 -o ch4 ch4_spherical_bessel_well.f90 && ./ch4
!=======================================================================
module sph_mod
  implicit none
  integer,  parameter :: dp = kind(1.0d0)
  real(dp), parameter :: pi = 3.141592653589793238_dp
contains
  real(dp) function j_series(l, x)
    integer,  intent(in) :: l
    real(dp), intent(in) :: x
    real(dp) :: df, t
    integer  :: k
    df = 1.0_dp
    do k = 1, 2*l+1, 2
       df = df*k
    end do
    j_series = 0.0_dp; t = 1.0_dp
    do k = 0, 29
       j_series = j_series + t
       t = -t*x*x/(2*(k + 1)*(2*l + 2*k + 3))
    end do
    j_series = x**l/df*j_series
  end function j_series

  subroutine j_miller(L, x, jv)
    integer,  intent(in)  :: L
    real(dp), intent(in)  :: x
    real(dp), intent(out) :: jv(0:L)
    real(dp), allocatable :: f(:)
```

```
    real(dp) :: j0, j1, scale
    integer  :: M, k
    if (x < 1.0d-3) then
       do k = 0, L
          jv(k) = j_series(k, x)
       end do
       return
    end if
    M = L + int(x) + 20 + int(sqrt(40*(L + x)))
    allocate(f(0:M+1)); f = 0.0_dp; f(M) = 1.0d-30
    do k = M, 1, -1
       f(k-1) = (2*k + 1)/x*f(k) - f(k+1)
       if (abs(f(k-1)) > 1.0d250) f(k-1:M+1) = f(k-1:M+1)*1.0d-250
    end do
    j0 = sin(x)/x; j1 = sin(x)/x**2 - cos(x)/x
    if (abs(j0) > abs(j1)) then
       scale = j0/f(0)
    else
       scale = j1/f(1)
    end if
    jv = f(0:L)*scale
  end subroutine j_miller

  subroutine y_up(L, x, yv)
    integer,  intent(in)  :: L
    real(dp), intent(in)  :: x
    real(dp), intent(out) :: yv(0:L)
    integer :: k
    yv(0) = -cos(x)/x
    if (L >= 1) yv(1) = -cos(x)/x**2 - sin(x)/x
    do k = 1, L-1
       yv(k+1) = (2*k + 1)/x*yv(k) - yv(k-1)
    end do
  end subroutine y_up

  subroutine j_up(L, x, jv)
    integer,  intent(in)  :: L
    real(dp), intent(in)  :: x
    real(dp), intent(out) :: jv(0:L)
    integer :: k
    jv(0) = sin(x)/x
    if (L >= 1) jv(1) = sin(x)/x**2 - cos(x)/x
    do k = 1, L-1
       jv(k+1) = (2*k + 1)/x*jv(k) - jv(k-1)
    end do
  end subroutine j_up

  real(dp) function jl(l, x)
    integer,  intent(in) :: l
    real(dp), intent(in) :: x
    real(dp) :: jv(0:l+1)
    call j_miller(l+1, x, jv); jl = jv(l)
  end function jl

  subroutine jl_zeros(l, S, z)
    integer,  intent(in)  :: l, S
    real(dp), intent(out) :: z(S)
    real(dp) :: x, x2, prev, cur, zz, dz, d, jv(0:l+1)
    integer  :: found, it
    found = 0
    x = merge(l + 0.5_dp, 1.0_dp, l > 0); prev = jl(l, x)
    do while (found < S)
       x2 = x + 0.1_dp; cur = jl(l, x2)
       if (sign(1.0_dp, cur) /= sign(1.0_dp, prev)) then
          zz = 0.5_dp*(x + x2)
          do it = 1, 50
             call j_miller(l+1, zz, jv)
             if (l > 0) then
                d = jv(l-1) - (l + 1)/zz*jv(l)
             else
                d = -jv(1)
             end if
             dz = jv(l)/d; zz = zz - dz
             if (abs(dz) < 1.0d-15*zz) exit
          end do
          found = found + 1; z(found) = zz
       end if
       x = x2; prev = cur
    end do
  end subroutine jl_zeros
end module sph_mod
```

```
module swell_mod
  use sph_mod
  implicit none
contains
  real(dp) function frob(r, l, E)
    real(dp), intent(in) :: r, E
    integer,  intent(in) :: l
    real(dp) :: a, s, p
    integer  :: k
    s = 0.0_dp; a = 1.0_dp; p = 1.0_dp
    do k = 0, 39
       s = s + a*p
       a = -2*E*a/((2*k + 2)*(2*k + 2*l + 3)); p = p*r*r
    end do
    frob = r**(l+1)*s
  end function frob

  real(dp) function simpson(y, h, n)
    integer,  intent(in) :: n
    real(dp), intent(in) :: y(0:n), h
    simpson = h/3*(y(0) + y(n) + 4*sum(y(1:n-1:2)) + 2*sum(y(2:n-2:2)))
  end function simpson

  subroutine shoot(E, l, N, naive, u)
    real(dp), intent(in)  :: E
    integer,  intent(in)  :: l, N
    logical,  intent(in)  :: naive
    real(dp), intent(out) :: u(0:N)
    real(dp) :: h, r(0:N), c(0:N)
    integer  :: j, js
    h = 1.0_dp/N; r = [(j*h, j=0,N)]
    c(0) = 1.0_dp; c(1:N) = 1 + h*h*(2*E - l*(l+1)/r(1:N)**2)/12
    u = 0.0_dp
    if (naive) then
       js = 0; u(1) = h**(l+1)
    else
       js = 1; u(0) = 0.0_dp; u(1) = frob(r(1), l, E); u(2) = frob(r(2), l, E)
    end if
    do j = js+1, N-1
       u(j+1) = ((12 - 10*c(j))*u(j) - c(j-1)*u(j-1))/c(j+1)
    end do
  end subroutine shoot

  subroutine eigen(l, Eex, N, naive, E, u)
    integer,  intent(in)  :: l, N
    real(dp), intent(in)  :: Eex
    logical,  intent(in)  :: naive
    real(dp), intent(out) :: E, u(0:N)
    real(dp) :: a, b, fa, mid
    integer  :: it
    a = 0.85_dp*Eex; b = 1.15_dp*Eex
    call shoot(a, l, N, naive, u); fa = u(N)
    do it = 1, 64
       mid = 0.5_dp*(a + b)
       call shoot(mid, l, N, naive, u)
       if (sign(1.0_dp, u(N)) == sign(1.0_dp, fa)) then
          a = mid; fa = u(N)
       else
          b = mid
       end if
    end do
    E = 0.5_dp*(a + b)
    call shoot(E, l, N, naive, u)
    u = u/sqrt(simpson(u*u, 1.0_dp/N, N))
  end subroutine eigen
end module swell_mod

program ch4
  use sph_mod
  use swell_mod
  implicit none
  integer, parameter :: NN = 400
  real(dp) :: jv(0:30), ju(0:30), yv(0:30), z(0:3,4), x, cp, E, u(0:NN), uex(0:NN), Eex, e1(7), e2(7), r
  real(dp) :: xc(3) = [0.5_dp, 3.0_dp, 20.0_dp]
  real(dp), allocatable :: uu(:)
  integer  :: i, k, l, s, Ns(7) = [20, 40, 80, 160, 320, 640, 1280]
  integer  :: llist(6) = [0,0,1,1,2,3], slist(6) = [1,2,1,2,1,1]

  print '(a)', repeat('=',78)
  print '(a)', 'PART A  Spherical Bessel functions (Fortran)'
  print '(a)', repeat('=',78)
  call j_miller(30, 1.0_dp, jv); call j_up(30, 1.0_dp, ju)
```

```
  print '(a)', ' j_l(1): upward vs Miller (relative difference)'
  do l = 5, 30, 5
     print '(a,i3,a,es24.15,a,es10.2)', '   l=', l, '  Miller=', jv(l), '  rel.diff upward=', abs(ju(l)/jv(l) - 1)
  end do
  do k = 1, 3
     x = xc(k)
     call j_miller(25, x, jv(0:25)); call y_up(25, x, yv(0:25))
     cp = 0.0_dp
     do l = 1, 25
        cp = max(cp, abs(x*x*(jv(l)*yv(l-1) - jv(l-1)*yv(l)) - 1))
     end do
     print '(a,f5.1,a,es9.1)', ' cross product x=', x, ':  max_l |x^2 (j_l y_{l-1} - j_{l-1} y_l) - 1| = ', cp
  end do
  print '(a,es24.15,a)', ' j_3(1e-4) (series) = ', jl(3, 1.0d-4), '   (exact 9.523809518518520e-15)'
  do l = 0, 3
     call jl_zeros(l, 4, z(l,:))
     print '(a,i1,a,4f16.12)', ' zeros j_', l, ':', z(l,:)
  end do

  print '(/a)', repeat('=',78)
  print '(a)', 'PART B  Spherical infinite well, Numerov, N = 400'
  print '(a)', repeat('=',78)
  print '(a2,a3,a17,a17,a10,a16,a12)', 'l', 's', 'E Numerov', 'z^2/2', '|dE|', 'norm', 'max|u-ue|'
  do k = 1, 6
     l = llist(k); s = slist(k); Eex = z(l,s)**2/2
     call eigen(l, Eex, NN, .false., E, u)
     uex(0) = 0.0_dp
     do i = 1, NN
        r = i/real(NN,dp)
        uex(i) = r*jl(l, z(l,s)*r)*sqrt(2.0_dp)/abs(jl(l+1, z(l,s)))
     end do
     if (sum(u*uex) < 0) u = -u
     print '(i2,i3,2f17.12,es10.1,f16.12,es12.1)', l, s, E, Eex, abs(E - Eex), simpson(u*u, 1.0_dp/NN, NN), &
          maxval(abs(u - uex))
  end do

  open(10, file='ch4_f_conv.dat', status='replace')
  print '(/a)', 'Error of E_{l,1} vs N: naive start | Frobenius values at first nodes'
  do l = 0, 2
     Eex = z(l,1)**2/2
     do i = 1, 7
        allocate(uu(0:Ns(i)))
        call eigen(l, Eex, Ns(i), .true.,  E, uu); e1(i) = abs(E - Eex)
        call eigen(l, Eex, Ns(i), .false., E, uu); e2(i) = abs(E - Eex)
        write(10,'(i3,i6,3es16.7)') l, Ns(i), 1.0_dp/Ns(i), e1(i), e2(i)
        deallocate(uu)
     end do
     print '(a,i1,a,7es9.1)', ' l=', l, ' naive    :', e1
     print '(a,i1,a,7es9.1)', ' l=', l, ' Frobenius:', e2
  end do
  close(10)
end program ch4
```

4.7 Results, normalisation and verification

Part A.

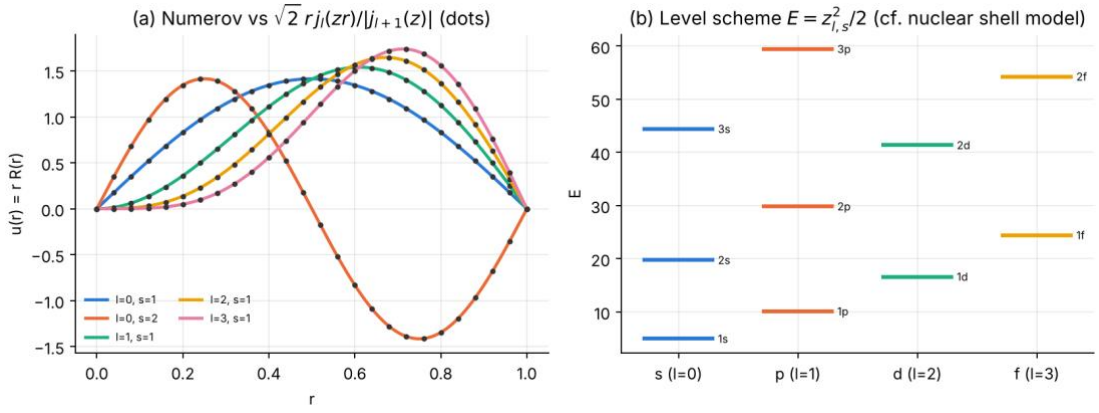
l	x	j_l (Miller)	rel. error Miller	rel. error upward recurrence
2	5	0.1347312100851252	0	0
5	1	9.256115861125814e-05	1.5e-16	4.4e-10
10	2	6.825300864974727e-08	1.9e-16	1.8e-04
20	30	−0.01471159335342906	2.1e-15	5.9e-16
30	1	5.566831266981347e-43	1.4e-16	2.2e+66
3	10^{-4}	9.523809518518520e-15	0	3.5e+11

y_l by upward recurrence is accurate to 3×10^{-16} even for $y_{30}(1) \approx -2.9 \times 10^{40}$. The cross product $x^2(j_l y_{l-1} - j_{l-1} y_l) = 1$ holds to 7×10^{-16} for $l \le 25$ and $x = 0.5, 3, 20$ — an independent check that needs no reference software. Agreement with SciPy `spherical_jn`: 1.4×10^{-15}. Zeros (e.g. $z_{1,1} = 4.493409457909$, $z_{2,1} = 5.763459196895$) agree with mpmath to $\le 2 \times 10^{-15}$.

Part B ($N = 400$, Frobenius start):

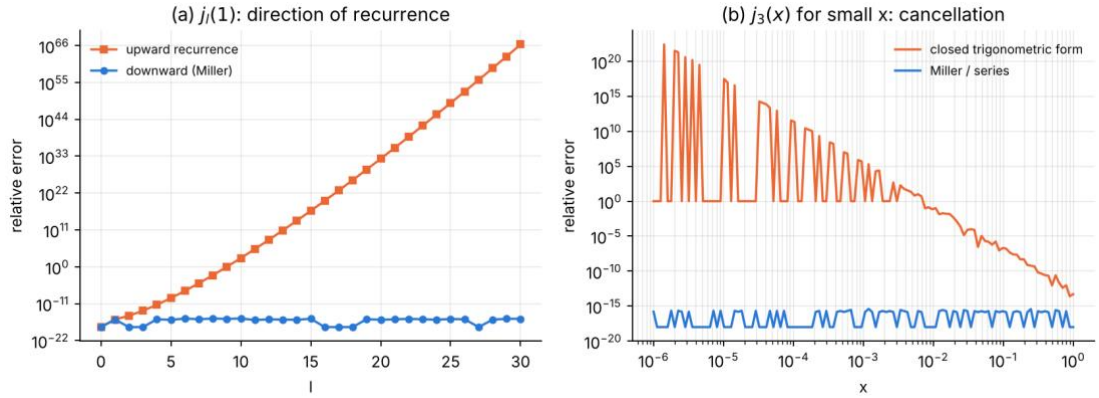
l	s	E Numerov	z²/2 (Part A)	|ΔE|	∫u²dr	max|u − u_ex|	⟨r²⟩	exact
0	1	4.934802200474	4.934802200545	7.1e-11	1.000000000000	5.2e-11	0.282673	0.282673
0	2	19.739208797098	19.739208802179	5.1e-09	1.000000000000	7.6e-12	0.320668	0.320668
1	1	10.095364277786	10.095364278213	4.3e-10	1.000000000000	2.3e-11	0.374606	0.374606
1	2	29.839757959217	29.839757972055	1.3e-08	1.000000000000	3.0e-10	0.347297	0.347297
2	1	16.608730956157	16.608730957134	9.8e-10	1.000000000000	1.1e-10	0.438700	0.438700
3	1	24.415596820069	24.415596821810	1.7e-09	1.000000000000	2.5e-10	0.486924	0.486924

(The exact $\langle r^2 \rangle$ was obtained by 50-digit quadrature of j_l^2.) Orthogonality between states of equal l: 1.5×10^{-11}.



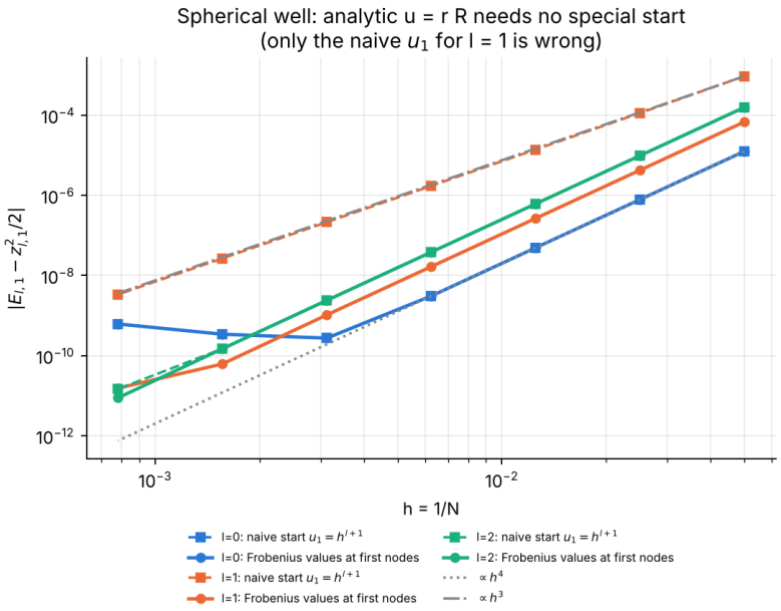
(a) Numerov $u(r)$ (lines) vs $\sqrt{2}\, r\, j_l(zr)/|j_{l+1}(z)|$ (dots). (b) Level scheme of the spherical well — the ordering behind the nuclear shell model.

4.8 Error analysis and convergence



(a) $j_l(1)$: the upward recurrence loses about one digit per step of l once $l > x$; Miller does not. (b) $j_3(x)$ from its closed trigonometric form: relative error grows like x^{-7} for small x (the result is $\sim x^3$, the terms $\sim x^{-4}$).

N	l=0 naive	l=0 Frobenius	l=1 naive	l=1 Frobenius	l=2 naive	l=2 Frobenius
20	1.25e-05	1.25e-05	9.37e-04	6.79e-05	1.57e-04	1.57e-04
40	7.83e-07	7.82e-07	1.13e-04	4.23e-06	9.76e-06	9.76e-06
80	4.89e-08	4.89e-08	1.38e-05	2.64e-07	6.10e-07	6.10e-07
160	3.05e-09	3.05e-09	1.71e-06	1.65e-08	3.81e-08	3.81e-08
320	2.71e-10	2.71e-10	2.13e-07	1.03e-09	2.38e-09	2.38e-09
640	3.40e-10	3.40e-10	2.66e-08	6.15e-11	1.48e-10	1.48e-10
1280	6.13e-10	6.13e-10	3.34e-09	1.53e-11	1.46e-11	8.76e-12
order	**3.85**	**3.85**	**3.01**	**4.01**	**3.91**	**4.01**



Spherical well: because $u = rR$ is analytic, Frobenius values at the first nodes are enough; only the naive start for $l = 1$ (where $lim_{r\to 0} k^2 u \neq 0$) loses one order.

Observations:

- For $l = 0$ the naive and Frobenius starts coincide: with $u_0 = 0$ the second start value only fixes the scale.
- The $l = 0$ error stops decreasing at $\approx 3 \times 10^{-10}$ and then grows: this is the **round-off floor** $\sim \epsilon N^2$ already met in Exercise 1 of the first course (the eigenvalue is determined by the sign of u_N, which is a small difference of large accumulated quantities).
- **Richardson** ($l = 1$): $E(1/80) = 10.095364013952$, $E(1/160) = 10.095364261698$, $E_R = 10.095364278214$, error 7.5×10^{-13} (direct error on the finer grid: 1.7×10^{-8}).
- Compared with Chapter 3: same equation type, same method — but the spherical problem has no start fault because the Liouville transformation $u = rR$ keeps the solution analytic, whereas $u = \sqrt{r}R$ does not.

4.9 Exercises

1. **Rayleigh's formula.** Derive j_2 and y_2 in closed form from Rayleigh's formula, and check the cross product $j_2 y_1 - j_1 y_2 = x^{-2}$ symbolically.
2. **Where does the upward recurrence fail?** Show that the relative error of upward $j_l(x)$ grows roughly like $|y_l(x)/j_l(x)|\,\epsilon$ and compare with the table ($l = 10$, $x = 2$: $|y_{10}/j_{10}| = 5 \times 10^{12}$, so the predicted error is $\approx 5 \times 10^{-4}$; observed 1.8×10^{-4}).
3. **Finite spherical well (deuteron model).** For $V = -V_0$ ($r < 1$), $V_0 = 20$, $l = 0$, derive the condition $q\cot q = -\kappa$, $q = \sqrt{2(V_0 - |E|)}$, $\kappa = \sqrt{2|E|}$, solve it, and reproduce the result with Numerov using the discontinuity treatment of the first course (Problem 2). *Answer:* two bound states, $E = -16.3537992$ and $E = -5.9978397$.
4. **Shell model.** Using the zeros from Part A, order the levels $z_{l,s}$ and compute the cumulative occupancy $\sum 2(2l + 1)$. *Answer:*

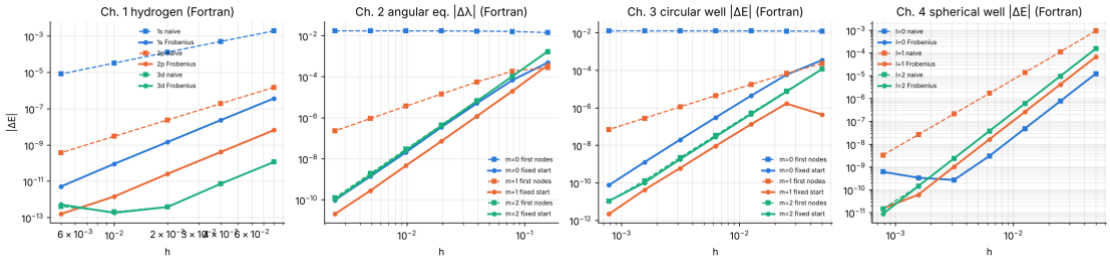
$1s(3.142), 1p(4.493), 1d(5.763), 2s(6.283), 1f(6.988), 2p(7.725), 1g(8.183), 2d(9.095), 1h(9.356), 3s(9.4$; occupancies 2, 8, 18, 20, 34, 40, 58, 68, 90, 92.

5. **Partial-wave check.** Verify numerically $\sum_{l=0}^{L}(2l+1)\,j_l(x)^2 \to 1$ as $L \to \infty$ (DLMF §10.60) for $x = 5$ and $x = 30$; how large must L be? *Answer:* the terms decay rapidly once $l > x$: for $x = 5$ the defect $1 - \sum$ is 2×10^{-7} at $L = 10$ and 10^{-14} at $L = 15$; for $x = 30$ it is 1.3×10^{-4} at $L = 35$, 5×10^{-8} at $L = 40$ and at round-off level for $L = 50$.

Chapter 5 — Cross-check of the Fortran and Python implementations

The four Fortran 2008 programs implement exactly the same algorithms as the Python programs (same recurrences, same Frobenius series, same grids, same start index $j_s = \mathrm{nint}(r_s/h)$, same bisection counts). All tables in Chapters 1–4 are reproduced by the Fortran codes to every printed digit; the only differences are at the round-off level (last digit of numbers below 10^{-12}). In addition, the Fortran versions use independent references that are unavailable in pure Python: the Fortran 2008 intrinsic `bessel_jn` (agreement with Miller's algorithm: 2.8×10^{-16} for $n \le 10$, $0.1 \le x \le 40$), and the exact value $|J_m(j_{m,1})| \le 2 \times 10^{-16}$ at the computed zeros.

The Fortran programs write their convergence data to `ch1_f_conv.dat` … `ch4_f_conv.dat`; `plot_fortran_results.py` produces the following figure.



Convergence plots drawn from the Fortran output files. Left to right: hydrogen (naive vs Frobenius start), angular equation, circular well (start at first nodes vs at a fixed point), spherical well.

Pitfalls met while writing the Fortran versions (worth knowing for any Fortran work):

- Fortran is case-insensitive: a variable `dP` silently shadows the kind parameter `dp`, and `N` and `n`, `Ex` and `ex` are the same name. All three occurred and were caught by the compiler.
- Array constructors cannot be indexed directly (`[1,2,3](i)` is illegal) — use a named array.
- The rounding of $r_s/h = 12.5$ differs between Python's `round` (to even: 12) and Fortran's `nint` (away from zero: 13); the Python codes use `int(x + 0.5)` to match.

Summary

	Laguerre / hydrogen	**Legendre / angular**	**Bessel J_m / circular well**	**spherical Bessel / spherical well**
Stable way to compute the function	forward recurrence	forward (normalised) recurrence	Miller (backward)	Miller for j_l, forward for y_l
What fails	explicit sum (cancellation)	explicit sum; unnormalised overflow	power series; forward recurrence $n > x$	upward j_l; closed forms at small x
Gauss rule / zeros	Gauss–Laguerre (Newton, Golub–Welsch)	Gauss–Legendre	McMahon + Newton	scan + Newton
Liouville variable	$u = rR$ (analytic)	$u = \sqrt{\sin\theta}\,\Theta$ (not analytic)	$u = \sqrt{r}R$ (not analytic)	$u = rR$ (analytic)
Shooting	two-sided, turning point	two-sided, both poles	outward, $u(1) = 0$	outward, $u(1) = 0$
Start fault	naive start: order 2 (s), 3 (p)	first nodes: order $\min(4,2m)$, none for $m = 0$	first nodes: order $\min(4,2m)$	naive start: order 3 for $l = 1$
Cure	Frobenius series	Frobenius up to fixed θ_s	Frobenius up to fixed r_s	Frobenius values
Accuracy reached	10^{-13}	10^{-11}	10^{-11}	10^{-11}

Key messages:

1. **Recurrences are the method of choice — in the stable direction.** Dominant solutions forwards, minimal solutions backwards (Miller). Explicit sums and closed forms are for small arguments only.
2. **Always verify with an identity that does not use the code being tested**: Jacobi–Anger, cross products, sums of weights, orthogonality, virial theorem.
3. **Separation of variables gives singular ODEs.** Numerov keeps its fourth order only if the solution is smooth on the integration interval: start from a Frobenius series at a fixed point, never at the first grid nodes of a non-analytic solution.
4. **A calculation can converge perfectly in h and still be wrong** (angular equation, $m = 0$: error 1.7×10^{-2} for every h). Convergence studies are necessary, not sufficient; comparison with an exact limit is indispensable.
5. Richardson extrapolation and Gauss quadrature are cheap ways to gain digits — provided the error model (h^4, analyticity) actually holds.

References

Reference works on special functions

- [DLMF] F. W. J. Olver, D. W. Lozier, R. F. Boisvert, C. W. Clark et al. (eds.), *NIST Digital Library of Mathematical Functions*, https://dlmf.nist.gov — chapters 3 (numerical methods: §3.5 quadrature, §3.6 linear difference equations and Miller's algorithm), 10 (Bessel functions: §10.2, §10.6, §10.9, §10.12, §10.21 zeros, §10.47–§10.60 spherical Bessel functions, §10.74 methods of computation), 14 (Legendre and related functions: §14.2, §14.7, §14.10, §14.17, §14.30), 18 (orthogonal polynomials: §18.3, §18.5, §18.8, §18.9, §18.39).
- F. W. J. Olver, D. W. Lozier, R. F. Boisvert, C. W. Clark (eds.), *NIST Handbook of Mathematical Functions*, Cambridge University Press (2010).
- M. Abramowitz and I. A. Stegun (eds.), *Handbook of Mathematical Functions*, NBS Applied Mathematics Series 55 (1964); chapters 8 (Legendre), 9–10 (Bessel), 22 (orthogonal polynomials).
- [EoM] "Special functions", *Encyclopedia of Mathematics*, https://encyclopediaofmath.org/wiki/Special_functions.

Textbooks

- G. N. Watson, *A Treatise on the Theory of Bessel Functions*, 2nd ed., Cambridge University Press (1944).
- N. N. Lebedev, *Special Functions and Their Applications*, Prentice-Hall (1965); Dover reprint (1972).
- G. Szegő, *Orthogonal Polynomials*, AMS Colloquium Publications 23 (1939; 4th ed. 1975).
- G. E. Andrews, R. Askey and R. Roy, *Special Functions*, Encyclopedia of Mathematics and its Applications 71, Cambridge University Press (1999).
- N. M. Temme, *Special Functions: An Introduction to the Classical Functions of Mathematical Physics*, Wiley (1996).
- G. B. Arfken, H. J. Weber and F. E. Harris, *Mathematical Methods for Physicists*, 7th ed., Academic Press (2013), ch. 14 (Bessel), 15 (Legendre), 18 (Laguerre and other special functions).

Numerical methods

- A. Gil, J. Segura and N. M. Temme, *Numerical Methods for Special Functions*, SIAM (2007), ISBN 978-0-898716-34-4.
- W. Gautschi, "Computational aspects of three-term recurrence relations", *SIAM Review* **9**, 24–82 (1967), doi:10.1137/1009002.
- J. C. P. Miller, *Bessel Functions, Part II*, British Association Mathematical Tables vol. X, Cambridge University Press (1952) — origin of the backward-recurrence algorithm.
- F. W. J. Olver, "Error analysis of Miller's recurrence algorithm", *Mathematics of Computation* **18**, 65–74 (1964).
- G. H. Golub and J. H. Welsch, "Calculation of Gauss quadrature rules", *Mathematics of Computation* **23**, 221–230 (1969), doi:10.2307/2004418.

- W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, *Numerical Recipes*, 3rd ed., Cambridge University Press (2007), §4.6 (Gaussian quadrature), §6.5 (Bessel functions).
- B. V. Numerov, "A method of extrapolation of perturbations", *Monthly Notices of the Royal Astronomical Society* **84**, 592 (1924).
- J. M. Thijssen, *Computational Physics*, 2nd ed., Cambridge University Press (2007).

Quantum mechanics

- H. A. Bethe and E. E. Salpeter, *Quantum Mechanics of One- and Two-Electron Atoms*, Springer (1957).
- L. D. Landau and E. M. Lifshitz, *Quantum Mechanics (Non-relativistic Theory)*, 3rd ed., Pergamon (1977), §36 and Mathematical Appendices.
- A. Messiah, *Quantum Mechanics*, vol. I, North-Holland (1961), Appendix B (special functions).
- J. J. Sakurai and J. Napolitano, *Modern Quantum Mechanics*, 2nd ed., Cambridge University Press (2017).
- D. J. Griffiths and D. F. Schroeter, *Introduction to Quantum Mechanics*, 3rd ed., Cambridge University Press (2018), ch. 4.
- A. Bohr and B. R. Mottelson, *Nuclear Structure*, vol. I, Benjamin (1969).